

컴퓨팅 사고와 문제해결

황숙희·조미영 지음

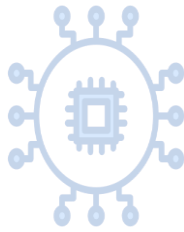
목 차

1부 : 이론

01	컴퓨팅 사고 이해	1
02	컴퓨팅 사고 요소	9
03	튜링기계	18
04	컴퓨터	26
05	데이터의 표현	32
06	저장	38
07	규칙기반 자료구조_스택큐	44
08	규칙기반 자료구조_트리그래프	51
09	알고리즘 성능	58
10	문제의 이해	66
11	컴퓨팅 사고와 문제해결_P문제와 알고리즘	71
12	컴퓨팅 사고와 문제해결_NP문제와 알고리즘	80

2부 : 실습

13	파이썬 소개와 설치	90
14	파이썬 변수, 자료형, 표준입출력	98
15	파이썬 제어문 (선택문, 반복문)	111
16	파이썬 테크닉 1 (함수, 모듈)	117
17	파이썬 테크닉 2 (클래스, 라이브러리)	122



SW중심대학사업 교재공동개발

1. 컴퓨팅 사고 이해

Computational Thinking



0 목차

- 컴퓨팅 사고 시작
- 컴퓨팅 사고 정의
- 컴퓨팅 사고 요소
- 컴퓨팅 사고 교육

1 컴퓨팅 사고 시작

개요

❖ Computational Thinking

- 계산에 관한 사고, 계산적 사고
- 컴퓨팅적 사고, 컴퓨팅 사고 (컴퓨터 사고(X), 계산하는 능력(X))
- 컴퓨터가 일을 처리(계산)할 수 있도록 만드는 인간의 사고 방식

❖ 사고력

- 생각하는 능력, 문제해결 능력
- 교육, 경험을 통해 배양
- 예) “우리나라 교통문제” → “우리나라 버스노선의 특징과 문제 ”

1 컴퓨팅 사고 시작

개요

❖ 컴퓨팅 사고 → 컴퓨팅 사고력

❖ 컴퓨터 교육 → 컴퓨팅 사고력 배양 → 문제해결 능력 높아짐

❖ 컴퓨터 교육 분야에서 나온 개념적 이론

❖ 컴퓨팅 사고 시작

- 1980년 시모어 파퍼트 (Seymour Papert) – 마인드스톰 (Mindstorms)
- 수학자, 컴퓨터 과학자/ 컴퓨터 교육, 인공지능
- 2006년 지넷윙 (Jeannette Wing) – ‘Computational Thinking’

1 컴퓨팅 사고 시작

Jeannette Wing – ‘Computational Thinking’

❖ Jeannette Wing

- 2013년 이전 카네기 멜론 대학 컴퓨터과학 교수
- 2017년 까지 마이크로 소프트 리서치 기업 부사장
- 2021년 컬럼비아 대학 연구 담당 부사장 임명
- 컬럼비아 대학 컴퓨터과학 교수



<지넷 윈 마이크로소프트리서치 기업 부사장>
[사진=MS]

❖ 2006년 ACM(Association for Computing Machinery – 국제 컴퓨터학회)

❖ www.cs.cmu.edu/~./15110-s13/Wing06-ct.pdf

1 컴퓨팅 사고 시작

Jeannette Wing – ‘Computational Thinking’

Viewpoint | Jeannette M. Wing

Computational Thinking
It represents a universally applicable attitude and skill set everyone, not just computer scientists, would be eager to learn and use.

Computational thinking builds on the power and limits of computing processes, whether they are executed by a human or by a machine. Computational methods and models give us the courage to solve problems and design systems that no one of us would be capable of tackling alone. Computational thinking confronts the reality of machine intelligence. What can humans do better than computers? and What can computers do better than humans? Most fundamentally it addresses the question: What is computable? Today, we know only parts of the answers to such questions.

Computational thinking is a fundamental skill for everyone, not just for computer scientists. To reading, writing, and arithmetic, we should add computational thinking to every child's analytical ability. Just as the printing press facilitated the spread of the three Rs, what is appropriately inescapable about this vision is that computing and computers facilitate the spread of computational thinking.

Computational thinking involves asking problems, designing systems, and understanding human behavior, by drawing on the concepts fundamental to computer science. Computational thinking includes a range of mental tools that reflect the breadth of the field of computer science.

Hard to solve a particular problem, we might ask: How difficult is it to solve? and What's the best way to solve it? Computer science rests on solid theoretical underpinnings to answer such questions precisely. Starting the difficulty of a problem accounts for the underlying power of the machine—the computing device that will run the solution. We must consider the machine's instruction set, its resource constraints, and its operating environment.

In solving a problem efficiently, we might further ask whether an approximate solution is good enough, whether we can use randomization to our advantage, and whether false positives or false negatives are allowed. Computational thinking is reformulating a seemingly difficult problem into one we know how to solve, perhaps by reduction, embedding, transformation, or simulation.

Computational thinking is thinking recursively. It is parallel processing. It is interpreting code as data and data as code. It is type checking as the generalization of dimensional analysis. It is recognizing both the virtues and the dangers of abstracting, or giving someone or something more than one name. It is recognizing both the cost and power of indirect addressing and procedure call. It is judging a program not just for correctness and efficiency but for aesthetics, and a system's design for simplicity and elegance.

Computational thinking is using abstraction and decomposition when attacking a large complex task or designing a large complex system. It is separation of concerns. It is choosing an appropriate representation for a problem or modeling the relevant aspects of a problem to make it tractable. It is using intuition to describe a system's behavior succinctly and declaratively. It is having the confidence we can safely use, modify, and influence a large complex system without understanding its every detail. It is

- ❖ What can humans do better than computer? (인간이 컴퓨터보다 나은 것)
- ❖ What can computers do better than humans? (컴퓨터가 인간보다 나은 것)
- ❖ What is computable? (계산가능한 것)
- ❖ ‘컴퓨팅 사고는 컴퓨터 공학의 기본 개념을 기반으로 문제를 해결하고, 시스템을 설계하는 것을 의미한다.’
- ❖ ‘컴퓨팅 사고는 컴퓨터 과학자 뿐만 아니라 모든 사람이 배워서 활용해야 하는 기본적인 사고이며, 기술이다. 읽기, 쓰기, 셈하기를 배우듯이 아이들이 기본적으로 갖춰야 할 기본적인 능력이다.’
- ❖ ‘컴퓨팅 사고는 추상화와 분해를 통해 복잡한 시스템을 설계하거나 어려운 문제를 해결하는 능력이며, 불확실한 상황에서 계획을 짜고, 학습을 하고 일정을 세울 수 있는 능력이다.’

1 컴퓨팅 사고 시작

Jeannette Wing – ‘Computational Thinking’

Viewpoint

modulating something in anticipation of multiple uses or prefetching and caching in anticipation of future use.

Computational thinking is thinking in terms of prevention, protection, and recovery from worst-case scenarios through redundancy, damage containment, and error correction. It is calling a global standard and contracts interfaces. It is learning to avoid race conditions when synchronizing meetings with one another.

Computational thinking is using heuristic reasoning to discover a solution. It is planning, learning, and scheduling in the presence of uncertainty. It is search, search, and more search, resulting in a list of

altery when nondeterminism and garbage collection take on the meanings used by computer scientists; and when trees are drawn upside down.

We have witnessed the influence of computational thinking on other disciplines. For example, machine learning has transformed statistics. Statistical learning is being used for problems on a scale, in terms of both data size and dimension, unimaginable only a few years ago. Statistics departments in all kinds of organizations are hiring computer scientists. Schools of computer science are embracing existing or starting up new statistics departments.

Computer scientists' recent interest in biology is driven by their belief that biologists can benefit

Thinking like a computer scientist means more than being able to program a computer. It requires thinking at multiple levels of abstraction.

Web pages, a strategy for winning a game, or a counterexample. Computational thinking is using massive amounts of data to speed up computation. It is making trade-offs between time and space and between processing power and storage capacity.

Consider these everyday examples. When your daughter goes to school in the morning, she puts in her backpack the things she needs for the day that's prefetching and caching. When your son loses his mittens, you suggest he remove his socks that's backtracking. As what parent do you stop mittens this and buy yourself a pair that's online algorithms. Which line do you wait at the supermarket that's queue management modeling for multi-server systems. Why does your telephone still work during a power outage that's independence of failure and redundancy in design. How do Complexly Automated Public Turing Tests in Tilt Computers and Human Apart, or CAPTCHA, authenticate humans that's exploiting the difficulty of solving hard AI problems to fool computing agents.

Computational thinking will become ingrained in everyone's lives when words like algorithm and pseudocode are part of everyone's vocabulary.

from computational thinking. Computer science's contribution to biology goes beyond the ability to search through vast amounts of sequence data looking for patterns. The hope is that data structures and algorithms—our computational abstractions and methods—can represent the structure of proteins in ways that elucidate their function. Computational biology is changing the way biologists think. Similarly, computational game theory is changing the way economists think, nanocomputing, the way chemists think, and quantum computing, the way physicists think.

This kind of thinking will be part of the skill set of not only other scientists but of everyone else. Ubiquitous computing is to today's computational thinking as tomorrow's ubiquitous computing was yesterday's dream that became today's reality: computational thinking is tomorrow's reality.

WHAT IT IS, AND ISN'T
Computer science is the study of computation—what can be computed and how to compute it. Computational thinking thus has the following characteristics.

- ❖ Thinking like a computer scientist means more than being able to program a computer.

(컴퓨터 과학자처럼 생각한다는 것은 컴퓨터 프로그램을 할 수 있는 것 이상을 의미한다.)

- ❖ 개념화된 컴퓨터 과학자들의 문제해결 기술

- 문제해결 방식 – 재귀적 사고 / 병렬처리
- 문제의 크기 – 추상화 / 분해
- 문제해결 과정 – 예측 / 보호 / 보완
- 문제 정의 – 경험에 의한 추리 – 문제 분석 / 설계

1 컴퓨팅 사고 시작

Jeannette Wing – ‘Computational Thinking’

Conceptualizing, not programming. Computer science is not computer programming. Thinking like a computer scientist means more than being able to program a computer. It requires thinking at multiple levels of abstraction.

Fundamental, not rote skill. A fundamental skill is something every human being must know to function in modern society. Rote means a mechanical routine. Historically, not until computer science solves the AI Grand Challenge of making computers think like humans will thinking be rote.

A way that humans, not computers, think. Computational thinking is a way humans solve problems; it is not trying to get humans to think like computers. Computers are dull and boring; humans are clever and imaginative. We humans make computers exciting. Equipped with computing devices, we use our cleverness to tackle problems we would not dare take on before the age of computing and build systems with functionality limited only by our imaginations.

Complements and combines mathematical and engineering thinking. Computer science inherently draws on mathematical thinking, given that, like all sciences, its formal foundations rest on mathematics. Computer science inherently draws on engineering thinking, given that we build systems that interact with the real world. The constraints of the underlying computing device force computer scientists to think computationally, not just mathematically. Being free to build virtual worlds enables us to engineer systems beyond the physical world.

Ideas, not artifacts. It's not just the software and hardware artifacts we produce that will be physically present everywhere and touch our lives all the time; it will be the computational concepts we use to approach and solve problems, manage our daily lives, and communicate and interact with other people and

For everyone, everywhere. Computational thinking will be a reality when it is as integral to human endeavors as it is an explicit philosophy.

Many people equate computer science with computer programming. Some parents see only a narrow range of job opportunities for their children who major in computer science. Many people think the fundamental research in computer science is done, and that only the engineering remains. Computational thinking is a grand vision to guide computer science educators, researchers, and practitioners as we act to change society's image of the field. We especially need to reach the pre-college audience, including teachers, parents, and students, sending them two main messages:

Intellectually challenging and engaging scientific problems remain to be understood and solved. The problem domains and solution domains are limited only by our own curiosity and creativity and. One can major in computer science and do anything. One can major in English or mathematics and go on to a multitude of different careers. Ditch computer science. One can major in computer science and go on to a career in medicine, law, business, politics, any type of science or engineering, and even the arts.

Professors of computer science should teach a course called "Why I Think Like a Computer Scientist" to college freshmen, making it available to non-majors, not just to computer science majors. We should expose pre-college students to computational methods and models. Rather than bemoan the decline of interest in computer science or the decline in funding for research in computer science, we should look to inspire the public's interest in the intellectual adventure of the field. We'll thus speed the rise, use, and power of computer science, aiming to make computational thinking commonplace. ■

JEANNETTE M. WING (wing@cs.cmu.edu) is the President's Professor of Computer Science at and head of the Computer Science Department at Carnegie Mellon University, Pittsburgh, PA.

© 2006 ACM 978-1-59593-010-9/06/0000

COMMUNICATIONS OF THE ACM, March 2006, Vol. 49, No. 3 35

- ❖ 컴퓨팅 사고의 오해와 특징

- Conceptualizing, not programming
(프로그래밍이 아닌 개념화)
- Fundamental, not rote skill
(단순반복 기술이 아닌 기본적인 기술)
- A way that humans, not computers, think
(컴퓨터가 아닌 인간의 사고 방법)
- Complements and combines mathematical and engineering thinking
(수학적 사고와 공학적 사고의 보완과 결합)
- Idea, not artifacts
(인공물이 아닌 아이디어)
- For everyone, everywhere
(모두를 위한 것)

1 컴퓨팅 사고 시작

Jeannette Wing – ‘Computational Thinking’

Conspicuously, not programming. Computer science is not computer programming. Thinking like a computer scientist means more than being able to program a computer. It requires thinking at multiple levels of abstraction; Fundamental, not rote skill. A fundamental skill is something every human being must know to function in modern society. Rote means a mechanical routine. Ironically, not until computer science solves the AI Grand Challenge of making computers think like humans will thinking be rote.

A way that humans, not computers, think. Computational thinking is a way humans solve problems; it is not trying to get humans to think like computers. Computers are dull and boring; humans are clever and imaginative. We humans make computers exciting. Equipped with computing devices, we use our cleverness to tackle problems we would not dare take on before the age of computing and build systems with functionality limited only by our imaginations.

Complement and combine mathematical and engineering thinking. Computer science inherently draws on mathematical thinking, given that, like all sciences, its formal foundations rest on mathematics. Computer science inherently draws on engineering thinking, given that we build systems that interact with the real world. The constraints of the underlying computing device force computer scientists to think computationally, not just mathematically. Being free to build virtual worlds enables us to engineer systems beyond the physical world.

Ideas, not artifacts. It's not just the software and hardware artifacts we produce that will be physically present everywhere and touch our lives all the time; it will be the computational concepts we use to approach and solve problems, manage our daily lives, and communicate and interact with other people; and

For everyone, everywhere. Computational thinking will be a reality when it is as integral to human endeavors as it disappears as an explicit philosophy.

Many people equate computer science with computer programming. Some parents see only a narrow range of job opportunities for their children who major in computer science. Many people think the fundamental research in computer science is done, and that only the engineering remains. Computational thinking is a grand vision to guide computer science educators, researchers, and practitioners as we act to change society's image of the field. We especially need to reach the pre-college audience, including teachers, parents, and students, sending them two main messages:

Intellectually challenging and engaging scientific problems remain to be understood and solved. The problem domain and solution domain are limited only by our own curiosity and creativity; and One can major in computer science and do anything. One can major in English or mathematics and go on to a multitude of different careers. Diverse computer science. One can major in computer science and go on to a career in medicine, law, business, politics, any type of science or engineering, and even the arts.

Professors of computer science should teach a course called "Ways to Think Like a Computer Scientist" to college freshmen, making it available to non-majors, not just to computer science majors. We should expose pre-college students to computational methods and models. Rather than bemoan the decline of interest in computer science or the decline in funding for research in computer science, we should look to inspire the public's interest in the intellectual adventure of the field. We'll thus spread the joy, awe, and power of computer science, aiming to make computational thinking commonplace. ■

JEANNETTE M. WING, longfellow@cs.cmu.edu is the President's Professor of Computer Science in and holds the Computer Science Department at Carnegie Mellon University, Pittsburgh, PA.

© 2004 ACM 0898-9290/04/0000-0000 \$5.00

❖ 필요성 강조

- 통계학, 생물학, 경제학, 화학, 물리학 등 다양한 학문 분야에서 폭 넓게 사용
- 유비쿼터스 (Ubiquitous) 시대 에 꼭 필요
- 컴퓨터 전문 용어의 일반화

❖ 전파 방법

- 컴퓨터 과학과 교수들은 “컴퓨터 과학자처럼 생각하는 방법” 과목 개설
- 컴퓨터 비 전공 학생들 대상 교육 필요
- 예비 신입생 과정 과목으로 개설

2 컴퓨팅 사고 정의

❖ 컴퓨터 과학자들이 컴퓨터를 이용하여 문제해결을 할 때 사용하는 기술을

개념화한 사고방식 → 컴퓨팅 사고력 → 문제해결능력

❖ 컴퓨터 과학자처럼 생각하기

❖ 컴퓨터 과학자 : 컴퓨터를 이용하여 문제해결을 하고자 하는 과학자

❖ 컴퓨터 과학자들의 문제해결 기술

- 시스템 성능 개선을 위한 기술 – 시스템 프로그램의 발전
- 컴퓨터로 해결 가능한 문제의 정의와 해결 계획 – 응용 소프트웨어 와 알고리즘 개발
- 문제의 크기에 따른 해결방법 – 분해, 패턴인식, 추상화
- 프로그래밍 기법 – 알고리즘 표현과 결과

3 컴퓨팅 사고 요소

여러가지 컴퓨팅 사고 요소

❖ Jeannete Wing 의 컴퓨팅 사고 요소

주요 요소	의미
재귀적 사고 (Recursive Thinking)	문제의 대표 해결 방법을 찾고 문제에 지속적으로 반복 적용하는 방법
병렬적 사고 (Parallel Processing)	독립된 문제들을 동시에 처리하여 문제 해결의 효율성을 높이는 방법
분해 (Decomposition)	어려운 문제를 쉽고 작은 문제로 분해하여 문제를 해결하는 방법
추상화 (Abstraction)	문제의 핵심을 추출하여 문제를 해결하는 방법
경험에 의한 추리적 사고 (Heuristic reasoning)	문제해결을 위하여 계획하고, 학습하고, 일정을 관리하는 방법
예측 (Prevention, Protection, Recovery)	문제점과 문제해결 과정의 오류를 예측하여 대비하는 방법

3 컴퓨팅 사고 요소

여러가지 컴퓨팅 사고 요소

- ❖ CSTA (Computer Science Teachers Association)
- ❖ ISTE (International Society for Technology in Education)

주요 요소	의미
데이터 수집	문제해결에 도움이 되는 정보 수집
데이터 표현	데이터를 해결방법에 따라 다시 표현
데이터 분석	문제해결을 위하여 데이터를 분석
분해	어려운 문제를 쉽고 작은 문제로 분해하여 문제를 해결하는 방법
패턴인식	문제의 대표 해결 방법이나 규칙을 찾는 방법
추상화	문제의 핵심을 추출하여 문제를 해결하는 방법
알고리즘	문제 해결 방법을 순서대로 나열하는 것
시뮬레이션	문제가 해결될 수 있는지 미리 알아보기 위한 모의 실험 방법
병렬화	독립된 문제들을 동시에 처리하여 문제 해결의 효율성을 높이는 방법
자동화	문제 해결 방법을 일반화 하여 여러가지 문제를 자동으로 해결하고자 하는 방법

3 컴퓨팅 사고 요소

여러가지 컴퓨팅 사고 요소

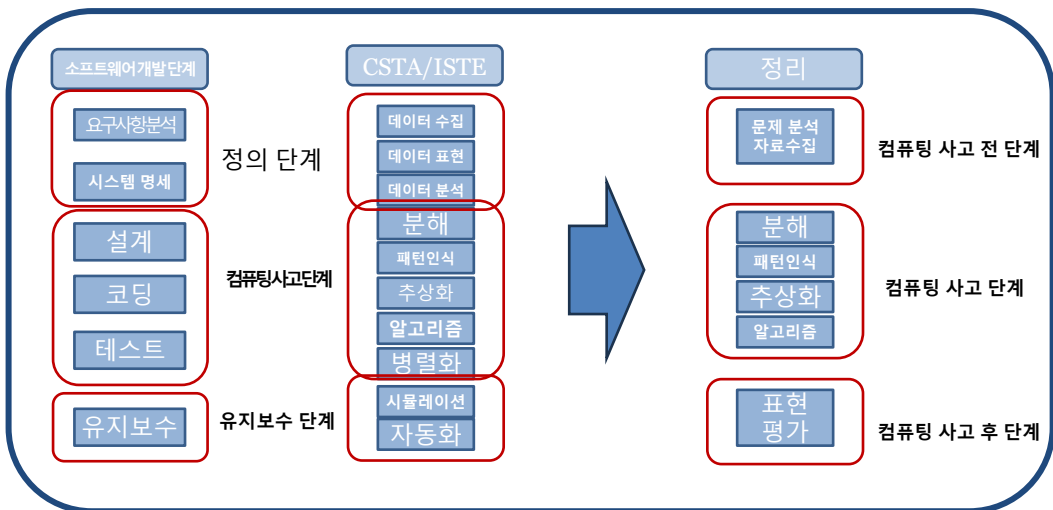
❖ 한국교육학술정보원 (2015년 발표)

주요 요소		의미
자료수집		문제해결에 도움이 되는 정보 수집
자료 분석		자료의 이해, 패턴 찾기, 결론 도출하기
구조화		문제를 그래프, 차트, 그림 등으로 시각화 하기
추상화	분해	문제를 관리 가능한 수준의 작은 문제로 나누기
	모델링	문제해결을 위한 핵심요소를 추출하고 모델 만들기
	알고리즘	문제해결을 위한 일련의 단계를 알고리즘으로 표현하기 (절차적 표현)
자동화	코딩	프로그래밍 언어를 이용해 문제해결 과정을 자동화 하기
	시뮬레이션	프로그램 실행하기
일반화		문제해결 과정을 다른 문제에 적용하기

3 컴퓨팅 사고 요소

소프트웨어 개발과 컴퓨팅 사고 요소

❖ 소프트웨어 개발 단계와 컴퓨팅 사고 요소의 관계



3 컴퓨팅 사고 요소

컴퓨팅 사고 요소

❖ 재정리

- 단계의 과정 / 문제 해결을 위한 요소
- 컴퓨팅 사고 전 단계 → 컴퓨팅 사고 단계 → 컴퓨팅 사고 후 단계
- 문제해결을 위한 컴퓨팅 사고 요소 : 분해, 패턴인식, 추상화, 알고리즘

컴퓨팅 사고 단계	컴퓨팅 사고 요소	정의
컴퓨팅 사고 전 단계	문제 분석 과 자료수집	문제를 이해하고, 문제의 해결 가능성과 필요성 정의
컴퓨팅 사고 단계	분해	복잡한 문제를 단순한 문제로, 어려운 문제를 쉬운 문제로 변환
	패턴인식	문제에서 공통적인 유사성이나 특징을 찾아냄
	추상화	중요한 특징을 추출
	알고리즘	문제를 해결하기위해 규칙을 순서대로 표현
컴퓨팅 사고 후 단계	표현과 평가	해결된 문제를 표현하고, 오류를 확인하고, 문제 해결을 확인

4 컴퓨팅 사고 교육

컴퓨팅 사고 교육 내용

❖ 컴퓨터에 대한 이해

- 컴퓨터 시스템 구조와 역사 - 컴퓨터 시스템의 발전 방향, 컴퓨터 시스템의 동작을 이해
- 컴퓨터 시스템의 소프트웨어와 하드웨어의 특징

❖ 컴퓨터 과학자들의 기술에 대한 이해

- 소프트웨어 개발에 대한 이해
- 프로그래밍 언어에 대한 이해

❖ 개념화된 사고력의 이해와 적용

- 컴퓨팅 사고 요소에 대한 이해 와 적용

4 컴퓨팅 사고 교육

컴퓨팅 사고 교육 방법

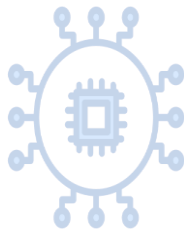
❖ 이론 교육

- 컴퓨터 시스템 이론
- 컴퓨팅 사고 요소 이론

❖ 코딩 교육

- 파이썬 프로그래밍 언어 기초 문법
- 파이썬 프로그래밍 언어를 통한 문제해결

❖ 프로젝트 진행 교육 (실제 소프트웨어 개발 경험)



SW중심대학사업 교재공동개발

2. 컴퓨팅 사고 요소

Computational Thinking



0 목차

➤ 컴퓨팅 사고 단계

➤ 컴퓨팅 사고 요소

- 분해 (Decomposition)
- 패턴인식 (Pattern Recognition)
- 추상화 (Abstraction)
- 알고리즘 (Algorithms)

1 컴퓨팅 사고 단계

컴퓨팅 사고 단계

❖ 재정리

- 컴퓨팅 사고 : 컴퓨터 과학자들이 컴퓨터를 통해 문제해결을 할 때 사용하는 기술(문제해결 능력 → 사고력)
- 컴퓨터 과학자들의 대표적인 기술 → 소프트웨어 개발 → 컴퓨팅 사고 단계
- 컴퓨터 과학자들의 문제해결 기술 → 컴퓨팅 사고 요소 → 분해, 패턴인식, 추상화, 알고리즘

컴퓨팅 사고 단계	컴퓨팅 사고 요소	정의
컴퓨팅 사고 전 단계	문제 분석과 자료수집	문제를 이해하고, 문제의 해결 가능성과 필요성 정의
컴퓨팅 사고 단계	분해	복잡한 문제를 단순한 문제로, 어려운 문제를 쉬운 문제로 변환
	패턴인식	문제에서 공통적인 유사성이나 특징을 찾아냄
	추상화	중요한 특징을 추출
	알고리즘	문제를 해결하기 위해 규칙을 순서대로 표현
컴퓨팅 사고 후 단계	표현과 평가	해결된 문제를 표현하고, 오류를 확인하고, 문제 해결을 확인

1 컴퓨팅 사고 단계

컴퓨팅 사고 전 단계

- ❖ 문제를 정확하게 이해해야 문제를 해결할 수 있음
- ❖ 문제를 해결할 필요성과 가치를 파악해야 함
- ❖ 문제를 해결할 수 있는지 확인해야 함
- ❖ 컴퓨팅 사고 요소 : 문제 분석과 자료수집

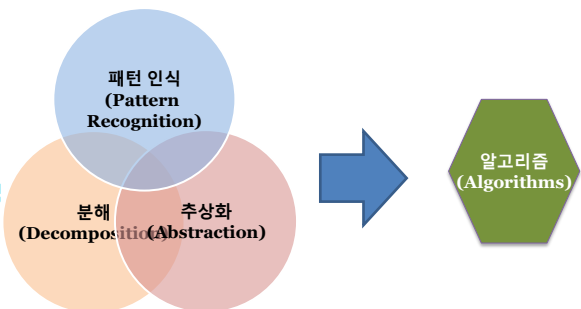
- 문제 정의
- 문제의 필요성
- 문제 해결 가능성
- 결과예측
- 자료수집

예) 교통문제 / 지구 온난화 문제 / 가장 빠른 길 찾기 문제 / 여름휴가문제 /
안전문제

1 컴퓨팅 사고 단계

컴퓨팅 사고 단계

- ❖ 문제를 해결하는 과정
- ❖ 문제해결 방법을 찾기 위해 정의된 4가지 컴퓨팅 사고 요소를
이용
- ❖ 단독 또는 복합적으로 사용할 수 있음
 - 분해 (Decomposition)
 - 패턴인식 (Pattern Recognition)
 - 추상화 (Abstraction)
 - 알고리즘 (Algorithms)
- ❖ 알고리즘은 모든 문제 해결의 핵심
- ❖ 알고리즘은 문제해결의 결과



1 컴퓨팅 사고 단계

컴퓨팅 사고 후 단계

❖ 알고리즘 표현과 평가 단계

❖ 알고리즘의 효율성, 알고리즘의 오류, 알고리즘의 적합성 확인

❖ 체크리스트

- 문제에 대한 정확한 답을 내는가?
- 정해진 시간 안에 답을 낼 수 있는가?
- 비슷한 다른 문제에도 적용할 수 있는가?
- 알고리즘에 오류는 없는가?
- 다른 해결 알고리즘은 없는가?
- 모든 조건을 만족 하는가?

2 컴퓨팅 사고 요소

컴퓨팅 사고 요소 - 분해 (Decomposition)

❖ 복잡한 문제를 작은 부분들로 나누어 문제를 해결하려는 기술을 개념화한 컴퓨팅 사고 요소

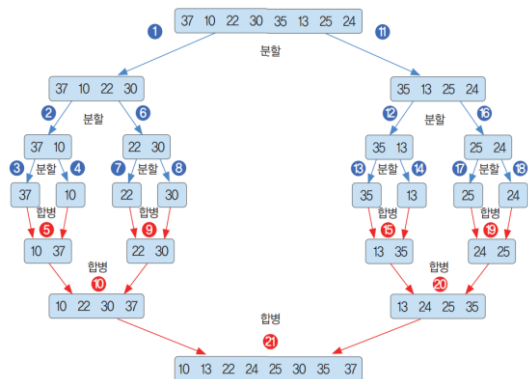
❖ 문제의 특성이나 데이터의 특성과 상관없이 무조건 작은 문제로 나누어 해결하는 기술

❖ 분해된 작은 문제를 모두 해결하면 자동으로 큰 문제가 해결될 수 있다는 생각

❖ 컴퓨터 과학에서 분해 이론을 적용한 알고리즘

▪ 분할정복 알고리즘 (합병 정렬 알고리즘)

- 문제의 크기를 가장 작은 상태로 분할
- 작은 크기의 데이터를 정렬(합병) 함
- 알고리즘
 - 분할 → 합병



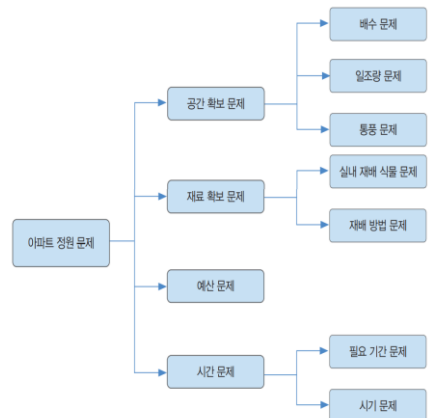
2 컴퓨팅 사고 요소

컴퓨팅 사고 요소 - 분해 (Decomposition)

- ❖ 컴퓨터 과학분야의 분할 기술을 컴퓨팅 사고 요소로 개념화 하여 일반 문제에 적용하여 문제를 해결할 수 있음

❖ 아파트 정원 꾸미기 문제

- 큰 문제 (아파트 정원 꾸미기 문제) 를 작은 문제로 분해
- 컴퓨팅 사고 전 단계의 문제 분석과 자료수집을 통해 나온 결과를 중심으로 문제를 분해
- 분해 요소를 통한 문제해결 알고리즘
 - 예산문제 → 공간확보문제 → 재료확보문제 → 시간문제
 - 시간문제 → 재료확보문제 → 예산문제 → 공간확보문제



2 컴퓨팅 사고 요소

컴퓨팅 사고 요소 - 패턴인식 (Pattern Recognition)

- ❖ 복잡한 문제에서 일정한 패턴을 찾아 문제를 해결하는 컴퓨팅 사고 요소
- ❖ 패턴은 공통적인 유사성이나 특징
- ❖ 복잡한 문제에서 패턴을 찾아 문제를 단순화 하고자 하는 생각
- ❖ 컴퓨터 과학에서 패턴인식 이론을 적용한 알고리즘

- 문자 패턴 추출 알고리즘
- 클러스터링 알고리즘 (K-means 알고리즘)
- 동적 알고리즘 (Dynamic Programming)
- 퀵 정렬 알고리즘 (Quick Sort)
- 이미지 처리, 침입탐지 시스템, 상품추천 시스템 등



2 컴퓨팅 사고 요소

컴퓨팅 사고 요소 - 패턴인식 (Pattern Recognition)

❖ 컴퓨터 과학분야의 패턴인식 기술을 컴퓨팅 사고

요소로 개념화 하여 일반 문제에 적용하여 문제를
해결할 수 있음

❖ 아파트 정원 꾸미기 문제

- 컴퓨팅 사고 전 단계의 문제 분석과 자료수집을 통해 나온

결과를 중심으로 패턴을 찾는다.

- 패턴을 중심으로 문제를 작게 재 정의한다.

- 패턴인식으로 정의한 알고리즘

- 규모문제 → 종류 문제 → 용도문제
- 용도문제 → 규모문제 → 종류문제
- 일반화 가능

- 일상생활의 문제 생각해보기
 - 우리집에 작은 정원을 만들 수 있을까?
 - 정원을 만들기 위한 필요한 공간은?
 - 배수가 가능한 공간, 햇빛 공급이 원활한 공간, 관리하기 편한 공간
 - 정원을 만들기 위해서는 무엇이 필요한가?
 - 정원의 규모에 따른 필요 요소, 플랜트 종류에 따른 필요 요소, 정원 용도에 따른 필요 요소
 - 정원을 만들기 위한 예산은?
 - 정원 규모에 따른 예산, 플랜트 종류에 따른 예산, 정원 용도에 따른 예산
 - 정원을 만들기 위해 필요한 기간은?
 - 정원 규모에 따른 기간, 플랜트 종류에 따른 기간, 정원 용도에 따른 기간

정원 규모
플랜트 종류
정원 용도

2 컴퓨팅 사고 요소

컴퓨팅 사고 요소 - 추상화 (Abstraction)

❖ 복잡한 문제에서 불필요한 특징을 제거하고 핵심적인 특징을 추출하여 문제를

해결하는 컴퓨팅 사고 요소

❖ 복잡한 문제에서 핵심적인 특징을 찾아 문제를 단순화 하고자 하는 생각



서울 전체를 자세하게 표현



서울 지하철 노선을 중심으로 표현



서울 지역구를 중심으로 표현

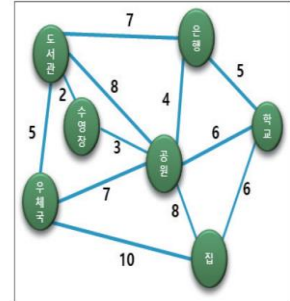
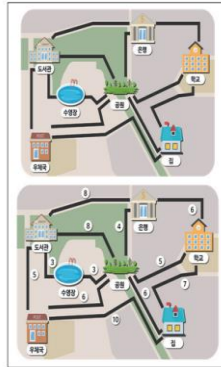
*필요한 부분, 중요한 부분을 통합하여 하나로 만드는 추상화는 단순히 간단하게 나타내는 것만을 의미하는 것이 아니라 특징에 맞게 **핵심을 부각시키는 방법**으로 필요 없는 것을 제외시키면서 표현한 것이다.

2 컴퓨팅 사고 요소

컴퓨팅 사고 요소 - 추상화 (Abstraction)

❖ 컴퓨터 과학에서 추상화 기술의 의미

- 컴퓨터로 수행하기 위하여 데이터를 변환
- 컴퓨터에서 필요한 핵심 데이터를 추출
- 최단거리 찾기 문제
- 이미지 처리 문제, 음성인식 문제 등
- 컴퓨터로 해결하고자 하는 모든 문제



<최단거리찾기 문제>

2 컴퓨팅 사고 요소

컴퓨팅 사고 요소 - 추상화 (Abstraction)

❖ 컴퓨터 과학분야의 추상화 기술을 통하여 일상생활의 문제를 컴퓨터로 해결할 수 있음

❖ 아파트 정원 꾸미기 문제

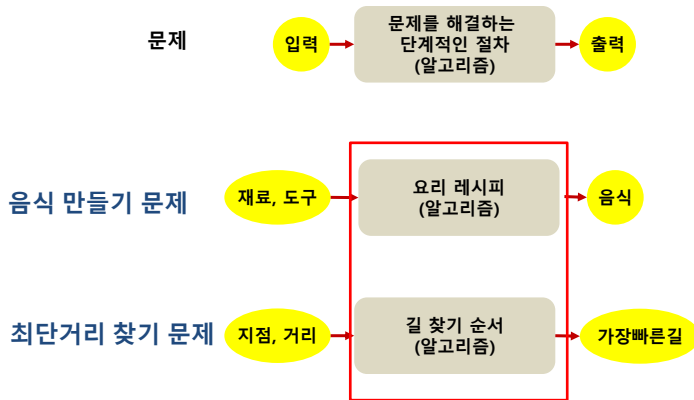
- 아파트 정원 꾸미기 문제를 해결 해 주는 프로그램을 개발
 - 꼭 필요한 값을 입력 값과 출력 값으로 재 정의
 - 정원의 규모, 플랜트 종류, 정원의 용도, 아파트 조건 → 숫자나 문자로 재 정의
 - 입력 : 규모, 종류, 용도, 아파트 조건 (크기, 방향, 설계도면 등)
 - 출력 : 정원 꾸미기 가능성 (Yes/No)

예상 기간, 예상 비용, 추천 플랜트, 추천 장소 등

2 컴퓨팅 사고 요소

컴퓨팅 사고 요소 - 알고리즘 (Algorithm)

- ❖ 어떠한 문제를 해결하기 위한 일련의 절차를 공식화 한 형태로 표현한 것
- ❖ 컴퓨터 과학 분야에서의 알고리즘은 어떤 일을 수행 할 수 있는 일련의 명령어 또는 규칙
- ❖ 알고리즘의 형식



2 컴퓨팅 사고 요소

컴퓨팅 사고 요소 - 알고리즘 (Algorithm)

- ❖ 컴퓨터 과학에서 알고리즘의 의미
 - 알고리즘을 찾았다. → 문제가 해결되었다.
 - 알고리즘이 있는가? → 해결될 수 있는 문제인가?
 - 컴퓨터로 실행하여 결과값을 얻을 수 있다.
 - 컴퓨터 알고리즘의 조건을 만족해야 한다.
 - 0개 이상의 입력 / 1개이상의 출력
 - 명확성 - 주어진 입력에 항상 올바른 출력을 해야함
 - 유한성 - 모든 명령어나 연산은 반드시 컴퓨터에서 실행 가능한 동작이나 연산이어야 함
 - 효과성 - 알고리즘은 반드시 종료되어야 함

2 컴퓨팅 사고 요소

컴퓨팅 사고 요소 - 알고리즘 (Algorithm)

❖ 컴퓨터 과학 분야에서 공식화된 알고리즘의 표현 방법

- 자연어
 - 인간의 언어로 알고리즘을 표현하는 방법
- 의사코드
 - 슈도코드(pseudo-code)
 - 프로그래밍언어를 흉내 내어 알고리즘을 표현하는 방법
 - 특정 프로그래밍 언어와 관계없이 논리의 명시를 위해 작성
- 순서도
 - 플로차트(flowchart)
 - 알고리즘을 약속된 도형을 이용하여 그림으로 표현하는 방법
- 프로그래밍 언어
 - 실제 컴퓨터로 실행시키기 위하여 컴퓨터의 명령어로 알고리즘을 표현하는 방법

알고리즘을
인간이 이해하기
쉽게 표현하는
방법

알고리즘을
컴퓨터가 실행할
수 있도록
표현하는 방법

2 컴퓨팅 사고 요소

컴퓨팅 사고 요소 - 알고리즘 (Algorithm)

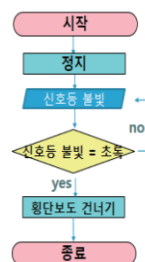
- ❖ 컴퓨터 과학분야의 알고리즘 기술을 컴퓨팅 사고 요소로 개념화 하여 일반 문제의
해결 절차 (알고리즘)를 표현할 수 있음
- ❖ 횡단보도 건너기 문제
 - ❖ 컴퓨팅 사고력을 기반으로 문제를 해결 하는 알고리즘을 정의

1. 횡단보도 앞에 정지한다.
2. 신호등을 확인한다.
3. 초록색이면 횡단보도를 건넌다.
4. 빨간색이면 2번으로 되돌아간다.

자연어 표현

```
input 불빛
while 불빛 == 초록색
    output 건너기
```

의사코드 표현



순서도 표현



SW중심대학사업 교재공동개발

3. 튜링기계

Computational Thinking



0 목차

- 인간의 도구
- 컴퓨터 시스템
- 튜링기계

1 인간의 도구

❖ 컴퓨터 시스템은 인간이 발명한 도구이다.

- 인간의 도구는 각각의 목적이 명확하다.
- 컴퓨터 시스템은 다양한 목적으로 사용될 수 있는 도구이다.
- 다른 도구들은 각각의 하드웨어적 사용법을 익혀야 한다.
- 컴퓨터 시스템 도구의 사용법은 소프트웨어 사용법을 익히는 것이다.



여러가지 도구



컴퓨터 시스템

2 컴퓨터 시스템

❖ 컴퓨터 시스템은 계산하는 도구이다.

- 계산 : 주어진 정보를 이용해 어떤 값이나 결과를 구하는 과정이다.
- 컴퓨터 시스템은 모든 문제를 계산하여 해결하는 도구이다.
- 컴퓨터 시스템의 계산 → **저장, 연산**
- 컴퓨터 시스템으로 문제를 해결하기 위해서는 모든 문제를 계산 (저장, 연산)하여 해결할 수 있도록 변환해야 한다.



2 컴퓨터 시스템

❖ 컴퓨터 시스템

❖ 하드웨어

- CPU, MM, I/O (CPU-MM 입출력장치) → 저장, 연산을 수행하는 장치
- 주변기기 (I/O 장치) → 인간과 컴퓨터 시스템의 소통을 할 수 있도록 하는 입출력장치

❖ 소프트웨어

- 시스템 소프트웨어 - 시스템 문제 해결
- 응용 소프트웨어 - 인간의 여러가지 문제 해결

3 튜링기계

앨런 튜링 (Alan Turing)



❖ 앨런 튜링 (Alan Turing)

- 영국의 수학자, 암호학자, 논리학자
- 컴퓨터 과학의 선구자
- 튜링기계
 - 1936년 “계산가능한 수에 대하여, 수리명제 자동생성 문제에 응용하면서” 논문에서 소개
 - 수학적으로 컴퓨터 시스템의 형태를 완벽하게 증명함
- 튜링테스트
 - 1950년 “컴퓨팅 기계와 지능” 논문에서 소개
 - 인간의 지능과 동등하거나 구별할 수 없는 지능적인 행동을 보여주는 기계의 능력에 대한 테스트
- 튜링상

3 튜링 기계

앨런 튜링 (Alan Turing) - 논문

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO
THE ENTSCHEIDUNGSPROBLEM

By A. M. TURING.

[Received 28 May, 1936.—Read 12 November, 1936.]

The “computable” numbers may be described briefly as the real numbers whose expressions as a decimal are calculable by finite means. Although the subject of this paper is ostensibly the computable numbers, it is almost equally easy to define and investigate computable functions of an integral variable or a real or computable variable, computable predicates, and so forth. The fundamental problems involved are, however, the same in each case, and I have chosen the computable numbers for explicit treatment as involving the least cautious technique. I hope shortly to give an account of the relations of the computable numbers, functions, and so forth to one another. This will include a development of the theory of functions of a real variable expressed in terms of computable numbers. According to my definition, a number is computable if its decimal can be written down by a machine.

In §§ 9, 10 I give some arguments with the intention of showing that the computable numbers include all numbers which could naturally be regarded as computable. In particular, I show that certain large classes of numbers are computable. They include, for instance, the real parts of all algebraic numbers, the real parts of the zeros of the Bessel functions, the numbers π , e , etc. The computable numbers do not, however, include all definable numbers, and an example is given of a definable number which is not computable.

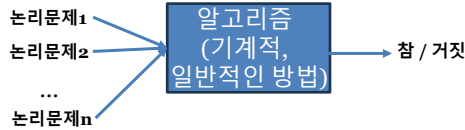
Although the class of computable numbers is so great, and in many ways similar to the class of real numbers, it is nevertheless enumerable. In § 8 I examine certain arguments which would seem to prove the contrary. By the correct application of one of these arguments, conclusions are reached which are superficially similar to those of Gödel†. These results

† Gödel, “Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme, I”, *Monatsh. Math. Phys.*, 38 (1931), 173–198.

❖ 논문의 배경

❖ 결정문제 (Entscheidungsproblem)

- 1928년 다비드 힐베르트(David Hilbert) 에 의해 문제 제기
- “모든 논리문제를 해결하는 기계적인 방법이 존재하는가?”
- 기계적인 방법 - 알고리즘



❖ 논리문제

- 계산 이론에서 참/거짓 답이 있는 질문

→ 자동으로 모든 결정문제를 해결할 수 있는 알고리즘은 존재하지 않는다 (불완전성 정리 - 1931년 쿠르트 괴델(Kurt Gödel))

→ 1936년 튜링이 다른 방법으로 증명

3 튜링 기계

앨런 튜링 (Alan Turing) - 논문

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO
THE ENTSCHEIDUNGSPROBLEM

By A. M. TURING.

[Received 28 May, 1936.—Read 12 November, 1936.]

The “computable” numbers may be described briefly as the real numbers whose expressions as a decimal are calculable by finite means. Although the subject of this paper is ostensibly the computable numbers, it is almost equally easy to define and investigate computable functions of an integral variable or a real or computable variable, computable predicates, and so forth. The fundamental problems involved are, however, the same in each case, and I have chosen the computable numbers for explicit treatment as involving the least cautious technique. I hope shortly to give an account of the relations of the computable numbers, functions, and so forth to one another. This will include a development of the theory of functions of a real variable expressed in terms of computable numbers. According to my definition, a number is computable if its decimal can be written down by a machine.

In §§ 9, 10 I give some arguments with the intention of showing that the computable numbers include all numbers which could naturally be regarded as computable. In particular, I show that certain large classes of numbers are computable. They include, for instance, the real parts of all algebraic numbers, the real parts of the zeros of the Bessel functions, the numbers π , e , etc. The computable numbers do not, however, include all definable numbers, and an example is given of a definable number which is not computable.

Although the class of computable numbers is so great, and in many ways similar to the class of real numbers, it is nevertheless enumerable. In § 8 I examine certain arguments which would seem to prove the contrary. By the correct application of one of these arguments, conclusions are reached which are superficially similar to those of Gödel†. These results

† Gödel, “Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme, I”, *Monatsh. Math. Phys.*, 38 (1931), 173–198.

❖ 앨런 튜링 논문

- 1936년 “On computable Numbers, with and Application to the Entscheidungsproblem”: “계산가능한 수에 대해서, 수리명제 자동생성 문제에 응용하면서”
- 모든 논리문제를 해결할 수 있는 알고리즘 (기계적인 방법)으로 ‘튜링기계’를 제안
- 논문의 구성

- 결정문제에 대한 정리 → 모든 문제를 해결하는 기계 제한 (튜링기계) →

튜링기계의 문제해결 방법과 문제를 예로 제시 → 튜링기계의

특징(Universal Machine) 정의 및 증명 → 튜링기계로 해결 할 수 없는

결정문제 (알고리즘이 없는 결정 불가능한 문제 제시 - 멈춤문제) →

불완전성정리 증명

3 튜링기계

튜링기계 - 구성요소

❖ 테이프 (Tape)

- 일정한 크기(squares)로 나누어진 유한한 크기의 기억장치
- 인간의 기억은 유한하다, 아날로그세상의 노트 역할
- 정의된 유한개의 심볼(symbol)을 쓸 수 있다.
 - 계산가능한 자연수 / 문제해결을 돕는 기호

❖ 헤드 (Head)

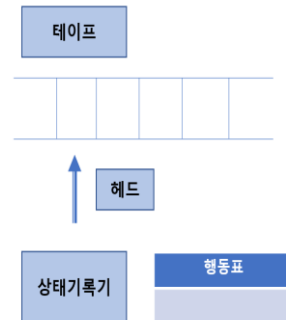
- 테이프에 심볼을 쓰고, 지우고, 읽는 장치
- 오른쪽 한 칸 또는 왼쪽 한 칸 이동하기(shift) 가능

❖ 상태 (m-configuration)

- 헤드의 동작을 제어하는 기계
- 문제를 해결하기 위해 헤드의 동작을 정의 - 유한개의 상태로 정의

▪ 행동표

- 상태에 정의된 각 헤드의 행동(조작)을 글로 표현
- 각 상태별 헤드가 읽은 현재값, 헤드의 조작, 다음상태를 글로 표현



3 튜링기계

튜링기계 - 동작 - 예제 1

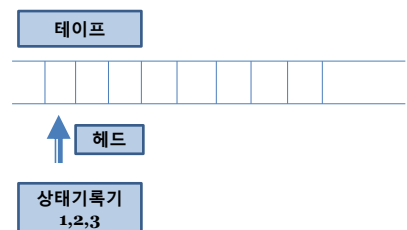
❖ 첫번째 튜링기계 - 0과 1을 테이프에 무한히 반복해서 쓰는 튜링기계

- 시작설정 : 헤드가 테이프의 어느 한 곳에 정지하여 읽고 있다.
- 문제해결 요소

- **필요한 심볼 정의** : 테이프에 쓰여질 값이나 기호
- **필요한 상태 정의** : 문제를 헤드의 동작으로 재정의
 - 0을 쓰는 동작 → 상태 1
 - 1을 쓰는 동작 → 상태 2
 - 헤드를 오른쪽으로 한 칸 이동하는 동작 → 상태 3

▪ 행동표 작성

- 행동표에 표현할 헤드 조작 기호 정의 - 명령어
 - 헤드가 테이프에 정해진 값이나 기호를 쓴다 → P(Print)
 - 헤드를 오른쪽으로 한 칸 이동하여 값을 읽는다 → R(Right shift read)
 - 헤드를 왼쪽으로 한 칸 이동하여 값을 읽는다 → L(Left shift read)
 - 헤드가 테이프에 쓰여진 값이나 기호를 지운다 → E(Erase)
- 정의된 명령어를 이용하여 행동표 작성

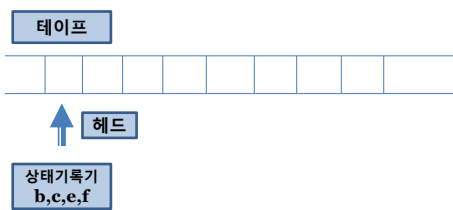


3 튜링기계

튜링기계 - 동작 - 예제 1

❖ 첫번째 튜링기계 - 0과 1을 테이프에 무한히 반복해서 쓰는 튜링기계

- 필요한 심볼 정의 : 0 / 1
- 필요한 상태 정의 :
 - 0을 쓴다 → 상태 b
 - 1을 쓴다 → 상태 e
 - 헤드를 오른쪽으로 이동하고 1을 쓰는 상태로 변환 → 상태 c
 - 헤드를 오른쪽으로 이동하고 0을 쓰는 상태로 변환 → 상태 f
- 행동표 작성
 - P,R,L,E 명령어를 이용하여 작성



행동표			
상태 (m-config)	읽은 값 (symbol)	헤드 조작 (operation)	다음상태 (final m-config)
b	none	P0	c
c	none	R	e
e	none	P1	f
f	none	R	b

3 튜링기계

튜링기계 - 동작 - 예제 1

❖ 같은 문제를 해결하는 여러가지 튜링기계

- 문제를 해결하기 위해 정의하는 동작 (상태) 과 각 상태의 실행 순서를 정의 (행동표)에 따라 다름



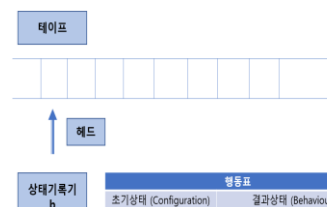
행동표			
초기상태 (Configuration)	결과상태 (Behaviour)		
현재상태 (m-config)	읽은 값 (symbol)	헤드 조작 (operation)	다음상태 (final m-config)
b	none	P0	c
c	none	R	e
e	none	P1	f
f	none	R	b

<튜링기계1>



행동표			
초기상태 (Configuration)	결과상태 (Behaviour)		
현재상태 (m-config)	읽은 값 (symbol)	헤드 조작 (operation)	다음상태 (final m-config)
b	none	P0,R	c
c	none	R	e
e	none	P1,R	f
f	none	R	b

<튜링기계2>



행동표			
초기상태 (Configuration)	결과상태 (Behaviour)		
현재상태 (m-config)	읽은 값 (symbol)	헤드 조작 (operation)	다음상태 (final m-config)
b	none	P0	b
	0	R,R,P1	b
	1	R,R,P0	b

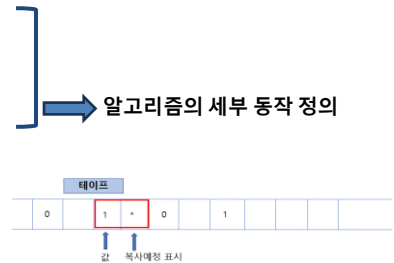
<튜링기계3>

3 튜링기계

튜링기계 - 동작 - 예제 2

❖ 두번째 튜링기계 - 1을 하나씩 증가시키면서 쓰는 튜링기계

- 필요한 상태 정의 :
 - 알고리즘을 먼저 정의 : 하나씩 증가시킨다 → 1을 하나 더 쓰고 앞에 쓴 1을 복사 한다(알고리즘)
 - 앞에 1을 찾아 복사예정 표시를 한다 → 상태 1
 - 맨 뒤로 가서 1을 쓴다 → 상태 2
 - 앞으로 가서 복사예정 표시를 지운다 → 상태 3
 - 증가완료 표시를 쓰고 처음으로 돌아간다 → 상태 4
- 데이터 저장을 위한 테이프 칸 수 정의
 - 값 심볼이 쓰여질 위치와 기호 심볼이 쓰여질 위치가 필요
- 필요한 심볼 정의 : 1(문제에서 요구한 값) / 0(한번 증가완료), @(시작), *(복사예정) (그 외 기호)
- 행동표 작성
 - P,R,L,E 명령어를 이용하여 작성

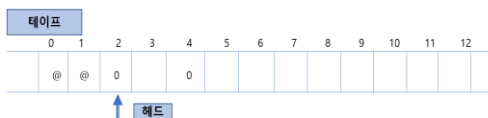


3 튜링기계

튜링기계 - 동작 - 예제 2

❖ 두번째 튜링기계 - 1을 하나씩 증가시키면서 쓰는 튜링기계

- 필요한 심볼 정의 : 1 / 0 / @ / *



행동표			
상태 (m-config)	읽은 값 (symbol)	헤드 조작 (operation)	다음상태 (final m-config)
b	none	P@, R, P@, R, Po, R, R, Po, L, L	o
o	0		q
	1	R, P*, L, L, L	o
q	Any (o or 1)	R, R	q
	none	P1, L	p
p	*	E, R	q
	@	R	f
	none	L, L	p
f	Any	R, R	f
	none	Po, L, L	o

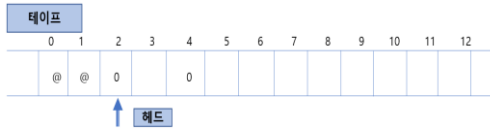
- 필요한 상태 정의 :
 - 앞에 1을 찾아 복사예정 표시를 한다 → 상태 1(o)
 - 0이면 상태 2로 이동한다. (복사가 끝났음, 1추가)
 - 1이면 오른쪽으로 한 칸 이동하여 표시한다. (다시 왼쪽으로 이동한다.)
 - 맨 뒤로 가서 1을 쓴다 → 상태 2(q)
 - 테이프에 값이 있으면, 오른쪽으로 두 칸 씩 이동한다. (맨 뒤로 이동)
 - 테이프에 값이 없으면, 1을 쓰고 상태 3으로 이동한다.
 - 앞으로 가서 복사예정 표시를 지운다 → 상태 3(p)
 - 복사예정 표시가 있으면, 지우고 상태 2로 이동한다. (1 하나 복사)
 - 표시가 없으면, 왼쪽으로 두 칸 씩 이동한다. (1을 찾음)
 - 시작표시가 있으면, 상태 4로 이동한다. (더 이상 1이 없고 맨 앞까지 왔음)
 - 증가완료 표시를 쓰고 처음으로 돌아간다 → 상태 4(f)
 - 테이프에 값이 있으면, 오른쪽으로 두 칸 씩 이동한다.
 - 테이프에 값이 없으면, 0을 쓰고 상태 1로 이동한다. (복사 끝)

행동표 작성

- P,R,L,E 명령어를 이용하여 작성

3 튜링기계

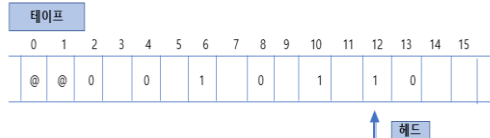
튜링기계 - 동작 - 예제 2



조작 : $O \rightarrow q \rightarrow q \rightarrow p \rightarrow p \rightarrow$
 $f \rightarrow f \rightarrow O$



조작 : $O \rightarrow o \rightarrow q \rightarrow q \rightarrow q \rightarrow q \rightarrow q \rightarrow p \rightarrow p \rightarrow q \rightarrow q \rightarrow q \rightarrow$
 $p \rightarrow p \rightarrow p \rightarrow p \rightarrow p \rightarrow p \rightarrow f \rightarrow f \rightarrow f \rightarrow f \rightarrow f \rightarrow f \rightarrow O$



행동표			
상태 (m-config)	읽은 값 (symbol)	헤드 조작 (operation)	다음상태 (final m-config)
b	none	P@, R, P@, R, Po, R, R, Po, L, L	o
o	o		q
	1	R, P*, L, L, L	o
q	Any (o or 1)	R, R	q
	none	P1, L	p
p	*	E, R	q
	@	R	f
	none	L, L	p
f	Any	R, R	f
	none	Po, L, L	o

3 튜링기계

튜링기계 - 특징 - Universal Machine

❖ Universal Machine - 만능기계

- 하나의 튜링기계를 자연수로 표현할 수 있다.
- 자연수로 표현한 튜링기계를 테이프에 표현할 수 있다.
- 튜링기계를 표현한 다른 튜링기계에서 똑같은 동작을 수행할 수 있다.

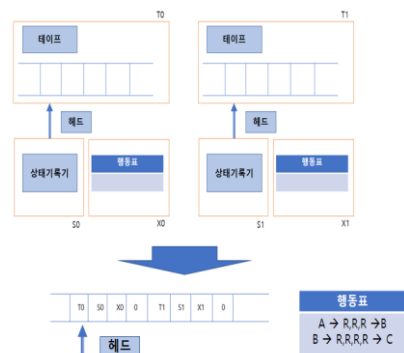
→ 튜링기계는 무한히 많이 만들 수 있다.

→ 튜링기계는 새로운 튜링기계의 부품으로 사용할 수 있다.

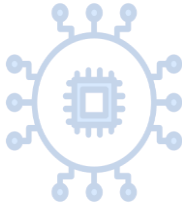
❖ 멈춤문제

- 튜링기계를 입력으로 받아 튜링기계가 멈출지 판단하는 문제 (멈춤문제)를 해결하는 튜링기계는 존재할 수 없다.
- 자연 수의 세계에서 멈춤문제를 해결하는 튜링기계가 존재한다면 자연수를 넘어가는 자연 수 가 존재한다는 것 → 모순

→ 자동으로 모든 문제를 해결할 수 있는 기계적인 방법(알고리즘)은 존재하지 않는다. (불완전성정리)



튜링기계	T1	T2	T3	Tn
T11	o		O	
T12	o	o		
T13		o	o	O
Tm			o	



SW중심대학사업 교재공동개발

4. 컴퓨터

Computational Thinking



0 목차

- 컴퓨터의 역사
- 소프트웨어 (Software)
- 프로그래밍 언어 (Programming Language)

1 컴퓨터의 역사

컴퓨터의 기원

❖ 컴퓨터 하드웨어 역사 (기원전 부터 1940년대 까지)

기원전 : 주판

1642년 : 파스칼의 계산기

1671년 : 라이프니츠의 계산기

1822년 : 찰스 배비지의 차분기관과 해석기관

1942년 : ABC(Atanasoff Berry Computer)

1944년 : Mark I, ENIAC

1945년 : EDVAC

1 컴퓨터의 역사

튜링기계와 폰 노이만 구조

❖ 튜링기계 (Turing Machine) - 1936년

- 테이프, 헤드, 상태기록기, 행동표 로 구성
- 수학 이론을 바탕으로 제안

❖ 폰 노이만 구조 (Von Neumann Architecture) - 1945년

- 중앙처리장치, 메모리, 프로그램 으로 구성
- 프로그램 내장 방식
- 기존 컴퓨터 스위치와 전선을 이용한 프로그램 방식의 단점을 보완

→ 현재 컴퓨터 : 중앙처리장치(CPU), 메모리(MM), 프로그램

1 컴퓨터의 역사

튜링기계와 폰 노이만 구조

❖ 튜링기계 (Turing Machine) 의 특징

- “모든 문제를 해결하는 튜링기계는 존재하지 않는다.”
- 수학적으로 증명

❖ 폰 노이만 구조 (Von Neumann Architecture) 의 문제점

- 병목현상
 - 중앙처리장치의 처리속도와 메모리 입출력 속도의 차이
 - 직렬처리 방식 / 순차처리 방식

→ 현대 컴퓨터의 발전

- 해결할 수 있는 문제와 해결할 수 없는 문제를 찾음
- 병목현상을 해결하기 위한 방안 모색 → 계층 메모리 구조, 병렬처리 방법 - 시스템 문제

1 컴퓨터의 역사

컴퓨터 과학자

❖ 컴퓨터 과학자

- 컴퓨터 (중앙처리장치, 메모리)를 이용하여 문제를 해결하는 소프트웨어를 만드는 사람

→ 저장/연산 으로 모든 문제해결 알고리즘을 표현

- 해결할 수 있는 문제와 해결할 수 없는 문제를 찾음

→ 알고리즘 연구

- 병목현상을 해결하기 위한 방안 모색 → 계층형 메모리 구조, 병렬처리 방식, 다중처리 방식

→ 시스템 프로그램 발전

- 데이터 저장 / 운영 문제, 데이터 통신 emd

→ 데이터베이스 연구, 데이터 통신 연구

1 컴퓨터의 역사

하드웨어 역사

❖ 컴퓨터 하드웨어 역사 (1950년대 이후)

- 1940년대 하드웨어 : 진공관, 전기선
- 1957년 반도체, 집적회로 발명 → 소형화, 안정성 중심의 발전

	1세대	2세대	3세대	4세대	5세대
연도	1946-1957	1957-1964	1965-1974	1975-1990	1991-현재
재료	진공관	트랜지스터	IC	LSI	VLSI
처리속도	$1/10^3$	$1/10^6$	$1/10^9$	$1/10^9$	$1/10^{12}$
특징	대형, 과열, 고장, 느린처리속도	부피감소 전력소모량 감소	소형화 기억용량 증가	소형화 기억용량 증가	소형화 기억용량 증가

2 소프트웨어

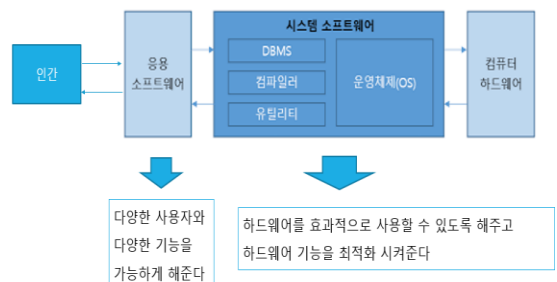
- ❖ 컴퓨터 과학자들 연구의 결과물
- ❖ 문제 해결의 결과물
- ❖ 알고리즘을 컴퓨터에서 정의한 연산으로 표현하여 컴퓨터가 실행할 수 있도록 만든 것

❖ 시스템 소프트웨어

- ❖ 시스템 문제 해결
- ❖ 하드웨어의 효율적 사용, 최적화
- ❖ 응용 소프트웨어 실행을 위한 플랫폼 제공

❖ 응용 소프트웨어

- ❖ 인간의 여러가지 문제 해결
- ❖ 운영체제에서 실행되는 모든 소프트웨어



2 소프트웨어

❖ 시스템 소프트웨어 종류

- 운영체제 (OS) : Windows, Unix, Linux, iOS 등
- 언어번역 프로그램 : C, C++, JAVA, Python 등
- 유틸리티 : 파일압축 프로그램, 디스크 관련 유틸리티, 로더, 링커 등
- DBMS (Data Base Management System) : Oracle, MS-SQL, MySQL 등

→ 효율적인 장치 관리, 안정성, 속도 를 중심으로 발전

→ 장치 성능 향상과 다양성으로 시스템 소프트웨어의 기능 다양화

❖ 응용 소프트웨어 종류

- 인간이 사용하는 모든 제한적 기능을 갖는 소프트웨어 (HWP, MS-office, Web, game등)

→기능 중심으로 발전

→UI 중요

2 소프트웨어

❖ 컴퓨터 소프트웨어 역사 (1950년대 이후)

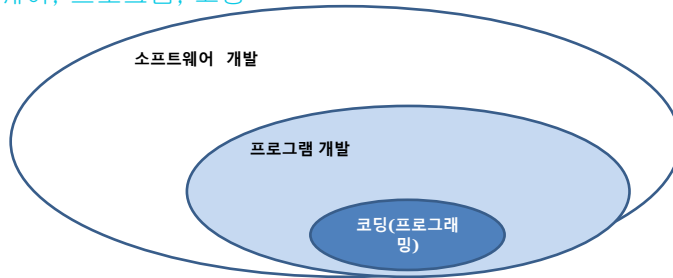
- 시스템 소프트웨어 개발 → 응용 소프트웨어 개발 → 인공지능 소프트웨어 개발

	1세대	2세대	3세대	4세대	5세대
연도	1946-1957	1957-1964	1965-1974	1975-1990	1991-현재
응용분야	과학계산 통계	생산관리 사무통계	의사결정 경영관리	자동생산 데이터베이스 실시간처리	자동설계
특징	기계어 사용	운영체제 개발 고급언어 개발 일괄처리방식	경영정보시스템 주변장치의 고속화 운영체제 발전 멀티프로그래밍 시분할 방식	개인용 컴퓨터 운영체제 발전 분산처리 네트워크 슈퍼컴퓨터	인공지능 병렬처리 초고속망 인터넷

3 프로그래밍 언어

❖ 해결된 문제의 알고리즘을 컴퓨터에서 실행 가능 하도록 명령어로 표현

❖ 소프트웨어, 프로그램, 코딩



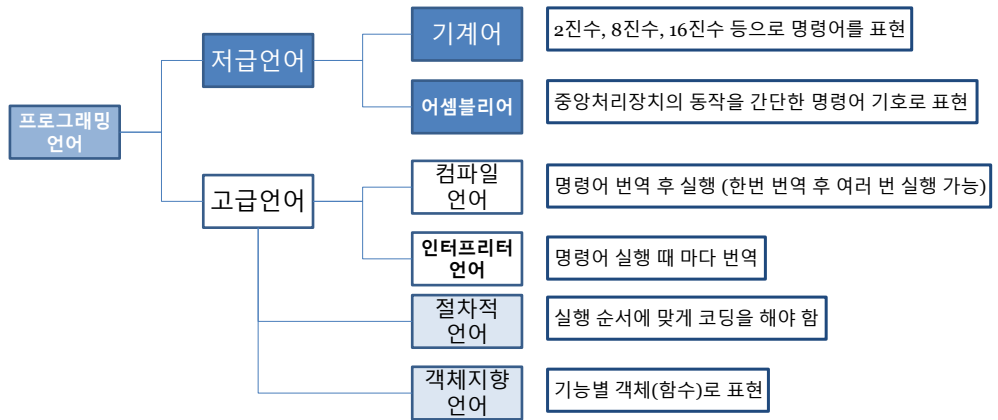
❖ 바둑대전 소프트웨어를 개발하고자 할 때, 바둑 기능 프로그램을 만들 어야 하며, 이때, 파이썬 프로그래밍 언어로 코딩(프로그래밍)할 수 있습니다.

3 프로그래밍 언어

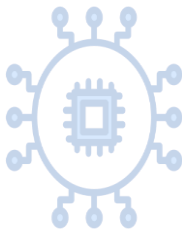
❖ 프로그래밍 언어의 분류

- 언어의 표현 방법에 따라 ➔ 저급언어 / 고급언어
(급언어는 저급언어(기계어)로 번역해야 함)
- 고급언어의 실행(번역) 방법에 따라 ➔ 컴파일러 / 인터프리터
- 고급언어의 코드 설계 방법에 따라 ➔ 절차적 언어 / 객체지향 언어 (비 절차적 언어)

3 프로그래밍 언어



	컴파일언어	인터프리터언어	절차적언어	객체지향언어
C	O		O	
C++	O		O	O
JAVA	O			O
Python		O	O	O



SW중심대학사업 교재공동개발

5. 데이터의 표현

Computational Thinking



0 목차

- 정보와 데이터
- 주 기억 장치
- 데이터 변환 과 데이터 단위

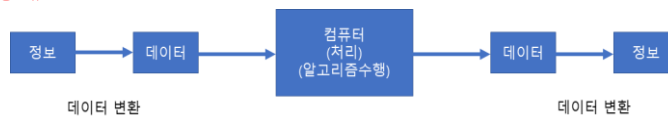
1 정보와 데이터

❖ 정보

- 사전적 정의 : 목적에 맞게 의미 있고, 관리하기 쉬운 형태로 정리 된 것
- 컴퓨터 시스템 관점의 정의 : 프로그램에 직접 사용될 수 없는 형태로 저장되어 있는
형태

❖ 데이터

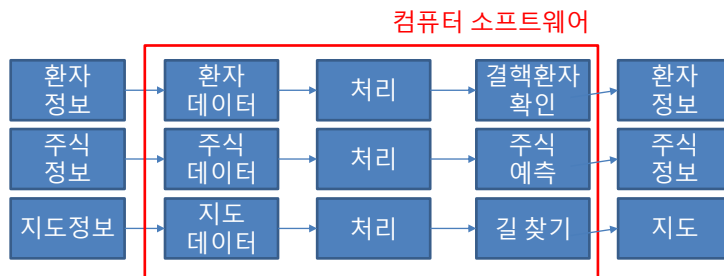
- 사전적 정의 : 어떤 물체나 현상을 측정하거나 관찰하여 얻어진 값
- 컴퓨터 시스템 관점의 정의 : 프로그램에 직접 사용될 수 있는 형태로 저장되어 있는
형태



1 정보와 데이터

❖ 컴퓨터를 이용한 문제 해결 (입력 - 처리 - 출력)

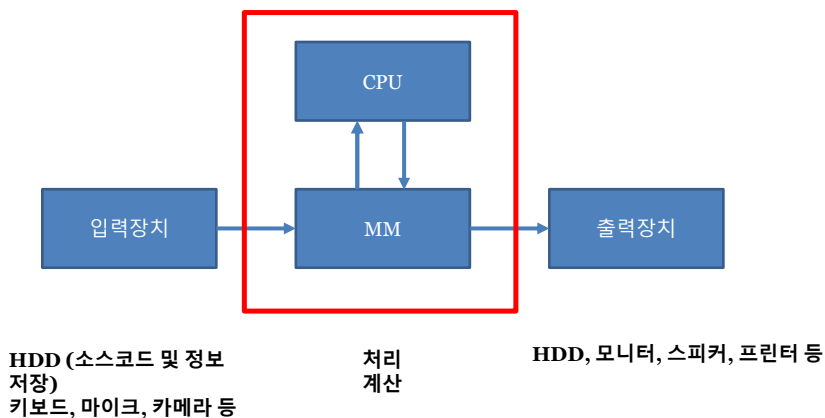
- 입력 : 컴퓨터에서 처리할 수 있는 방식으로 **저장**
- 처리 : **연산/저장** 반복
- 출력 : 처리결과를 **저장**, 저장된 데이터를 인간의 방식(시각, 청각, 촉각)으로 변환



1 정보와 데이터

❖ 컴퓨터를 이용한 문제 해결 (입력 - 처리 - 출력)

❖ 컴퓨터 소프트웨어 실행 과정



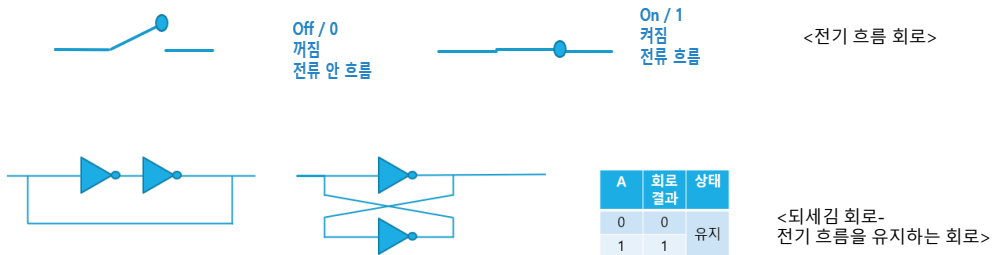
2 주 기억 장치

❖ MM (Main Memory), RAM (Random Access Memory)

❖ 휘발성 기억장치

❖ 전기(전자) 회로로 구성 - 전기가 흐르는 상태 (1)/ 전기가 흐르지 않는 상태(0)
를 유지

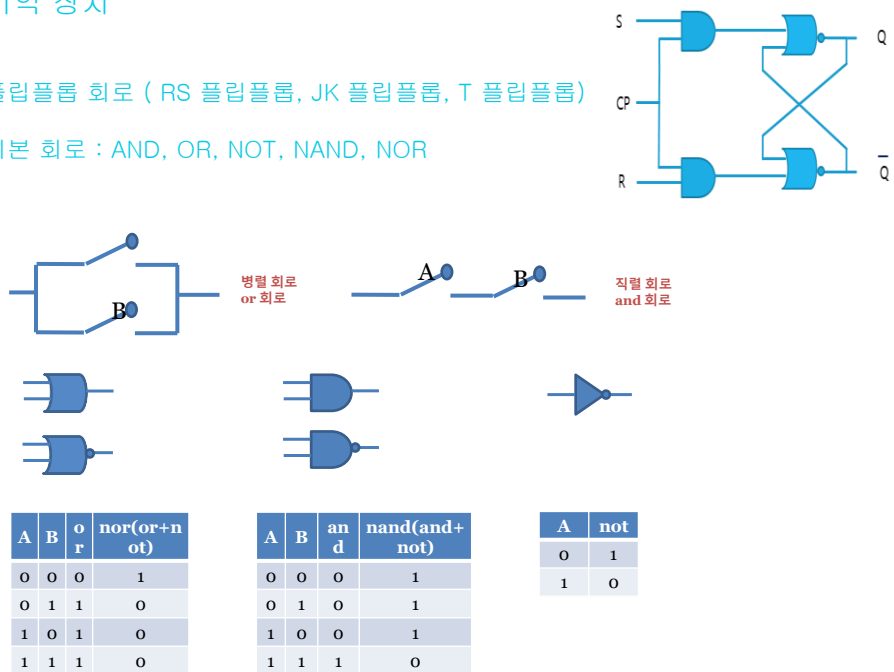
❖ 주 기억 장치 회로 - 플립플롭 회로 (RS 플립플롭, JK 플립플롭, T 플립플롭)



2 주 기억 장치

❖ 플립플롭 회로 (RS 플립플롭, JK 플립플롭, T 플립플롭)

❖ 기본 회로 : AND, OR, NOT, NAND, NOR



3 데이터 변환

- ❖ 모든 정보를 0,1 데이터로 변환하여 저장해야 함
- ❖ 입력 변환
 - 연산을 위한 변환
 - 무조건 0과 1로 변환
- ❖ 출력 변환
 - 인간이 이해할 수 있는 출력형식으로 변환 (이미지/ 소리로 변환)

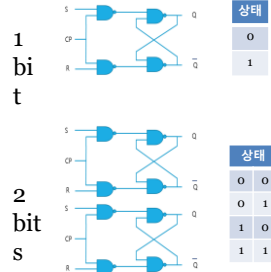
3 데이터 변환

- ❖ 입력 변환 종류
 - 숫자변환 (표준입력)
 - 2진법 사용
 - 문자변환 (표준입력)
 - 아스키 코드 (ASCII Code) / 유니 코드 (Unicode)
 - 이미지 변환 (특수변환)
 - 픽셀변환
 - 소리 변환 (특수변환)
 - 샘플링 (아날로그 → 디지털)

3 데이터 단위

❖ 데이터 저장 단위

- 비트 (bit)
 - 0 또는 1을 하나 저장하는 단위
 - 1비트 - 0,1 ($2^1=2$) 2개의 상태 저장
 - 2비트 - 00,01,10,11 ($2^2=4$) 4개의 상태 저장
 - 3비트 - ($2^3=8$) 8개의 상태 저장
- 바이트 (byte)
 - 0 또는 1을 8비트로 저장하는 단위
 - 8비트 - ($2^8=256$) 256개의 상태 저장
 - 8bit = 1byte
- 워드 (word)
 - 컴퓨터 설계 시 정해지는 처리 단위
 - 처리를 8비트 단위로 하는 컴퓨터 : 8bit=1byte=1word
 - 처리를 16비트 단위로 하는 컴퓨터 : 16bit=2byte=1word
 - 처리를 64비트 단위로 하는 컴퓨터 : 64bit=8byte=1word



8 bits

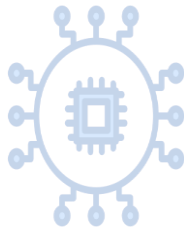
2진수	10진수
0000 0000	0
0000 1111	15
1111 1111	255

3 데이터 단위

❖ 데이터 저장 단위

여러가지 데이터 단위	
1 Byte	8 bit
1 KB	1,024 byte
1 MB	1,024 KB
1 GB	1,024 MB
1 TB	1,024 GB

단위	저장 예
Bit	True/False
Byte	1개 문자, 1개 숫자
KB	몇 개의 문단
MB	1분 길이 MP3 노래
GB	30분 길이의 HD 영화
TB	약 200편의 FHD 영화



SW중심대학사업 교재공동개발

6. 저장

Computational Thinking



0 목차

- 저장
- 자료형
- 자료구조

1 저장

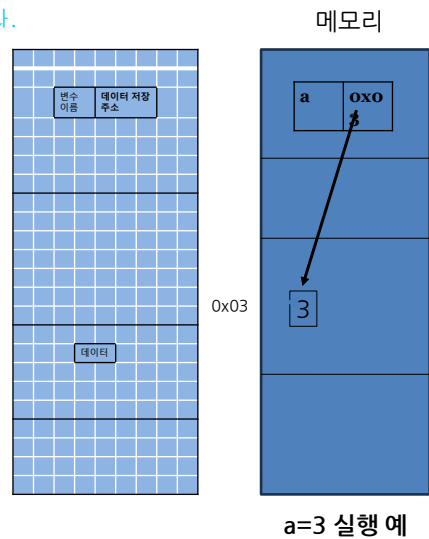
❖ 모든 연산은 저장된 데이터를 기반으로 이루어진다.

❖ 주 기억 장치에 변환된 데이터를 저장한다.

❖ 저장도 연산이다. (할당 연산)

❖ 변수

- RAM (Random Access Memory) 가능 방식
- 저장된 데이터의 위치 주소 값
- 저장된 데이터의 삭제/수정 가능
- 한번 저장하고 여러 번 사용 가능
- “=” 연산자를 사용하여 정의 → 할당하다



2 자료형

❖ Data Type

❖ 데이터 변환 형식 → 어떻게 변환하여 저장할 것인가?

❖ 변환 데이터의 크기 → 주 기억 장치 저장 공간 확보

❖ 연산 (산술연산, 비교연산) 결과 → 값의 범위 결정

❖ 모든 프로그래밍 언어는 각각 다양한 자료형을 제공한다.

❖ 자료형의 종류

- 숫자 (정수 / 소수)
- 문자 (문자열 / 문자)

a=3	a='3'	a=3.0
b=4	b='4'	b=4.0
c=a+b	c=a+b	c=a+b
print(c)	print(c)	print(c)

2 자료형

❖ C, C++ 자료형 예

종류	자료형	메모리 크기	값의 범위
문자형	char	1byte	-128~+127
정수형	int	4byte	-2,147,483,648~+2,147,483,647
	unsigned int	4byte	0~4,294,967,295
	long long int	8byte	-9,223,372,036,854,775,808~9,223,372,036,854,775,807
	short	2byte	-32,768~32,767
실수형	float	4byte	3.4*e-38 ~ 3.4*e38
	double	8byte	1.7*e-308 ~ 1.7*e308

<C>

종류	자료형	메모리 크기	값의 범위
문자형	char	1byte	-128 ~ 127
	unsigned char	1byte	0 ~ 255
정수형	__int8	1byte	-128~127
	__int16	2byte	-32,768 ~ 32,767
	unsigned int	2byte	-32,768 ~ 32,767
	(signed) short(int)	2byte	-32,768 ~ 32,767
	unsigned short(int)	2byte	0 ~ 65,535
	__int32	4byte	-2,147,483,648~+2,147,483,647
	(signed) int	4byte	-2,147,483,648~+2,147,483,647
	unsigned int	4byte	0~4,294,967,295
	(signed) long(int)	4byte	-2,147,483,648~+2,147,483,647
	unsigned long(int)	4byte	-2,147,483,648~+2,147,483,647
	__int64	8byte	-9,223,372,036,854,775,808~9,223,372,036,854,775,807
실수형	float	4byte	3.4*e-38 ~ 3.4*e38
	double	8byte	1.7*e-308 ~ 1.7*e308
	Long double	8byte	1.2*e-4,932 ~ 1.2*e4932

<C++>

2 자료형

❖ JAVA, Python 자료형 예

종류	자료형	메모리 크기	값의 범위
문자형	char	4byte	16비트 유니코드 문자 데이터
정수형	byte	1byte	-128~127
	short	2byte	-32,768 ~ 32,767
	int	4byte	-2,147,483,648~+2,147,483,647
	long	8byte	-9,223,372,036,854,775,808~9,223,372,036,854,775,807
실수형	float	4byte	3.4*e-38 ~ 3.4*e38
	double	8byte	1.7*e-308 ~ 1.7*e308
불린형	boolean	1byte	0,1

<JAVA>

종류	자료형	메모리 크기	값의 범위
문자형	str	거의 무제한	정의되는 모든 문자 - 자료구조 형
정수형	int	거의 무제한	정의되는 모든 수
실수형	float	8byte	1.7*e-308 ~ 1.7*e308
복소수형	complex	시스템에서 정의	정의되는 복소수
불린형	boolean	1byte	0,1

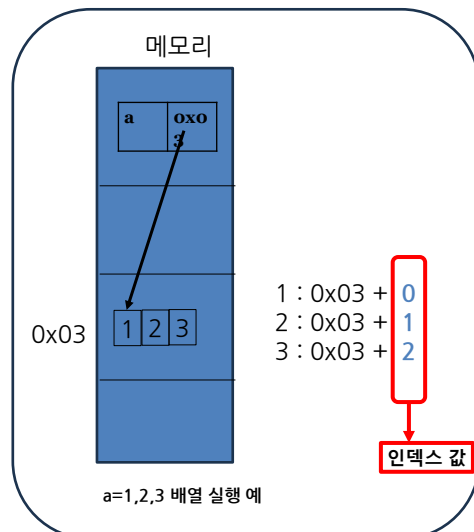
<Python>

3 자료구조

- ❖ 데이터를 구조적으로 저장하는 방식
- ❖ 하나의 변수에 여러 개의 데이터를 저장하는 방식
- ❖ 데이터 값의 모임, 데이터 간의 관계도 함께 표현할 수 있음
- ❖ 저장된 데이터의 효율적 관리가 중요
 - 검색 / 삽입 / 삭제 / 수정
- ❖ 자료구조의 중요성
 - 메모리를 효율적으로 사용하기 위하여 데이터와 문제에 따라 자료구조 선택 중요
 - 문제 해결 방법에 따라 자료구조 선택이 달라짐
 - 프로그램 설계 시 가장 먼저 고려되어야 함
 - 프로그램의 성능에 직접적인 영향을 미침

3 자료구조

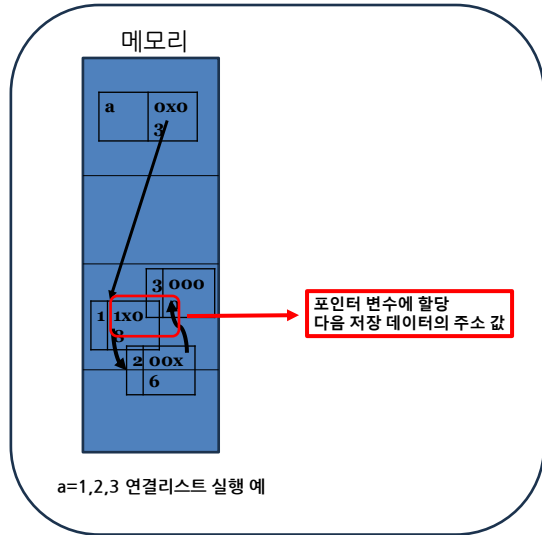
- ❖ 구조적으로 저장하는 방식
 - 배열
 - 직접접근 가능
 - 삽입/삭제 비 효율적
 - 연속된 메모리 공간 요구
 - 인덱스 : 주소 증가치



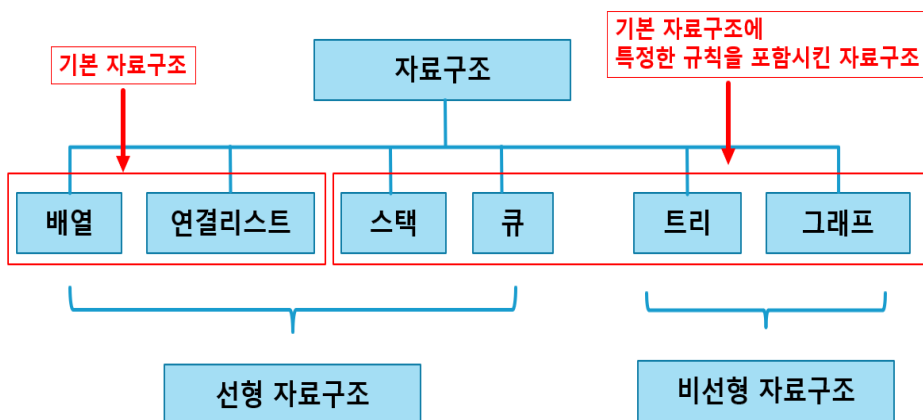
3 자료구조

❖ 구조적으로 저장하는 방식

- 연결리스트
 - 순차접근 속도 저하
 - 삽입/삭제 효율적
 - 메모리 공간의 효율적 사용
 - 연결리스트
 - 포인터 : 주소저장변수



3 자료구조



3 자료구조

- ❖ 모든 프로그래밍 언어는 다양한 자료구조를 제공한다.
- ❖ C, C++, JAVA, Python 자료구조 예

프로그래밍 언어	자료구조
C	배열, 연결 리스트
C++	List, stack, queue, vector, deque, set, multiset, map, multimap
JAVA	List, set, queue, map
파이썬	List, set, tuple, dictionary

3 자료구조

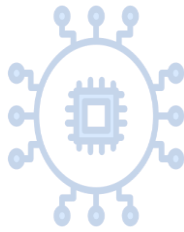
- ❖ 두 정수의 합을 구하는 표현 방법
- ❖ 변수를 이용한 표현방법 과 자료구조를 이용한 표현방법
- ❖ 파이썬 프로그래밍 언어를 이용

```
a=3
b=4
c=a+b
print(c)
```

변수를 이용한 표현방법

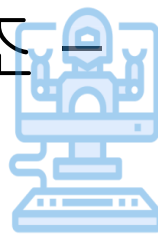
```
a=[3,4]
c=a[0]+a[1]
print(c)
```

리스트 자료구조를 이용한 표현방법



SW중심대학사업 교재공동개발

7. 규칙기반 자료구조 – 스택 큐



Computational Thinking

0 목차

- 규칙기반 자료구조와 알고리즘
- 규칙기반 자료구조
- 스택
- 큐

1 규칙기반 자료구조와 알고리즘

❖ 자료구조와 알고리즘의 오해

- 자료구조는 어렵다.
- 자료 형과 자료구조는 같은 말이다.
- 자료구조는 전문 프로그래머만 알아야 하는 전문 지식이다.
- 자료구조와 알고리즘은 같은 의미이다.

- ✓ 자료구조가 무엇인가
- ✓ 자료구조가 왜 필요한가
- ✓ 자료구조와 알고리즘은 어떤 **관계**가 있는가
- ✓ 자료구조와 알고리즘은 어떤 **차이**가 있는가

1 규칙기반 자료구조와 알고리즘

❖ 자료구조

- ❖ 여러 개의 데이터를 메모리에 저장
- ❖ 효율적 메모리 활용 중심으로 개발

❖ 규칙기반 자료구조

- ❖ 특정 문제를 자료구조에 기반하여 해결하기 위하여 특정 규칙을 포함하여 저장
- ❖ 문제의 효율적 해결방법을 **저장** 중심으로 찾음

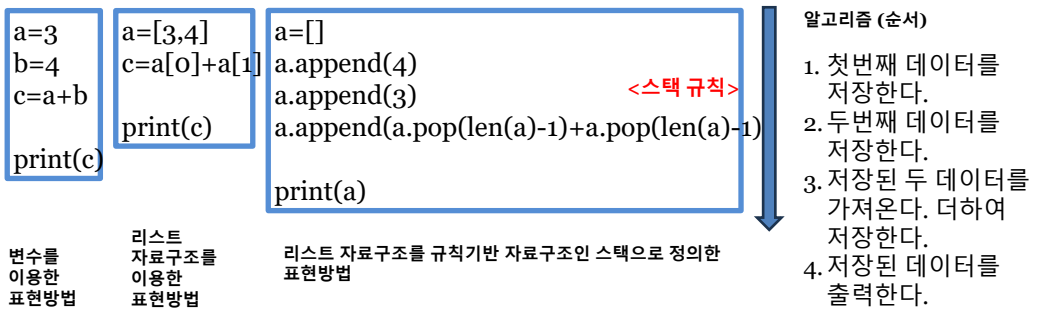
❖ 알고리즘

- ❖ 문제의 효율적 해결방법을 **연산** 중심으로 찾음

1 규칙기반 자료구조와 알고리즘

❖ 두 정수의 합을 구하는 표현 방법

- 변수를 이용한 표현방법
- 자료구조를 이용한 표현방법
- 규칙기반 자료구조를 이용한 표현 방법

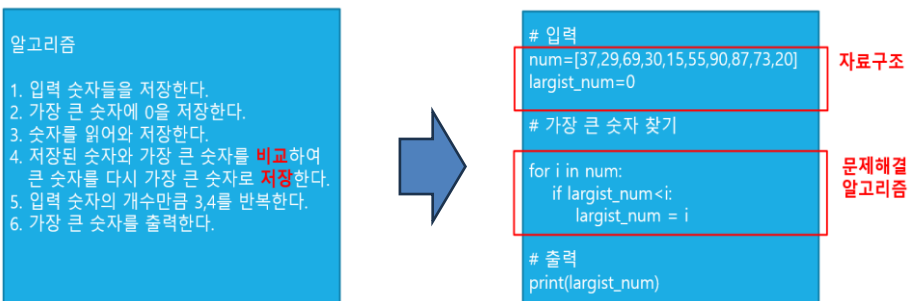


1 규칙기반 자료구조와 알고리즘

❖ 가장 큰 숫자 찾기 문제

입력 값 : 37, 29, 69, 30, 15, 55, 90, 87, 73, 20
출력 값 : 90 (가장 큰 숫자)

→ 알고리즘 (연산중심)을 이용한 문제해결



1 규칙기반 자료구조와 알고리즘

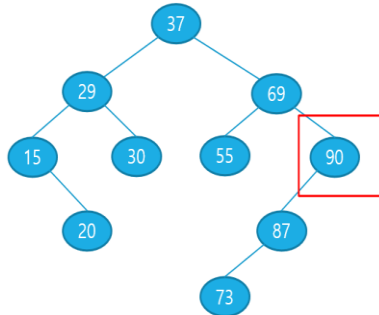
❖ 가장 큰 숫자 찾기 문제

입력 값 : 37, 29, 69, 30, 15, 55, 90, 87, 73, 20

출력 값 : 90 (가장 큰 숫자)

→ 규칙기반 자료구조를 이용한 문제해결

부모 노드 보다 크면 오른쪽,
작으면 왼쪽에 저장 하는 알고리즘을 포함한 트리 자료구조



부모 노드의 가장 오른쪽에
있는 값이 가장 큰 숫자

1 규칙기반 자료구조와 알고리즘

❖ 가장 큰 숫자 찾기 문제

입력 값 : 37, 29, 69, 30, 15, 55, 90, 87, 73, 20

출력 값 : 90 (가장 큰 숫자)

→ 규칙기반 자료구조를 이용한 문제해결

알고리즘

1. 트리 자료구조 규칙으로 데이터를 저장한다.
2. 가장 오른쪽 노드에 저장된 값을 출력한다.



```
def make_tree():
```

```
    num=[37,29,69,30,15,55,90,87,73,20]
```

```
    in_d=0
```

```
    root=Node(num[0])
```

```
    for i in range(1,len(num)):
```

```
        v = num[i]
```

```
        w_node = root
```

```
        while w_node != None:
```

```
            if w_node.data < v: # right
```

```
                if w_node.right == None: # 끝이면 추가
```

```
                    child = Node(v)
```

```
                    w_node.right = child
```

```
                    w_node=None
```

```
                else:
```

```
                    w_node = w_node.right
```

```
            else: # left
```

```
                if w_node.left == None: # 끝이면 추가 아니면 다음 노드로
```

```
                    child = Node(v)
```

```
                    w_node.left = child
```

```
                    w_node = None
```

```
                else:
```

```
                    w_node = w_node.left
```

```
def search_right_only(node,level,ans):
```

```
    ans.append(node.data)
```

```
    if node.right != None:
```

```
        search_right_only(node.right,level+1,ans)
```

트리
자료구조
알고리즘

문제해결

1 규칙기반 자료구조와 알고리즘

❖ 자료구조와 규칙기반 자료구조

- 기본 자료 형 / 기본 자료구조
- 규칙기반 자료구조 – 확장 형 자료구조 - 스택, 큐, 트리, 그래프

❖ 알고리즘

- 문제중심의 알고리즘
 - 탐색 문제
 - 정렬 문제
 - 최단거리문제
- 문제 해결방식에 따른 알고리즘
 - 분할 방법
 - 그리디 방법
 - 동적 방법

2 규칙기반 자료구조

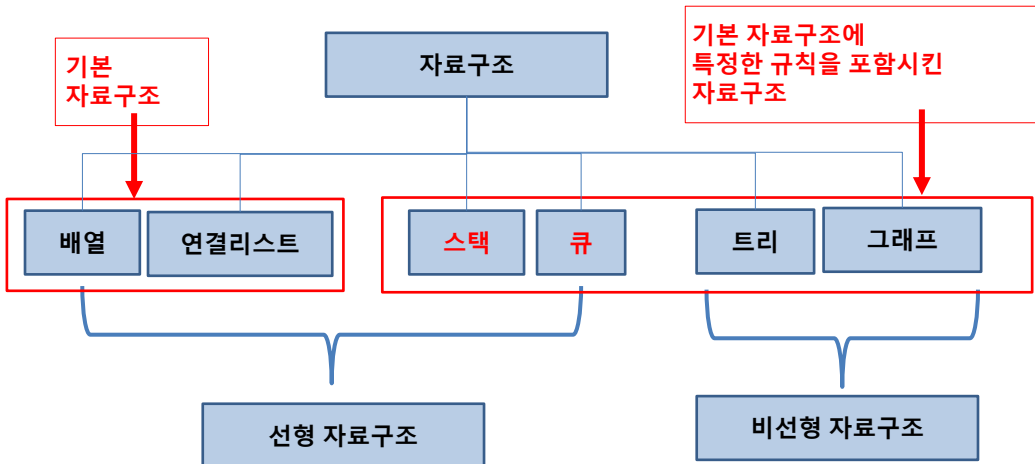
❖ 자료구조에는 효율적인 데이터 관리 (검색/삽입/삭제/수정) 를 위하여 저장 규칙이 필요.

❖ 규칙기반 자료구조는 ‘특정문제’ 를 자료구조 기반으로 해결한 것. → 문제의 특성에 따라 선택하여 문제해결에 이용

❖ 규칙기반 자료구조를 이용하면 특정 문제해결에 도움을 줄 수 있음.

- 괄호 개수 확인 문제 (대칭 문제) – 스택 자료구조
- 콜센터 고객 대기시간 관리 문제 (순서관리 문제) – 큐 자료구조
- 데이터들의 순서와 상관없는 비 선형 문제 – 트리 자료구조, 그래프 자료구조

2 규칙기반 자료구조



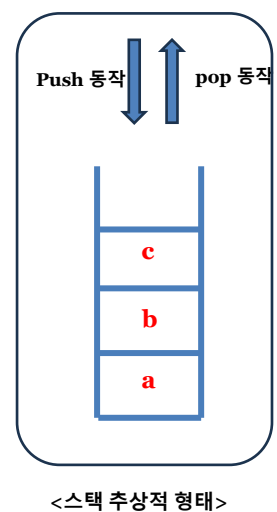
3 스택 (STACK)

❖ 규칙

- Last In First Out (LIFO)
- 가장 마지막에 데이터 저장 (삽입) – push 동작
- 가장 마지막 데이터를 읽음 (검색, 삭제, 수정) – pop 동작

❖ 프로그래밍 언어로 표현하는 방법

- 배열 자료구조를 이용한 스택 구현
- 연결 리스트 자료구조를 이용한 스택 구현
- 각 프로그래밍 언어에서 제공하는 자료구조를 이용하여 구현
 - 파이썬 : 리스트를 이용한 스택



3 스택 (STACK)

❖ 괄호확인 문제

- `print("계산결과는{o}입니다.".format((int(a)+int(b))))`
- (스택 자료구조를 이용하여 문제해결 – 데이터 저장구조를 이용하여 해결)
 - 왼쪽 괄호를 만나면 스택에 PUSH() 한다.
 - 오른쪽 괄호를 만나면 스택에서 POP() 한다.
 - 스택에 데이터가 없으면, 괄호의 개수가 맞는 것이다.
- (알고리즘을 이용하여 문제해결 – 데이터 저장구조를 이용하지 않고 해결)
 - 왼쪽 괄호를 만나면 괄호카운트를 하나 증가시킨다.
 - 오른쪽 괄호를 만나면 괄호카운트를 하나 감소시킨다.
 - 괄호카운트가 0이면 괄호의 개수가 맞는 것이다.

```
a_count=[]
def push(push_data):
    a_count.append(push_data)

def pop():
    a_count.remove(a_count[-1])

a=input()
for i in a:
    if i == '(':
        push(i)
    elif i == ')':
        pop()

if len(a_count)==0:
    print('OK')
else:
    print('error')
```

```
a=input()
open_count=0
for i in a:
    if i == '(':
        open_count+=1
    elif i == ')':
        open_count-=1

if open_count==0:
    print("ok")
else:
    print("error")
```

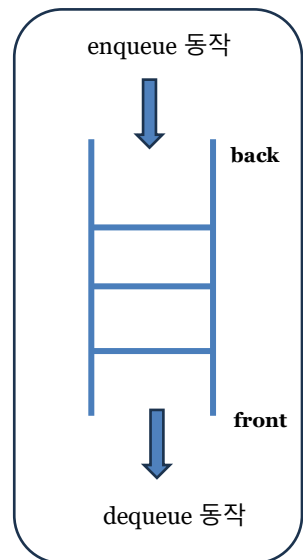
4 큐 (Queue)

❖ 규칙

- First In First Out (FIFO)
- 가장 마지막에 데이터 저장 (삽입) – enqueue 동작
- 가장 처음 저장된 데이터를 읽음 (검색, 삭제, 수정) – dequeue 동작

❖ 프로그래밍 언어로 표현하는 방법

- 배열 자료구조를 이용한 큐 구현
 - 연결 리스트 자료구조를 이용한 큐 구현
 - 각 프로그래밍 언어에서 제공하는 자료구조를 이용하여 구현
- 파이썬 : 리스트를 이용한 큐



<큐 추상적 형태>

4 큐 (Queue)

❖ 은행업무 프로그램 (콜센터 고객 대기시간 관리 문제)

- 대기표를 뽑으면 큐의 가장 마지막에 대기표 번호 삽입
- 은행 업무순서 버튼을 누르면 큐의 가장 앞에 저장된 대기표 번호 출력 (큐에서는 삭제)

```

v_no=100
queue=[]

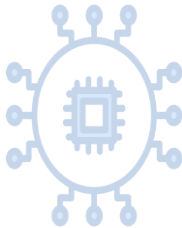
def enqueue(data):
    queue.append(data)

def dequeue():
    dequeue_object=None
    if len(queue)==0:
        print("Queue is Empty")
    else:
        dequeue_object=queue[0]
        queue.remove(queue[0])
        return dequeue_object

pop() 함수대체가능 →

while True:
    a=int(input("대기표 뽑기:1,업무처리:2,종료:3\n"))
    if a==1:
        enqueue(v_no)
        v_no+=1
        print("현재 대기표번호:",v_no)
    elif a==2:
        b=dequeue()
        print("현재 업무처리번호:",b)
    else:
        break
    
```

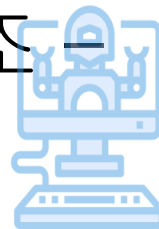
대기표 뽑기:1,업무처리:2,종료:3
 1 현재 대기표번호: 101
 대기표 뽑기:1,업무처리:2,종료:3
 1 현재 대기표번호: 102
 대기표 뽑기:1,업무처리:2,종료:3
 1 현재 대기표번호: 103
 대기표 뽑기:1,업무처리:2,종료:3
 2 현재 업무처리번호: 100
 대기표 뽑기:1,업무처리:2,종료:3
 2 현재 업무처리번호: 101
 대기표 뽑기:1,업무처리:2,종료:3
 1 현재 대기표번호: 104
 대기표 뽑기:1,업무처리:2,종료:3
 2 현재 업무처리번호: 102
 대기표 뽑기:1,업무처리:2,종료:3
 3



SW중심대학사업 교재공동개발

8. 규칙기반 자료구조 트리 그래프

Computational Thinking

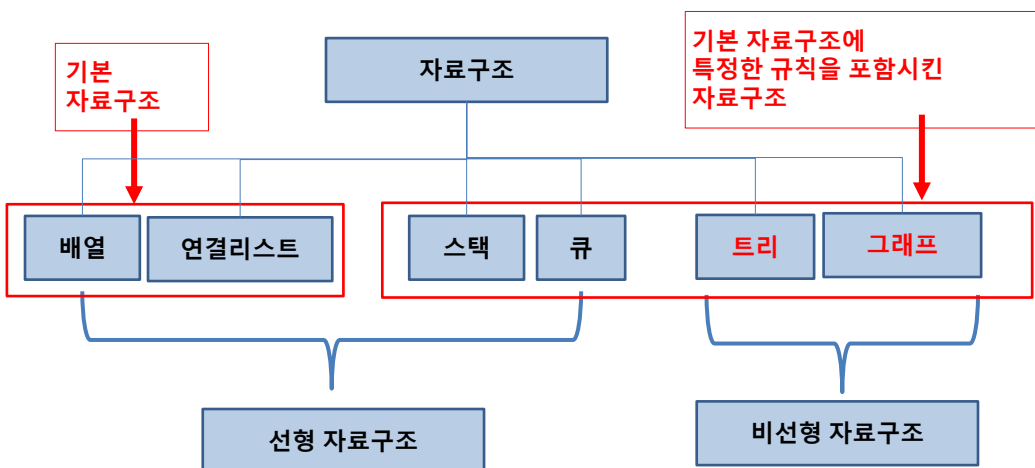


0 목차

➤ 그래프

➤ 트리

0 규칙기반 자료구조



1 그래프 (GRAPH)

❖ 그래프 이론

- 그래프 이론의 최초의 연구 : 레온하르트 오일러의 논문 - 쾨니히스베르크의 다리 문제
- 쾨니히스베르크의 다리 문제
- “7개의 다리는 한 번씩만 건너서 쾨니히스테르크의 모든 지역을 돌아 제자리로 돌아 올 수 있을까?”

❖ 논문 결과

- “모든 다리를 한번만 거치고 첫 시작 위치로 돌아오는 길은 없다”
- 이 문제의 해답을 찾기 위한 연구의 부산물로 그래프 이론이라는 수학의 한 분야가 생겨남
- 논문에 포함된 이 스케치는 현대에 이르러 그래프 구조의 원형이 됨
- 그래프는 수 백년 동안 수학, 과학, 공학, 사회, 경제학에 이르기까지 폭넓은 분야에서 응용됨
- 컴퓨터 분야에서도 그래프 이론을 토대로 한 자료구조가 만들어짐

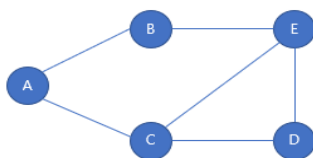
1 그래프 (GRAPH)

❖ 그래프 정의

- 그래프는 정점과 간선의 모음 들의 결합
- 정점의 집합을 V , 간선의 집합을 E , 그래프를 G 라고 했을 때 $G=(V,E)$

❖ 그래프 용어

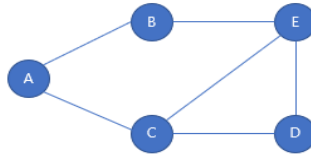
- 인접 (adjacent) : 간선으로 연결되어 있는 두 정점 , 이웃 관계에 있다고 말할 수 있다.
→ $(A,B), (A,C), (B,E), (C,E), (C,D), (D,E)$
- 경로 (Path) : 간선을 통해 서로 이웃 된 각 정점들로 그래프 안에서 형성된 길
→ ‘정점 A에서 E 까지는 A,B,E 와 A,C,E 두 개의 경로를 이루고 있다.’



1 그래프 (GRAPH)

❖ 그래프 용어

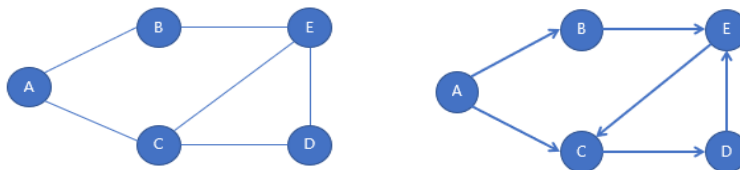
- 차수 (degree) : 정점에 속한 간선들의 수를 말한다.
→ 정점 A,B,D의 차수는 2 이고, 정점 E,C 의 차수는 3
- 연결 (Connected) : 무 방향 그래프 내에서 두 정점 사이의 경로가 존재하면 두 정점이 연결 되었다고 말한다. 그래프 내의 모든 정점들이 모두 연결 되어 있으면 그래프가 연결되었다고 말한다.
- 사이클 (Cycle) : 어느 경로가 정점 하나를 두 번 이상 거치도록 되어 있는 경로, 처음과 마지막 정점이 같은 단순 경로를 말한다.



1 그래프 (GRAPH)

❖ 그래프 종류

- 무방향 그래프 (Undirected graph) : 간선에 방향성이 없는 그래프, 간선을 나타내는 정점의 쌍에 순서가 없다.
→ (A,B)와 (B,A)는 동일한 간선을 표현
- 방향 그래프 (directed graph) : 간선에 방향성이 있는 그래프, 간선을 나타내는 정점의 쌍에 순서가 있다.
→ <A,B> 로 표현하며, 이때 A는 꼬리(Tail), B는 머리(Head)로 표현하기도 한다.

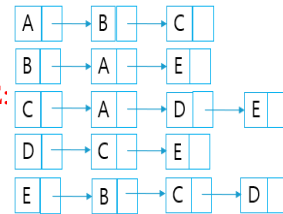
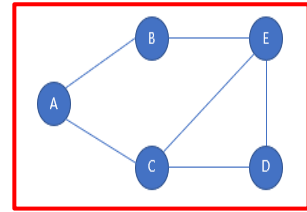


1 그래프 (GRAPH)

❖ 프로그래밍 언어로 그래프 표현

- 정점 / 간선 저장
- 연결 리스트 기본 자료구조 형식을 이용하여 그래프를 표현
 - 정점 중심 표현
 - 파이썬으로 저장 표현(딕셔너리 자료형):

→ `graph_array={A:[B,C],B:[A,E],C:[A,D,E],D:[C,E],E:`

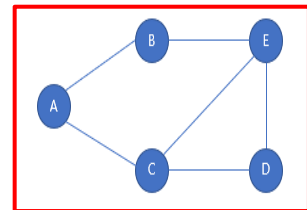


1 그래프 (GRAPH)

❖ 프로그래밍 언어로 그래프 표현

- 정점 / 간선 저장
- 배열 기본 자료구조 형식을 이용하여 그래프를 표현
 - 간선 중심 표현
 - 파이썬으로 저장 표현(이중리스트 자료형/ 딕셔너리 자료형):

→ `graph_array=[[0,1,1,0,0],[1,0,0,0,1],[1,0,0,1,1],[0,1,0,0,1],[0,1,1,1,0]]`
 → `graph_array={A:[B,C],B:[A,E],C:[A,E],D:[C,E],E:[B,C,D]}`



	A	B	C	D	E
A	0	1	1	0	0
B	1	0	0	0	1
C	1	0	0	1	1
D	0	0	1	0	1
E	0	1	1	1	0

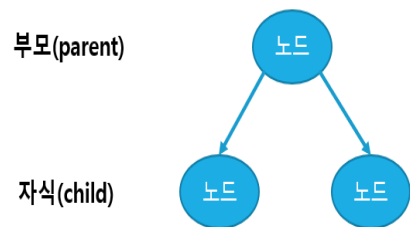
2 트리 (TREE)

❖ 트리

- 노드(정점)와 간선으로 구성
- 그래프의 일종으로 여러 노드가 한 노드를 가리킬 수 없는 구조
- 한 방향 그래프
- 부모 자식 노드로 구성 (자식 노드는 하나의 부모 노드만 갖는다.)

❖ 트리 종류

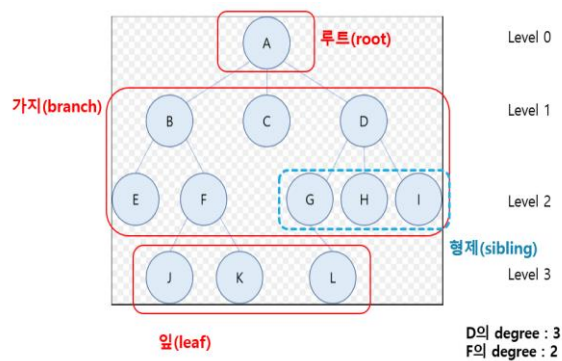
- B트리, 이진트리, 완전이진트리, 균형이진트리



2 트리 (TREE)

❖ 트리 용어

- 노드 : 루트노드, 부모노드, 자식노드, 가지노드, 잎노드, 형제노드
- 간선 : 깊이 (Level), 차수 (degree)



2 트리 (TREE)

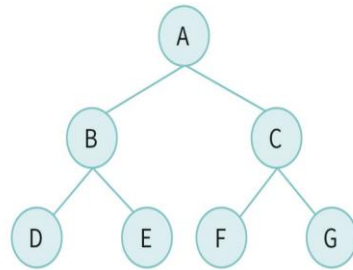
❖ 프로그래밍 언어로 트리 표현

- 연결 리스트 기본 자료구조 형식을 이용하여 그래프를 표현
- 배열 기본 자료구조 형식을 이용하여 그래프를 표현
- 파이썬으로 저장 표현(딕셔너리 자료형):

→ `tree_array={A:[B,C],B:[D,E],C:[F,G]}`

→ `p_node_array=[A,B,C]`

→ `c_node_array=[[B,C],[D,E],[F,G]]`



3 그래프 와 트리

❖ 그래프와 트리 특징 비교

	그래프	트리
정의	노드와 그 노드를 연결하는 간선을 하나로 모아 놓은 자료구조	그래프의 한 종류
방향성	방향/무방향 그래프 모두 존재	방향 그래프
사이클	사이클 가능 순환/비순환 그래프 모두 가능	사이클 불가능 비순환 그래프
루트 노드	루트 노드 개념이 없음	한 개의 루트 노드만 존재
부모-자식	부모-자식 개념 없음	부모-자식 관계
모델	네트워크 모델	계층모델
간선의 수	그래프에 따라 간선의 수가 다름 간선이 없을 수도 있음	노드가 N인 트리는 항상 N-1의 간선을 가짐

3 그래프 와 트리

❖ 그래프 자료구조를 이용한 문제해결 예

- 최단거리 문제

→ 프림 알고리즘, 크러스컬 알고리즘, 다익스트라 알고리즘

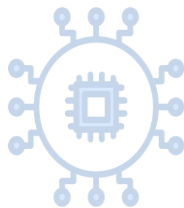
❖ 그래프 자료구조를 이용한 문제해결 예

- 탐색 문제

→ 이진탐색 알고리즘

- 정렬 문제

→ 힙정렬 알고리즘



SW중심대학사업 교재공동개발

9. 알고리즘 성능

Computational Thinking



0 목차

- 컴퓨터 알고리즘
- 컴퓨터 알고리즘 성능
- 알고리즘 성능 평가 – 시간 복잡도

1 컴퓨터 알고리즘

❖ 알고리즘

- 문제를 해결하기 위한 절차를 공식화한 형태로 표현하는 것

❖ 컴퓨터 알고리즘

- 문제를 **컴퓨터를 통하여 해결**하고자 할 때 컴퓨터가 처리할 일 또는 명령어의 순서
- 문제가 해결되었다 → 알고리즘을 찾았다 → 컴퓨터가 알고리즘대로 수행하면 컴퓨터에 의해 원하는 답을 얻을 수 있다.
- 문제를 컴퓨터를 이용해 해결하기 위한 절차를 공식화한 형태로 표현하는 것
- 문제에 대한 알고리즘을 컴퓨터에 적용하여 컴퓨터를 통해 문제 해결의 결과를 얻기 위하여 프로그래밍 언어로 표현
- 프로그래밍 언어로 표현된 알고리즘을 컴퓨터로 실행

1 컴퓨터 알고리즘

❖ 유클리드(Euclid) 최대공약수 알고리즘

- 기원전 300년 경에 만들어진 알고리즘
 - 두 개 이상의 수가 공통으로 가지고 있는 약수 중 가장 큰 수
- 두 수의 최대공약수는 큰 수를 작은 수로 나눈 나머지와 갓은 수와의 최대 공약수가 같다.
- 컴퓨터로 문제해결 → 컴퓨터 알고리즘으로 표현 (저장/연산)

알고리즘

1. 두수를비교하여큰수를찾는다.
2. 큰수를작은수로나누어나머지를구한다.
3. 만약에나머지가0이면작은수가최대공약수
4. 만약에나머지가0이아니면다시나누기반복
5. 27과12의최대공약수찾기

```
27>12
27/12 ... 나머지 3
3≠ 0
12/3 ... 나머지 0
따라서 최대 공약수는 3
```

```
a=int(input())
b=int(input())
min_num=0
max_num=0
```

```
if a>b:
    min_num=b
    max_num=a
else:
    min_num=a
    max_num=b
```

```
while True:
    c=max_num%min_num
    if c==0:
        print("최대공약수:",min_num)
        break
    else:
        max_num=min_num
        min_num=c
```

1 컴퓨터 알고리즘

❖ 컴퓨터 알고리즘의 요건

- 입력 : 0개 이상의 외부 입력
- 출력 : 1개 이상의 출력
- 정확성 : 주어진 입력에 대해 올바른 해를 주어야 함
- 유한성 : 일정한 시간 내에 반드시 종료되어야 함
- 수행성 : 모든 명령은 컴퓨터에서 수행 가능해야 함
 - 모든 입력 값 N 에 대해
 - 유한 시간 내에
 - 정확한 결과

→ 알고리즘의 효율성 → 알고리즘 성능

2 컴퓨터 알고리즘 성능

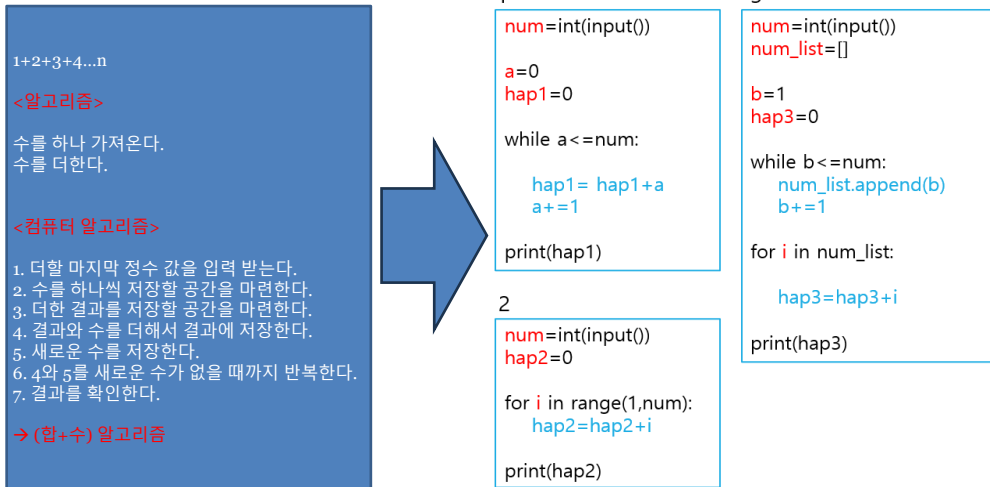
- ❖ 컴퓨터 알고리즘을 컴퓨터로 실행할 때 드는 비용
- ❖ 비용 단위 : 시간 / 메모리
- ❖ 알고리즘의 복잡도로 알고리즘의 성능 평가
 - **입력 크기**에 대한 알고리즘 **수행 시간 (연산의 횟수)**
 - **기본연산** : 비교, 데이터 읽기, 데이터 갱신, 숫자계산 등의 **단순연산**
 - **입력 과 연산 할 때 메모리 사용 공간**

2 컴퓨터 알고리즘 성능

- ❖ 알고리즘 복잡도
 - **시간 복잡도 (Time Complexity)** : ‘시간’ 알고리즘을 위해 필요한 **연산의 횟수**
 - **공간 복잡도 (Space Complexity)** : ‘크기’ 알고리즘을 위해 필요한 **메모리 양**
- ❖ 알고리즘의 복잡도는 주로 시간 복잡도를 의미

3 컴퓨터 알고리즘 평가 - 시간 복잡도

❖ 1부터 n 까지 정수의 합 구하기



3 컴퓨터 알고리즘 평가 - 시간 복잡도

❖ 1부터 n 까지 정수의 합 구하기

< 1번 표현방법 >

```
num=int(input())

a=0
hap1=0

while a<=num:

    hap1= hap1+a
    a+=1

print(hap1)
```

메모리 : 변수 num,a,hap
시간 : 계산 횟수
비교 연산 1회 * 데이터의 개수
더하기 연산 2회 * 데이터의 개수

< 2번 표현방법 >

```
num=int(input())
hap2=0

for i in range(1,num):
    hap2=hap2+i

print(hap2)
```

메모리 : 변수 num,hap2,i
시간 : 계산 횟수
i 값 갱신 연산 1회 * 데이터의 개수
더하기 연산 1회 * 데이터의 개수

< 3번 표현방법 >

```
num=int(input())
num_list=[]

b=1
hap3=0

while b<=num:
    num_list.append(b)
    b+=1

for i in num_list:

    hap3=hap3+i

print(hap3)
```

메모리 : 변수 num,num_list,b,hap3
시간 : 계산 횟수
비교 연산 1회 * 데이터의 개수
더하기 연산 2회 * 데이터의 개수
i, append() 갱신 연산 * 데이터의 개수

3 컴퓨터 알고리즘 평가 - 시간 복잡도

❖ 1부터 n 까지 정수의 합 구하기

```
num=int(input())
a=0
hap1=0
while a<=num:
    hap1= hap1+a
    a+=1
print(hap1)
```

n=1 수행횟수 = 3x1회
n=2 수행횟수 = 3x2=4
n=3 수행횟수 = 3x3=6
n=2 수행횟수 = 3x4=8
...
n=n 수행횟수 = 3xn=3n

시간 복잡도 함수: $3n$
→ 알고리즘 성능 : $O(n)$

```
num=int(input())
num_list=[]
b=1
hap3=0
while b<=num:
    num_list.append(b)
    b+=1
for i in num_list:
    hap3=hap3+i
print(hap3)
```

n=1 수행횟수 = 3x1+2x1
n=2 수행횟수 = 3x2+2x2
n=3 수행횟수 = 3x3+2x3
n=2 수행횟수 = 3x4+2x4
...
n=n 수행횟수 = 3xn+2xn=5n

시간 복잡도 함수: $5n$
→ 알고리즘 성능 : $O(n)$

3 컴퓨터 알고리즘 평가 - 시간 복잡도

❖ 최선의 경우 (Best case)

- 최선의 상황에서 작업 완료 시까지 **가장 빠른 시간**을 계산

❖ 최악의 경우 (Worst case)

- 최악의 상황에서 작업 완료 시까지 **가장 느린 시간**을 계산

❖ 평균의 경우 (Average case)

- 여러 상황에서 '총 실행시간/시행 횟수'를 계산

→ 알고리즘 성능 평가 시 최악의 경우 성능을 사용

3 컴퓨터 알고리즘 평가 - 시간 복잡도

❖ 빅 오 표기법 (Big - O)

- 알고리즘의 시간 복잡도를 표현할 때 사용하는 표기법
- 최악의 경우를 표현하는 표기법
- 함수의 상한 만을 표현
- 대충 표기법

❖ 시간 복잡도 함수 예

입력 데이터수 n 에 대한 수행횟수 증가 함수

N=1 수행횟수= $3 \times 1^2 + 1 + 3 = 7$ (3)
 N=2 수행횟수= $3 \times 2^2 + 2 + 3 = 17$ (12)
 N=3 수행횟수= $3 \times 3^2 + 3 + 3 = 33$ (27)
 N=4 수행횟수= $3 \times 4^2 + 4 + 3 = 55$ (48)
 N=5 수행횟수= $3 \times 5^2 + 5 + 3 = 83$ (75)
 ...



$$3n^2 + n + 3 \rightarrow O(n^2)$$

3 컴퓨터 알고리즘 평가 - 시간 복잡도

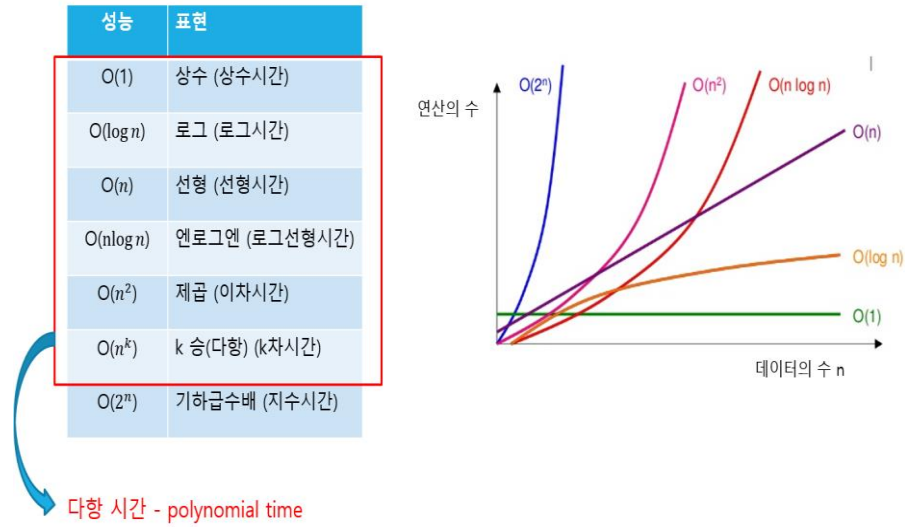
❖ 빅 오 표기에 따른 알고리즘 성능의 의미

성능	의미	표현
$O(1)$	입력 데이터의 크기와 상관없이 복잡도가 동일하게 유지	상수 (상수시간)
$O(\log n)$	입력 데이터의 수 로그에 비례 (입력 데이터 중에 반 이상이 실행하지 않음)	로그 (로그시간)
$O(n)$	입력 데이터의 수에 정비례	선형 (선형시간)
$O(n \log n)$	입력 데이터의 수와 입력 데이터의 수 로그 곱에 비례	엔로그엔 (로그선형시간)
$O(n^2)$	입력 데이터의 수 제곱에 비례 (데이터 하나당 제공만큼의 연산)	제곱 (이차시간)
$O(n^k)$	입력 데이터의 수 2^k 에 비례	k 승(다항) (k차시간)
$O(2^n)$	입력 데이터의 수 2^n 에 비례	기하급수배 (지수시간)

n	n^2	n^3	2^n
1	1	1	2
2	4	8	4
3	9	27	8
4	16	64	16
5	25	125	32
6	36	216	64
7	49	343	128
8	64	512	256
9	81	729	512
10	100	1000	1,024
20	400	8000	1,048,576
30	900	27000	1,073,741,824

3 컴퓨터 알고리즘 평가 - 시간 복잡도

❖ 빅 오 표기에 따른 알고리즘 성능의 의미



3 컴퓨터 알고리즘 평가 - 시간 복잡도

❖ 10억개의 숫자를 정렬

- $O(n^2)$ 알고리즘은 약 300년이 걸림
- $O(n \log n)$ 알고리즘은 5분 걸림

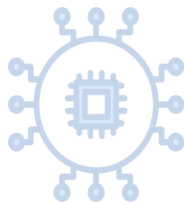
PC에서 실행	1,000개	1백만 개	10억 개
$O(n^2)$	1초 미만	2시간	300년
$O(n \log n)$	1초 미만	1초미만	5분

- 현실적인 입력 데이터 수
- 하드웨어 성능

3 컴퓨터 알고리즘 평가 – 시간 복잡도

❖ 대표 컴퓨터 알고리즘의 성능

실행비용	알고리즘 예
$O(1)$	배열의 n 번째 요소에 접근 스택 자료구조에서 PUSH/POP 동작
$O(\log n)$	이진 탐색 알고리즘
$O(n)$	연결 리스트 자료구조에서 데이터 순회 최대/최소값 찾기 알고리즘
$O(n \log n)$	퀵 정렬 알고리즘 힙 정렬 알고리즘 합병 정렬 알고리즘
$O(n^2)$	버블 정렬 알고리즘 선택 정렬 알고리즘 삽입 정렬 알고리즘



SW중심대학사업 교재공동개발

10. 문제의 이해

Computational Thinking



0 목차

- 문제의 분류
- NP- 완전문제

1 문제의 분류

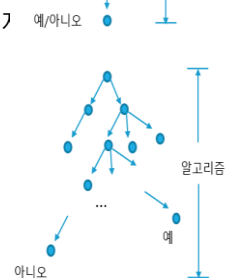
❖ 결정문제 (Deterministic Problem)

- 계산이론에서 어떤 형식 체계에서 예-아니오 답이 있는 질문
- 현재상태에서 다음 상태로 변할 때 다음 상태가 유일하게 결정
- 튜링기계기반 해결되는 문제



❖ 비 결정문제 (Non-deterministic Problem)

- 현재상태에서 다음 상태로 변할 때 다음 상태가 유일하게 결정되지 않음
- 상황에 따라 다름
- 여러 개 이거나 없을 수도 있음
- 계산 경로가 트리 형태
- 문제 해결이 보장되지 않음 (좋은 선택을 했을 때 좋은 결과)



1 문제의 분류

❖ 문제

- 컴퓨터로 해결하고자 하는 모든 문제
- 결정 문제를 기반으로 하는 문제
- 비 결정 문제를 컴퓨터로 해결하기 위한 문제해결의 조건 재정의

❖ 문제의 해결

- 모든 입력 값 N 에 대해 → 현실 입력 값에 대해
- 유한 시간 내에 (다항 시간 성능 알고리즘) → 알고리즘의 복잡도 계산 (다항 시간 성능)
- 정확한 결과 → 근사값, 확률값, 확인문제 전환
 - ✓ 정확한 값을 도출하기 위한 인간의 경험치 도입(휴리스틱)
 - ✓ 좋은 선택을 하면 정답 : 정답율을 높이기 위해 좋은 선택을 하도록 함
 - ✓ 정확한 답이 아닌 근사치로 결과값을 도출
 - ✓ 답을 확인하는 시간이 다항 시간

1 문제의 분류

❖ P (Polynomial) 문제

- 쉬운 문제
- 다항식 시간에 해결되는 문제
- $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^k)$

❖ NP (Non-deterministic Polynomial) 문제

- 다항식 시간에 해결되는 비 결정 문제
- 휴리스틱한 방법/ 근사값 결과/정답 확인 등의 방법으로 다항식 시간에 해결되는 문제
- NP-Hard (NP-하드) 문제 / NP-Complete (NP-완전) 문제

2 NP-완전문제

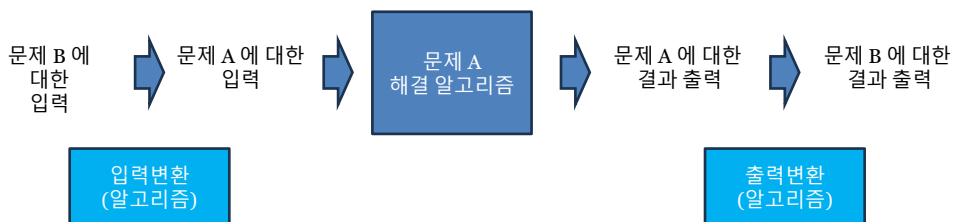
❖ NP 완전문제 (NP-Complete Problem)

- 어떤 NP 문제에 대해서 다항식 시간에 해를 구할 수 있으면, 문제변환으로 다른 모든 NP 문제들도 다항식 시간에 해를 구할 수 있는 문제
- NP 문제 중 가장 어려운 문제, 종결자 문제
- 1971년 스티븐 쿡(Stephen Cook)
 - NP-완전 개념을 확립
 - SAT(Satisfiability) 문제 (논리식이 참일 수 있는지 결정하는 문제)가 NP-complete 문제임을 밝힘
- 알려진 NP-완전 문제는 수천 가지

2 NP-완전문제

❖ NP 완전문제 (NP-Complete Problem)

- 문제의 변환



B 문제의 시간 복잡도 = 입력변환 알고리즘 시간 복잡도 + 문제 A 알고리즘 시간 복잡도 + 출력변환 알고리즘 시간 복잡도
= 다항식 시간 + 다항식 시간 + 다항식 시간
= $O(\log n) + O(n^2) + O(n \log n) \rightarrow O(n^2)$

2 NP-완전문제

❖ NP 하드 (NP-Hard Problem)

- NP- 완전문제는 NP-하드 문제 이다.

❖ P, NP, NP-hard, NP-Complete 의 관계

- $\forall P \in NP \rightarrow$ 사실
- $\forall NP \notin P \text{ or } \forall NP \in P \rightarrow$ 증명되지 않음
- $P = NP \text{ or } P \neq NP \rightarrow$ 증명되지 않음
- 현재 튜링기계에 기반한 컴퓨터 세계에서는
 $P \neq NP$ 로 추측



3 문제의 예와 알고리즘

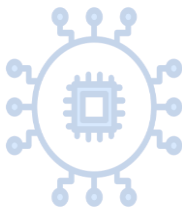
❖ P 문제 예 - 해결된 문제

- 탐색 문제
 - 순차 탐색 알고리즘, 이진 탐색 알고리즘 등
 - 경험적 탐색 알고리즘 (최적 해 탐색 $\rightarrow$ NP문제)
 - 데이터 검색
- 정렬 문제
 - 삽입 정렬 알고리즘, 선택 정렬 알고리즘, 버블 정렬 알고리즘
 - 합병 정렬 알고리즘, 퀵 정렬 알고리즘, 힙 정렬 알고리즘
- 압축 문제
 - 허프만 압축 알고리즘

3 문제의 예와 알고리즘

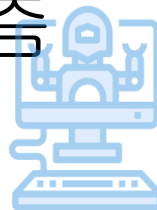
❖ NP 문제 예 - 해결되지 않은 문제

- SAT (Satisfiability) 문제
 - 충족 가능성 문제 : 변수들로 이루어진 논리식에서 논리식이 참이 되는 변수 값이 존재하는지 찾는 문제
 - $(X \text{ and } Y) \text{ or } (X \text{ and } Y \text{ and } Z)$ 를 참으로 만드는 X, Y, Z 을 찾는 문제 (존재 하는가? 무엇인가?)
 - 키를 i80이 되도록 하는 음식이 있을까? (시금치 and 소고기) or (양파 and 돼지고기 and 마늘)
 - 어떤 조건을 만족하면 주식이 오를까?
- 해밀턴 경로 문제
 - 주어진 모든 지점을 한번씩만 방문하고 제자리로 돌아오는 경로 찾기 문제
- TSP (Traveling Salesman Problem) 문제
 - 최소한의 비용으로 주어진 모든 지점을 순회하고 돌아오는 경로 찾기 문제
 - 최단 경로 찾기 문제
- 0-1배낭 문제
 - 조합 최적화 문제
 - 배낭에 담을 수 있는 최대 무게가 정해져 있을 때, 배낭에 담을 물건들의 가치가 최대가 되도록 부분 집합을 찾는 문제



SW중심대학사업 교재공동개발

11. P문제와 알고리즘



Computational Thinking

0 목차

➤ 탐색문제

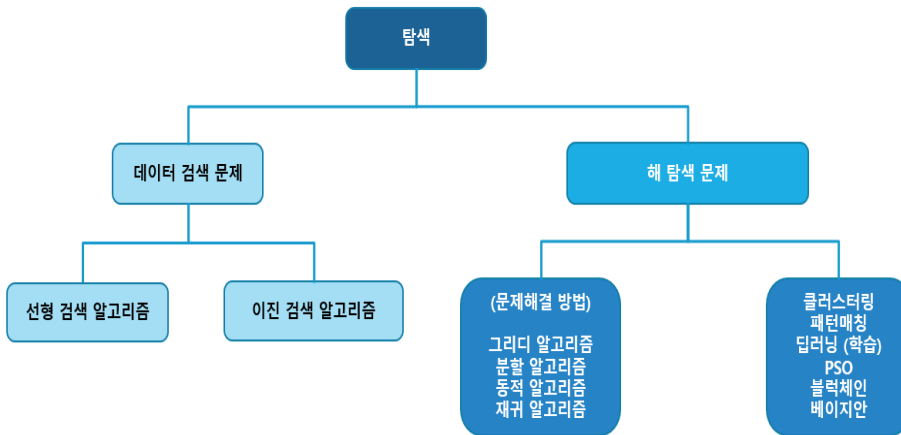
➤ 정렬문제

1 탐색문제

- ❖ 탐색 문제는 방대한 데이터에서 목적에 맞는 데이터를 찾아내기 위한 알고리즘이다.
- ❖ 탐색 문제는 탐색 뿐만 아니라, 데이터의 삽입, 삭제 연산 기능을 위해서도 사용되므로, 탐색 뿐만 아니라 빈번히 이루어 지는 삽입과 삭제 동작의 성능에도 영향을 미친다.
 - 컴퓨터 과학에서 탐색은 프로그램 성능에 큰 영향을 미치므로 효율적인 탐색 방법이 필요
- ❖ 검색 알고리즘 : 데이터 검색 → P 문제
- ❖ 탐색 알고리즘 : 해(근사값) 탐색 → NP 문제

1 탐색문제

❖ 탐색 문제의 분류와 알고리즘



검색 문제 (P문제) 해결

해 탐색 문제 (NP문제) 적용

1 탐색문제

❖ 데이터 검색 문제를 해결한 알고리즘

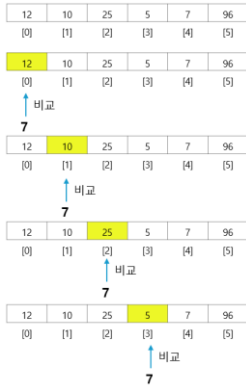
- 선형 검색 알고리즘
 - 순차탐색 = 순차검색 = 선형탐색 = 선형검색
 - 가장 간단한 검색 방법 / 전수검사
 - 기본 연산 : 비교연산

선형 검색 기본 알고리즘

데이터를 하나 가져온다.
찾고자 하는 데이터와 비교한다.

1 탐색문제

❖ 선형 검색 알고리즘 성능



의사코드

저장데이터=[12,10,25,5,7,96]

인덱스←0

찾고자하는 값←7

while 인덱스 < 리스트 크기

if 리스트[인덱스]==찾고자하는 값

print ("탐색성공")

break

인덱스←인덱스+1

선형 검색 기본 알고리즘

데이터를 하나 가져온다.

찾고자 하는 데이터와 비교한다.

i=1 일때 1번

i=2 일때 1번

i=3 일때 1번

...

i=n일때 1번

→1+1+1+...+1 → n 번

→n → O(n)

1 탐색문제

❖ 데이터 검색 문제를 해결한 알고리즘

- 이진검색 알고리즘
 - 정렬된 데이터 기반 검색
 - 정렬 알고리즘이 필요
 - 규칙기반 자료구조 활용 가능
 - 기본연산 : 비교연산

이진 검색 기본 알고리즘

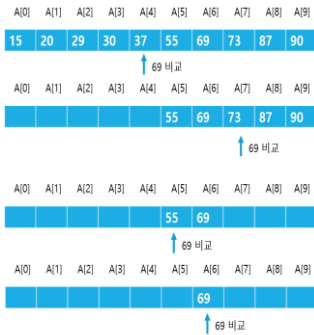
중간에 위치한 데이터를 가져온다.

찾고자 하는 데이터와 비교한다.

데이터 공간의 위치를 결정한다.

1 탐색문제

❖ 이진 검색 알고리즘 성능 - 기본 자료구조 기반



의사코드

```
정렬된리스트=[15,20,29,30,37,55,69,73,87,90]
찾고자하는 값=69
low←0
high←9
while low <= high
  middle←(low+high)/2
  if 찾고자하는 값==리스트[middle]
    print ("탐색성공")
    break
  elif 찾고자하는 값 > 리스트[middle]
    low←middle+1
  else
    high←middle-1
```

이진 검색 기본 알고리즘

중간에 위치한 데이터를 가져온다.
 찾고자 하는 데이터와 비교한다.
 데이터 공간의 위치를 결정한다.

n개 데이터 일 때 1번

$\frac{n}{2}$ 개 데이터 일 때 1번

$\frac{1}{2} \times \frac{n}{2}$ 개 데이터 일 때 1번

$\frac{1}{2} \times \frac{1}{2} \times \frac{n}{2}$ 개 데이터 일 때 1번

...

$((\frac{1}{2})^k \times n)$ 가 1일 때 마지막 1번

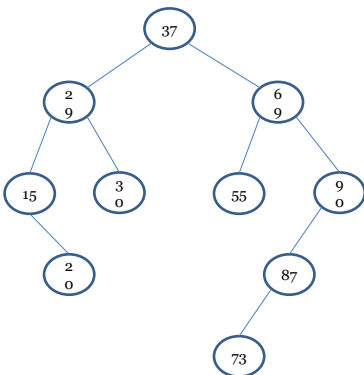
K: 수행 횟수

$\rightarrow (\frac{1}{2})^k \times n = 1$ (데이터가 1개남을때까지)

$\rightarrow k = \log_2 n \rightarrow O(\log n)$

1 탐색문제

❖ 이진 검색 알고리즘 성능 - 규칙기반 자료구조 (이진 트리 자료구조) 기반



의사코드

```
트리 구성 알고리즘
while 자식노드==0
  생성된 트리
if 부모노드 ==찾는데이터
  elif 부모노드 < 찾는데이터
    생성된 트리 ← 오른쪽 트리
  else:
    생성된 트리 ← 왼쪽 트리
```

일반자료구조사용시:

성능=정렬알고리즘성능+이진검색성능=
 $O(n) + O(\log n) \approx O(n)$
 (정렬알고리즘성능에따라달라질수있음)

이진트리자료구조사용시:

성능=트리구성성능+이진검색성능= $(n) +$
 $O(\log n) \approx O(n)$

예) 정렬된 데이터 $n = 1024$ 일때 $O(\log n)$ 알고리즘으로 수행하면,

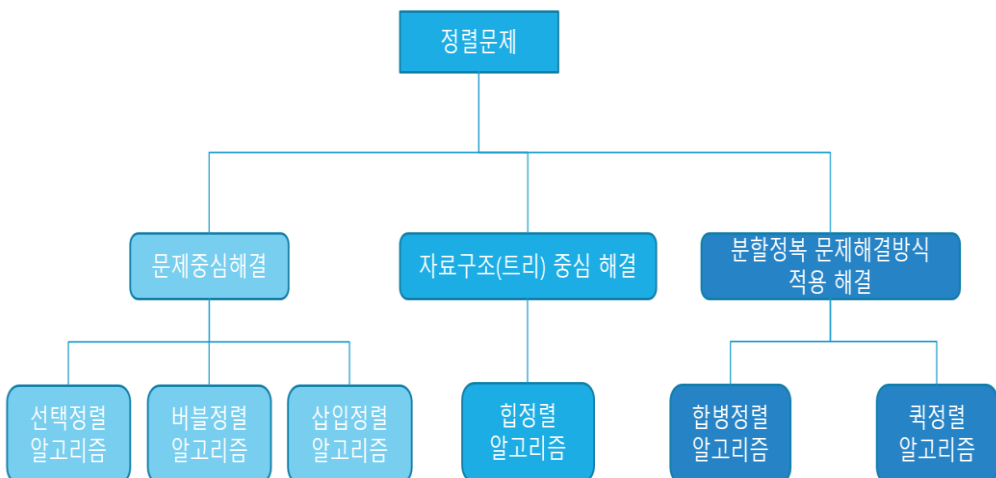
$\log 1024$ 가 수행횟수가 되며, $1024 = 2^{10}$ 이므로, 최악의 경우 10번만 수행하면 답을 찾을 수 있다.

2 정렬 문제

- ❖ N개의 숫자가 입력으로 주어졌을 때, 사용자가 지정한 기준에 맞게 순서를 찾는 문제
 - ❖ 검색 문제를 빠르게 해결하기 위하여 연구된 문제
 - ❖ 대표적인 P 문제로 컴퓨터로 해결된 문제
 - 모든 입력데이터에 대해
 - 일정한 시간 안에 (다항시간)
 - 정답(순서)을 찾음
- 알고리즘이 있다. 알고리즘을 찾았다. 문제가 해결되었다.
- ❖ 정렬 알고리즘의 기초 연산
 - 비교(선택)
 - 교환

2 정렬 문제

- ❖ 정렬 문제의 알고리즘



2 정렬문제

❖ 선택 정렬 알고리즘

- 기본 자료구조를 사용 – 배열, 연결 리스트
- 데이터 중에 가장 작은 값(큰 값)을 찾아 맨 앞에 보내는 방법
- 선택과 교환 연산
- 기본 알고리즘
 - 최소값을 '선택' 한다.
 - 가장 앞에 저장된 데이터와 '교환'한다.
 - '선택'과 '교환'을 반복한다. (가장 앞의 위치를 하나씩 바꾸어준다.)

index	0	1	2	3	4	5
value	40	70	60	30	10	50

최소값 선택

index	0	1	2	3	4	5
value	10	70	60	30	40	50

최소값 교환

2 정렬문제

❖ 선택 정렬 알고리즘 동작과 성능

1	index	0	1	2	3	4	5
	value	40	70	60	30	10	50
2	index	0	1	2	3	4	5
	value	10	70	60	30	40	50
3	index	0	1	2	3	4	5
	value	10	70	60	30	40	50
4	index	0	1	2	3	4	5
	value	10	30	60	70	40	50
5	index	0	1	2	3	4	5
	value	10	30	60	70	40	50
6	index	0	1	2	3	4	5
	value	10	30	40	70	60	50
7	index	0	1	2	3	4	5
	value	10	30	40	70	60	50
8	index	0	1	2	3	4	5
	value	10	30	40	50	60	70

▪ 프로그래밍 언어로 표현

```
arr=[40,70,60,30,10,50]
for i in range(len(arr)-1):
```

```
    small_idx = i
    for j in range(i + 1, len(arr)):
        if arr[small_idx] > arr[j]:
            small_idx = j
```

```
    arr[i], arr[small_idx] = arr[small_idx], arr[i]
```

```
print(arr)
```

선택

교환

▪ 선택 정렬 기본 알고리즘

최소값을 '선택' 한다.

가장 앞에 저장된 데이터와

'교환'한다.

'선택'과 '교환'을 반복한다.

입력 데이터가 n 개 일때

i=0 일때 (n-1)번 비교연산 +1번 교환연산

i=1 일때 (n-2)번 비교연산 +1번 교환연산

i=2 일때 (n-3)번 비교연산 +1번 교환연산

...

i=(n-1)일때 ((n-1)+1)=0번+1번 교환연산

→ ((n-1) + (n-2) + (n-3) + ... + 0) + (n-1)

→ $\frac{n(n-1)}{2} + (n-1) \rightarrow O(n^2)$

(10+9+8+...+0) == (0+1+2+...+10)

❖ 선택 정렬 알고리즘 은 입력상태와(정렬도,역정렬,랜덤) 상관없이 일정한 성능을 보임

2 정렬 문제

❖ 버블 정렬 알고리즘

- 기본 자료구조를 사용 – 배열, 연결 리스트
- 인접한 두 데이터를 비교하여 큰 수(작은 수)를 뒤로 보내는 방법(위로 이동 – 버블)
- 입력을 전체적으로 한번 처리하는 것을 패스(pass) 라고 정의
- 1 pass 후 가장 큰 수가 가장 아랫 쪽(가장 뒤)에 위치한다.
- 기본 알고리즘
 - 인접한 두 수를 ‘비교’한다.
 - 인접한 두 수를 ‘교환’한다.
 - ‘비교’와 ‘교환’을 반복한다.

index	0	1	2	3	4	5
value	40	70	60	30	10	50

인접한 두 수
비교/교환

2 정렬 문제

❖ 버블 정렬 알고리즘 동작과 성능

▪ 기본 알고리즘

인접한 두 수를 ‘비교’한다.
인접한 두 수를 ‘교환’한다.
‘비교’와 ‘교환’을 반복한다.

1 pass

index	0	1	2	3	4	5
value	40	70	60	30	10	50

index	0	1	2	3	4	5
value	40	60	70	30	10	50

index	0	1	2	3	4	5
value	40	60	30	70	10	50

index	0	1	2	3	4	5
value	40	60	30	10	70	50

index	0	1	2	3	4	5
value	40	60	30	10	50	70

▪ 프로그래밍 언어로 표현

arr=[40,70,60,30,10,50]

for i in range(len(arr)-1,0,-1):

for j in range(i):

if arr[j] > arr[j+1]:

arr[j],arr[j+1] = arr[j+1], arr[j]

비교
교환

i=(n-1) 일때 (n-1)번
i=(n-2) 일때 (n-2)번
i=(n-3) 일때 (n-3)번
...
i=0 일때 0번

→ (n-1) + (n-2) + (n-3) + ... + 0

→ $\frac{n(n-1)}{2} \rightarrow O(n^2)$

(10+9+8+...+0) == (0+1+2+...+10)

❖ 버블 정렬 알고리즘은 입력상태와(정렬도, 역정렬, 랜덤) 상관없이 일정한 성능을 보임

2 정렬 문제

❖ 삽입 정렬 알고리즘

- 기본 자료구조를 사용 – 배열, 연결 리스트
- 삽입 정렬 알고리즘은 정렬된 부분과 정렬이 되지 않은 부분으로 나누어 정렬하는 알고리즘
- 기본 알고리즘
 - 정렬 안된 부분의 첫번째 값을 선택한다.
 - 정렬 된 부분으로 삽입한다. (정렬 된 부분의 마지막 데이터 부터 비교하여 교환한다.)

index	0	1	2	3	4	5	6
value	40	60	70	50	10	20	30

정렬 된 부분

정렬 안된
부분

index	0	1	2	3	4	5	6
value	40	60	70	50	10	20	30

정렬 안된 부분의 처음데이터를
정렬 된 부분의 마지막 데이터와
비교/ 교환

2 정렬 문제

❖ 삽입 정렬 알고리즘 동작과 성능

index	0	1	2	3	4	5	6
value	40	60	70	50	10	20	30

index	0	1	2	3	4	5	6
value	40	60	70	50	10	20	30

index	0	1	2	3	4	5	6
value	40	60	70	50	10	20	30

index	0	1	2	3	4	5	6
value	40	60	50	70	10	20	30

index	0	1	2	3	4	5	6
value	40	50	60	70	10	20	30

index	0	1	2	3	4	5	6
value	40	50	60	70	10	20	30

▪ 프로그래밍 언어로 표현

arr=[40,60,70,50,10,20,30]

for i in range(1,len(arr)):

for j in range(i, 0,-1):

if arr[j-1] > arr[j]:

arr[j],arr[j-1]=arr[j-1],arr[j]

for i in range(1,len(arr)):

while i > 0 and arr[i] < arr[i-1]:

arr[i], arr[i-1] = arr[i-1], arr[i]

i -= 1

▪ 기본 알고리즘

정렬 안된 부분의 첫번째 값을
선택한다.

정렬 된 부분으로 삽입한다.

i=1 일때 1번

i=2 일때 2번

i=3 일때 3번

...

i=(n-1)일때 (n-1)번

→ 1+2+3+...+(n-1)

→ $\frac{n(n-1)}{2} \rightarrow O(n^2)$

(10+9+8+...+0) == (0+1+2+...+10)

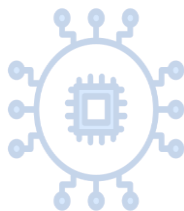
❖ 삽입 정렬 알고리즘 은 입력상태에 따라 수행시간이 달라짐

❖ 정렬도가 높은 경우엔 비교횟수가 작아져 $O(n)$ 성능을 보임

2 정렬문제

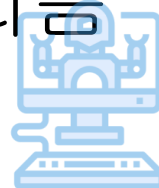
❖ 정렬 문제의 알고리즘 별 성능 비교

정렬알고리즘	최선	평균	최악	(60,000)개 데이터 정렬 시간(sec)
선택 정렬	n^2	n^2	n^2	10.842
버블 정렬	n^2	n^2	n^2	22.894
삽입 정렬	n	n^2	n^2	7.438
힙 정렬	$n \log_2 n$	$n \log_2 n$	$n \log_2 n$	0.034
합병 정렬	$n \log_2 n$	$n \log_2 n$	$n \log_2 n$	0.026
퀵 정렬	$n \log_2 n$	$n \log_2 n$	n^2	0.014



SW중심대학사업 교재공동개발

12. NP문제와 알고리즘



Computational Thinking

0 목차

- NP 완전 문제
- 그리디 문제해결 방법
- 분할정복 문제해결 방법
- 동적 문제해결 방법
- 알고리즘 성능

1 NP 완전 문제 (NP-Complete Problem)

- ❖ NP 문제 : 다항시간안에 최적 해를 찾는 비 결정 문제
- ❖ NP-완전 문제 해결 → 모든 NP 문제가 해결 → 다항시간안에 최적 해를 찾는 알고리즘을 찾음
- ❖ 해 탐색 문제, 최적 해 문제, 근사치 문제, 분류 문제 등
- ❖ NP완전문제
 - SAT 문제
충족 가능성 문제, 변수들로 이루어진 논리식에서 논리식이 참이 되는 변수 값이 존재하는지 찾는 문제
 - 해밀턴 경로 문제
주어진 모든 지점을 한번 씩만 방문하고 제자리로 돌아오는 경로를 찾는 문제
 - TSP 문제
최소한의 비용으로 주어진 모든 지점을 순회하고 돌아오는 경로를 찾는 문제
 - 0-1 배낭문제
배낭에 담을 수 있는 최대 무게가 정해져 있을 때, 배낭에 담을 물건들의 가치가 최대가 되도록 부분집합을 찾는 문제

1 NP 완전 문제 (NP-Complete Problem)

❖ NP 완전문제 해결 대표적인 방법

- 그리디 방법
크루스칼 알고리즘, 프림 알고리즘 (TSP 문제, 해밀턴 경로 문제)
- 분할-정복 방법
퀵정렬 알고리즘, 합병정렬 알고리즘 (정렬문제)
- 동적 방법
플로이드 워셜 알고리즘 (TSP 문제)

❖ NP 완전문제 해결 알고리즘 종류

- 휴리스틱 알고리즘, 근사치 알고리즘, 해 탐색 알고리즘, 최적화 알고리즘, 학습 알고리즘, 인공지능 알고리즘 등

2 그리디(Greedy) 문제해결 방법

❖ Greedy Algorithm

❖ 가장 좋은 값, 가장 나쁜 값, 최대값, 최소값을 욕심 내어 선택하는 방법

❖ 욕심쟁이 방법, 탐욕적 방법

❖ 특징

- 한번 선택한 것은 절대 바꾸지 않음
- 계속해서 정해진 기준으로 좋을 값 또는 나쁜 값 만을 선택하여 문제 해결
- 방법이 단순하여 문제에 적용하기 쉬움
- 제한된 문제들만 해결할 수 있는 방법

2 그리디(Greedy) 문제해결 방법

❖ 동전 거스름돈 문제 - 조합 최적화 문제

- 문제 : 최소 동전의 개수 찾기
- 조건 : 동전의 종류 (500원/100원/50원/10원)
 - 입력 : 거스름돈 액수
 - 출력 : 최소 동전 개수

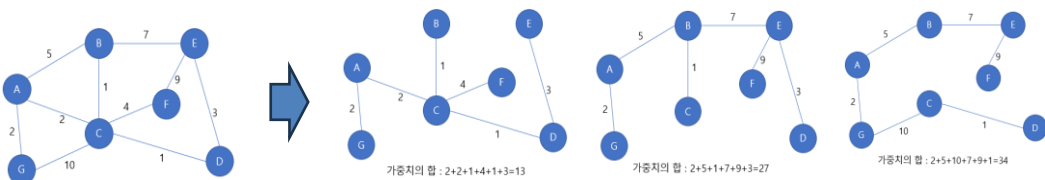
❖ 문제해결 알고리즘

- 거스름돈 액수에서 뺄 수 있는 동전의 종류 중 가장 큰 동전의 종류 액수를 뺀다.
- 뺀 동전의 개수를 증가시킨다.
- 입력 → 670 , 출력 → 5개 (500원 1, 100원 1, 50원 1, 10원 2)

2 그리디(Greedy) 문제해결 방법

❖ 최소 신장트리 문제

- 모든 노드들을 연결시킨 그래프 (사이클 없이) 중 가중치의 합이 가장 작은 그래프 찾기 문제
- 여러 지점을 연결해야 하는 문제에서 최소 구축비용으로 구축하는 문제에 활용
- 통신망, 도로망, 유통망 등



<그래프>

<신장트리>

2 그리디(Greedy) 문제해결 방법

❖ 최소 신장트리 문제

- 모든 노드들을 연결시킨 그래프 중 가중치의 합이 가장 작은 그래프 찾기 문제
- 여러 지점을 연결해야 하는 문제에서 최소 구축비용으로 구축하는 문제에 활용
- 통신망, 도로망, 유통망 등

❖ 크루스컬 알고리즘

1. 가중치를 기준으로 간선을 정렬한다.
2. **가중치가 가장 작은 간선을 선택**한다.
3. 간선이 사이클을 만들는지 확인한다.
 - 3-1. 사이클을 만들면 최소신장트리에 추가하지 않는다.
 - 3-2. 사이클을 만들지 않으면 최소신장트리에 추가한다.
4. 2,3을 반복한다.
5. 최소신장트리를 결과로 출력한다.

❖ 프림 알고리즘

1. 모든 노드의 값을 무한대 값으로 초기화 한다.
2. 그래프에서 임의의 노드를 선택하고 노드의 값을 0으로 변경한다.
3. 선택된 노드와 인접한 노드의 값을 가중치 값으로 변경한다.(중복될 경우 작은 값으로 변경)
4. **값이 가장 작은 노드를 선택하여 최소 신장 트리에 추가**한다.
5. 3,4를 반복한다.
6. 최소신장트리를 결과로 출력한다.

3 분할-정복 문제해결 방법

❖ Divide-Conquer Algorithm

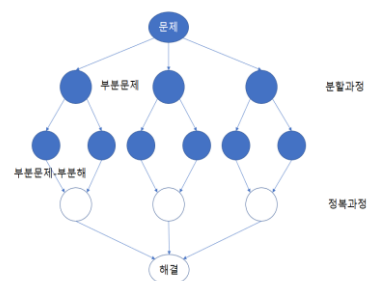
❖ 주어진 문제의 입력 데이터를 작게 줄여서 문제를 분할 하여 정복하는 방법

❖ 분할단계 - 정복 단계

- ❖ 같은 분할 규칙을 적용하여 입력 데이터의 크기가 문제조건상 가장 작을 때 까지 분할
- ❖ 분할 된 데이터들을 문제 조건에 맞게 정복

❖ 부분문제 - 부분 해

- ❖ 같은 해를 구하는 규칙을 적용하여 부분 해를 구한다.
- ❖ 부분 해를 취합(정복) 하여 전체 문제를 해결한다.



3 분할-정복 문제해결 방법

❖ 정렬문제

- 정렬문제에 적용하여 성능을 높임
- 검색문제를 해결하기 위하여 연구된 문제
- 대표적인 P 문제
- 합병정렬 알고리즘, 퀵정렬 알고리즘

3 분할-정복 문제해결 방법

❖ 합병정렬 알고리즘

- Merge Sort Algorithm
- 1945년 존 폰 노이만 (John von Neumann) 이 개발
- 알고리즘

1. 입력된 데이터를 **2개로 분할**한다. (분할)
2. 분할된 데이터들 각각에 대하여 1번을 반복한다.
3. 분할된 데이터들을 **합병**한다. (정복)
4. 정렬된 데이터를 출력한다.

❖ 퀵정렬 알고리즘

- Quick Sort Algorithm
- 찰스 앤토니 리처드 호어 (Charles Antony Richard Hoare) 가 개발
- 알고리즘

1. 피벗 값을 정한다.
2. 피벗 값보다 작은 데이터는 왼쪽으로, 큰 데이터는 오른쪽으로 이동한다. (정복)
3. 피벗의 위치를 피벗보다 작으면서 가장 오른쪽에 있는 데이터의 위치와 교환한다. (분할)
4. 분할된 데이터들에 대해 각각 1,2,3을 반복한다.
5. 정렬된 데이터를 출력한다.

3 분할-정복 문제해결 방법

❖ 퀵정렬 알고리즘

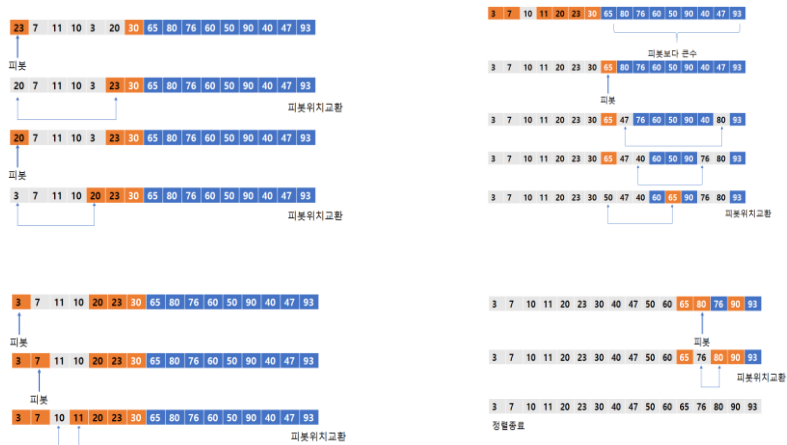
1. 피벗 값을 정한다.
2. 피벗 값보다 작은 데이터는 왼쪽으로, 큰 데이터는 오른쪽으로 이동한다.(정복)
3. 피벗의 위치를 피벗보다 작으면서 가장 오른쪽에 있는 데이터의 위치와 교환한다.(분할)
4. 분할된 데이터들에 대해 각각 1,2,3을 반복한다.
5. 정렬된 데이터를 출력한다.



3 분할-정복 문제해결 방법

❖ 퀵정렬 알고리즘

1. 피벗 값을 정한다.
2. 피벗 값보다 작은 데이터는 왼쪽으로, 큰 데이터는 오른쪽으로 이동한다.(정복)
3. 피벗의 위치를 피벗보다 작으면서 가장 오른쪽에 있는 데이터의 위치와 교환한다.(분할)
4. 분할된 데이터들에 대해 각각 1,2,3을 반복한다.
5. 정렬된 데이터를 출력한다.



3 동적 문제해결 방법

❖ Dynamic Programming Algorithm

❖ 1953년 리처드 벨만 (Richard Ernest Bellman)이 고안한 문제해결 방법

❖ 동적 (Dynamic) : 시간 변적, 다단계적 – 멋져보이는 그럴듯한 용어

❖ 적용되는 문제의 특징

- 반복되는 부분문제가 있는 문제
- 부분문제의 답이 항상 같은 문제 (같은 규칙)
- 부분문제의 답이 큰문제의 답에 영향을 주는 문제

3 동적 문제해결 방법

❖ 동적방법에서 수행하는 세가지 방법W

❖ 메모이제이션 (Memoization)

❖ 재사용 (Re-use)

❖ 상향식 방법 (Bottom-Up)

❖ 재귀적 방법과 차이

❖ 재귀 방법 : 반복을 통해 문제를 해결 할때 자기자신을 불러와 문제를 해결하는 방법

❖ 대표적인 하향식 방법 (Top-Down)

3 동적 문제해결 방법

❖ 피보나치 수 문제

❖ 피보나치 수 : 첫째 와 둘째 항이 1 이며, 그 외의 모든 항은 바로 앞 두 상의 합인 수열

❖ 1,1,2,3,4,5,8,13,21,34,55,89,144,233,377 ...

❖ 입력 : 찾고자 하는 피보나치 수의 순서 (10 : 10번째 피보나치 수)

❖ 출력 : 피보나치 수 (55)

❖ 점화식 :

$$F_n = F_{n-1} + F_{n-2}$$

$$F_8(21) = F_7(13) + F_6(8)$$

$$F_7(13) = F_6(8) + F_5(5)$$

$$F_6(8) = F_5(5) + F_4(3)$$

$$F_5(5) = F_4(3) + F_3(2)$$

$$F_4(3) = F_3(2) + F_2(1)$$

$$F_3(2) = F_2(1) + F_1(1)$$

3 동적 문제해결 방법

❖ 피보나치 수 문제 해결 방법

❖ 점화식을 사용한 문제해결

- 기본 알고리즘을 이용하여 문제해결

❖ 재귀 방법을 이용한 문제해결

- 재귀적으로 자기자신 (점화식)의 결과를 저장하여 문제해결

❖ 동적 방법을 이용하여 문제해결

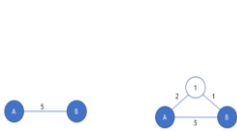
- 부분문제를 정의하고 가장 작은 문제부터 해결하는 Bottom-Up 방식으로 문제해결

3 동적 문제해결 방법

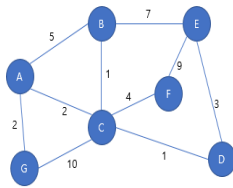
❖ 플로이드 워셜 알고리즘

- 주어진 가중치 그래프에서 주어진 두 지점의 최소비용을 찾아주는 알고리즘
- 알고리즘

- 가중치 그래프에서 각 노드와 간선의 가중치를 저장 (초기화 : 경유지점이 없을 때로 가정 - 인접한 간선이 없으면 무한대)
- 노드 선택 - 경유 노드로 결정하고 모든 노드에 대한 경로 비용을 계산
- 계산된 경로 비용과 이전에 저장된 경로비용중에 더 작은 값을 선택하여 저장
- 2,3을 반복
- 저장된 모든 노드들에 대한 최소비용 리턴



부분문제



그래프

	A	B	C	D	E	F	G
A	0	5	2	∞	∞	∞	2
B	5	0	1	∞	7	∞	∞
C	2	1	0	1	∞	4	10
D	∞	∞	1	0	3	∞	∞
E	∞	7	∞	3	0	9	∞
F	∞	∞	4	∞	9	0	∞
G	2	∞	10	∞	∞	∞	0

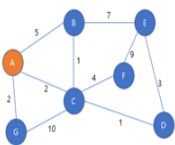
초기화 데이터

	A	B	C	D	E	F	G
A	0	3	2	3	6	6	2
B	3	0	1	2	5	5	5
C	2	1	0	1	4	4	4
D	3	2	1	0	3	5	5
E	6	5	4	3	0	8	8
F	6	5	4	5	8	0	8
G	2	5	4	5	9	8	0

결과 데이터

3 동적 문제해결 방법

❖ 플로이드 워셜 알고리즘

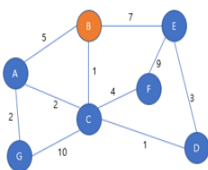


	A	B	C	D	E	F	G
A	0	5	2	∞	∞	∞	2
B	5	0	1	∞	7	∞	∞
C	2	1	0	1	∞	4	10
D	∞	∞	1	0	3	∞	∞
E	∞	7	∞	3	0	9	∞
F	∞	∞	4	∞	9	0	∞
G	2	∞	10	∞	∞	∞	0



	A	B	C	D	E	F	G
A	0	5	2	∞	∞	∞	2
B	5	0	1	∞	7	∞	7
C	2	1	0	1	∞	4	4
D	∞	∞	1	0	3	∞	∞
E	∞	7	∞	3	0	9	∞
F	∞	∞	4	∞	9	0	∞
G	2	7	4	∞	∞	∞	0

(B,C) : $\min \{ (B,A)+(A,C), (B,C) \} = \min \{ 5+2, 1 \} = 1$ (현재값)
 (B,D) : $\min \{ (B,A)+(A,D), (B,D) \} = \min \{ 5+\infty, \infty \} = \infty$ (현재값)
 (B,E) : $\min \{ (B,A)+(A,E), (B,E) \} = \min \{ 5+\infty, 7 \} = 7$ (현재값)
 (B,F) : $\min \{ (B,A)+(A,F), (B,F) \} = \min \{ 5+\infty, \infty \} = \infty$ (현재값)
 (B,G) : $\min \{ (B,A)+(A,G), (B,G) \} = \min \{ 5+2, \infty \} = 7$ (경유 한 값으로 변경)
 (C,B) : $\min \{ (C,A)+(A,B), (C,B) \} = \min \{ 2+5, 1 \} = 1$ (현재값)
 (C,D) : $\min \{ (C,A)+(A,D), (C,D) \} = \min \{ 2+\infty, 1 \} = 1$ (현재값)
 (C,E) : $\min \{ (C,A)+(A,E), (C,E) \} = \min \{ 2+\infty, \infty \} = \infty$ (현재값)
 (C,F) : $\min \{ (C,A)+(A,F), (C,F) \} = \min \{ 2+\infty, 4 \} = 4$ (현재값)
 (C,G) : $\min \{ (C,A)+(A,G), (C,G) \} = \min \{ 2+2, 10 \} = 4$ (경유 한 값으로 변경)



	A	B	C	D	E	F	G
A	0	5	2	∞	∞	∞	2
B	5	0	1	∞	7	∞	7
C	2	1	0	1	∞	4	4
D	∞	∞	1	0	3	∞	∞
E	∞	7	∞	3	0	9	∞
F	∞	∞	4	∞	9	0	∞
G	2	7	4	∞	∞	∞	0



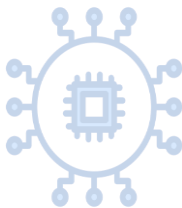
	A	B	C	D	E	F	G
A	0	5	2	∞	12	∞	2
B	5	0	1	∞	7	∞	7
C	2	1	0	1	8	4	4
D	∞	∞	1	0	3	∞	∞
E	14	7	8	3	0	9	14
F	∞	∞	4	∞	9	0	∞
G	2	7	4	14	14	∞	0

(A,C) : $\min \{ (A,B)+(B,C), (A,C) \} = \min \{ 5+1, 2 \} = 2$ (현재값)
 (A,D) : $\min \{ (A,B)+(B,D), (A,D) \} = \min \{ 5+\infty, \infty \} = \infty$ (현재값)
 (A,E) : $\min \{ (A,B)+(B,E), (A,E) \} = \min \{ 5+7, \infty \} = 12$ (경유 한 값으로 변경)
 (A,F) : $\min \{ (A,B)+(B,F), (A,F) \} = \min \{ 5+\infty, \infty \} = \infty$ (현재값)
 (A,G) : $\min \{ (A,B)+(B,G), (A,G) \} = \min \{ 5+7, 2 \} = 2$ (현재값)

4 알고리즘 성능

❖ 문제해결 방법을 이용한 알고리즘의 성능

알고리즘	성능
크러스컬	$n \log_2 n$
프림	$n \log_2 n$
합병 정렬	$n \log_2 n$
퀵 정렬	$n \log_2 n$
플로이드 – 워셜	n^3



SW중심대학사업 교재공동개발

13. 파이썬 소개와 설치



Computational Thinking

0 목차

- 파이썬 소개
- 파이썬 설치 및 실행
- 파이썬 실행 시 주의사항
- 파이썬 실행 프로그램

1 파이썬 소개

- 1991년 귀도 반 로섬(Guido van Rossum)이 개발한 대화형 프로그래밍 언어
- 네덜란드 소프트웨어 엔지니어
- <Monty Python's Flying Circus>에서 따온 이름



[출처]
https://en.wikipedia.org/wiki/Guido_van_Rossum

1 파이썬 소개

- 초보자부터 전문가까지 다양한 사용자를 보유하고 있다
- 동적 타이핑(dynamic typing) 범용 프로그래밍 언어
 - 실행 시간에 자료형을 검사한다
- 다양한 플랫폼에서 사용이 가능하다
- 라이브러리(모듈)가 풍부하다
- 다른 언어로 쓰인 모듈들을 연결할 수 있다
- 다른 프로그래밍 언어에 비해 간결하다

```
>>> print("안녕하세요. 여러분")
안녕하세요. 여러분
>>>
```

파이썬

```
#include <stdio.h>

int main(void) {
    printf("안녕하세요. 여러분");
    return 0;
}
```

C언어

[출처]
<https://ko.wikipedia.org/wiki/%ED%8C%8C%EC%9D%B4%EC%8D%A>
C

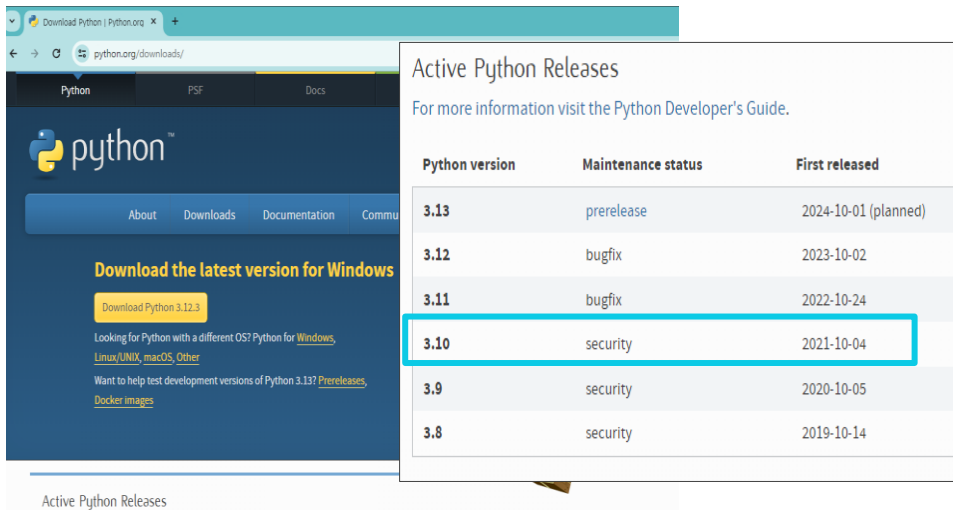
2 파이썬 설치

- 사용할 PC의 OS에 맞는 installer를 설치
- <https://www.python.org/downloads/>



2 파이썬 설치

- <https://www.python.org/downloads/>
- 안정화된 파이썬 버전을 사용하는 것을 추천



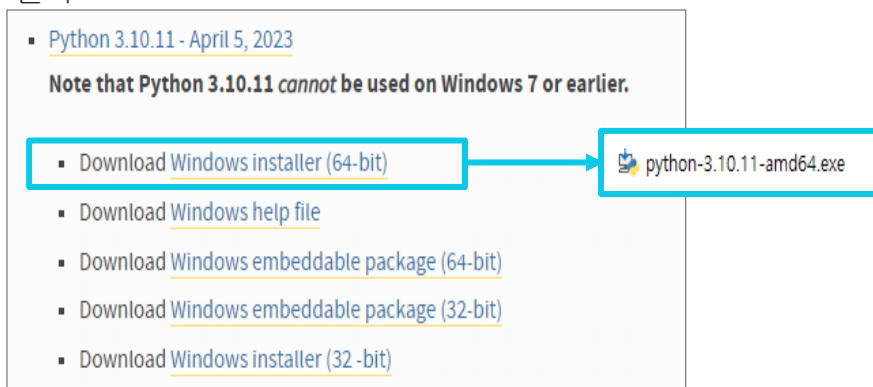
Active Python Releases

For more information visit the [Python Developer's Guide](#).

Python version	Maintenance status	First released
3.13	prerelease	2024-10-01 (planned)
3.12	bugfix	2023-10-02
3.11	bugfix	2022-10-24
3.10	security	2021-10-04
3.9	security	2020-10-05
3.8	security	2019-10-14

2 파이썬 설치

- <https://www.python.org/downloads/windows/>
- 내 PC 우 클릭하여 컴퓨터 bit 확인 후 installer 클릭



Python 3.10.11 - April 5, 2023

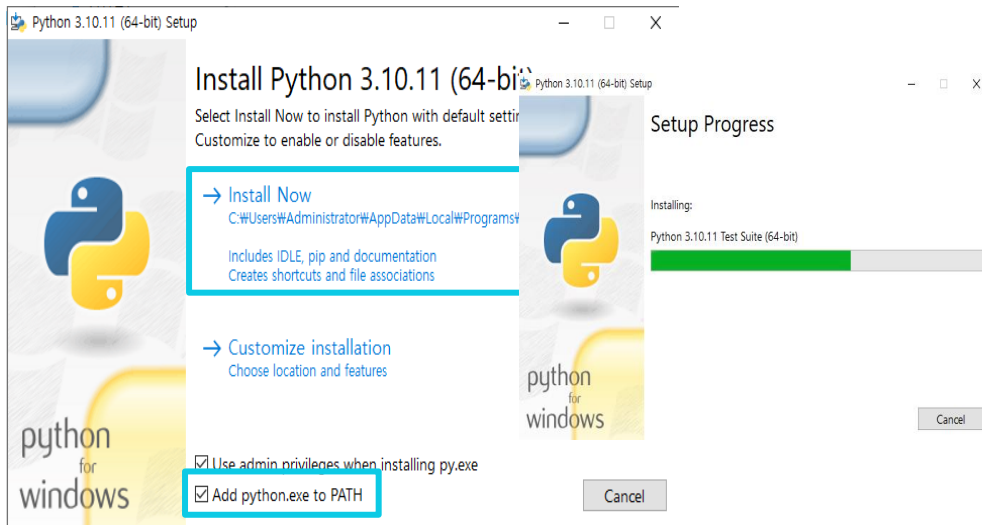
Note that Python 3.10.11 cannot be used on Windows 7 or earlier.

- Download [Windows installer \(64-bit\)](#)
- Download [Windows help file](#)
- Download [Windows embeddable package \(64-bit\)](#)
- Download [Windows embeddable package \(32-bit\)](#)
- Download [Windows installer \(32-bit\)](#)

python-3.10.11-amd64.exe

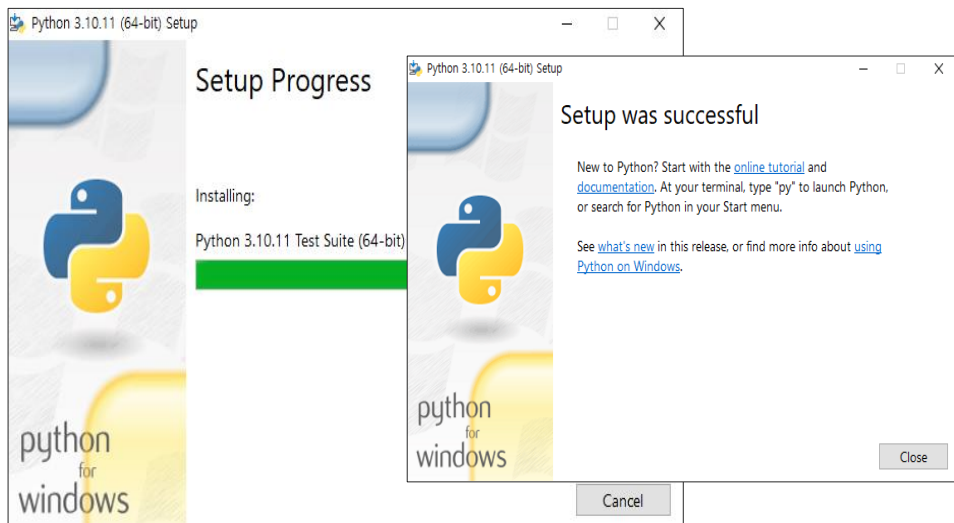
2 파이썬 설치

- 설치 환경 확인 후 진행



2 파이썬 설치

- 설치 환경 확인 후 진행



2 파이썬 실행

- 설치 완료 후 [시작] - [python 3.10] - [IDLE (Python 3.10 64-bit)] 클릭
- Shell창에서는 코드를 실행할 수 있지만 작성한 코드를 저장 할 수 없다

6

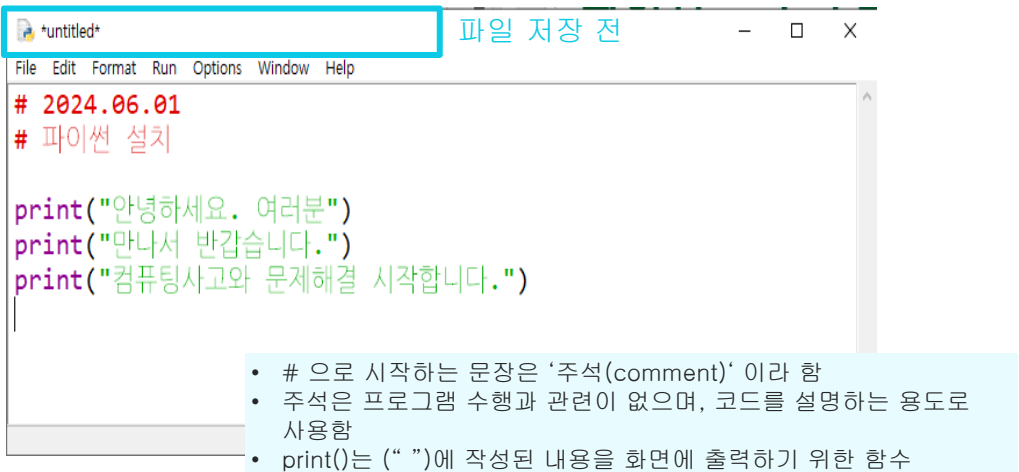
2 파이썬 실행

- 프로그램은 IDLE 편집 창(에디터창)에서 작성한다
- IDLE Shell창에서 [File]-[New File]을 선택하여 편집 창을 실행한다

6

2 파이썬 실행

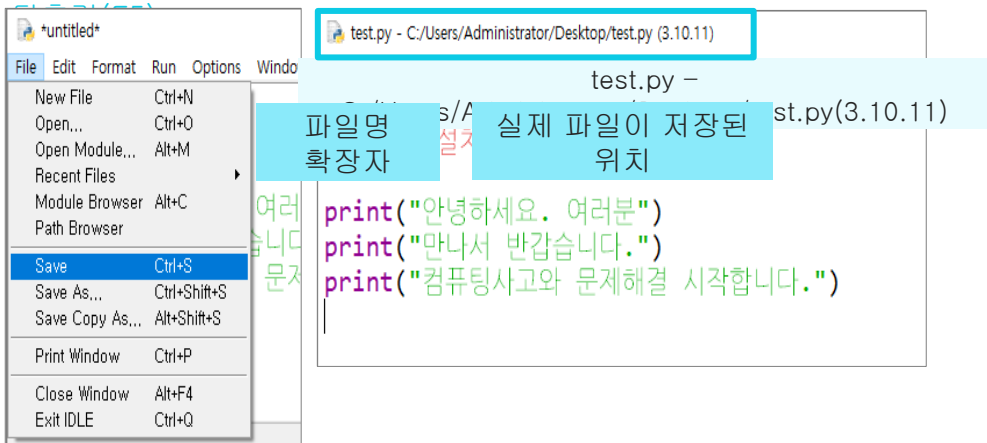
- IDLE 편집창에서 간단한 파이썬 프로그램을 작성해보자



6

2 파이썬 실행

- IDLE 편집창에서 간단한 파이썬 프로그램을 실행해보자
- 파일 저장 : 편집 창 메뉴에서 [File] - [Save] 를 선택, 단축키(ctr + s)
- 파일 실행 : 편집 창 메뉴에서 [Run] - [Run Module] 을 선택,



6

2 파이썬 실행

- 코드가 작성된 대로 실행 결과를 IDLE Shell 창에 표시된다

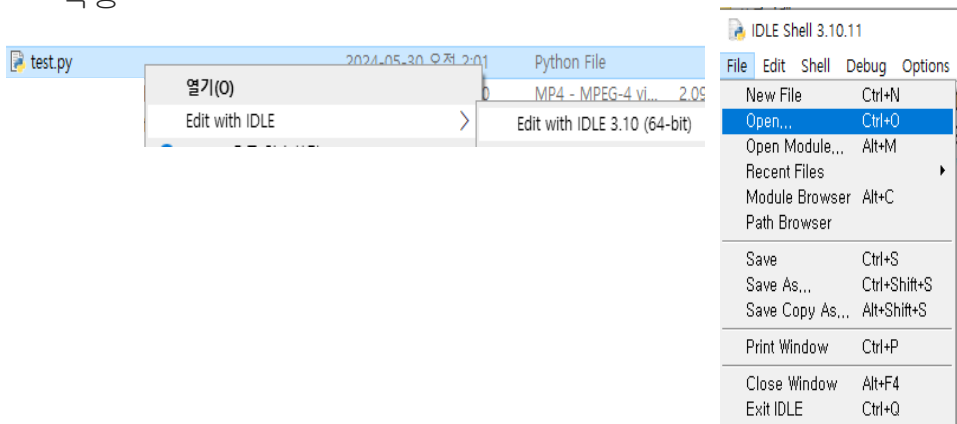
```

Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more
information.
>>>
== RESTART: C:/Users/Administrator/Desktop/test.py ==
안녕하세요. 여러분
만나서 반갑습니다.
컴퓨팅사고와 문제해결 시작합니다.
>>>
>>>
Ln: 9 Col: 0
  
```

6

3 파이썬 실행 시 주의사항

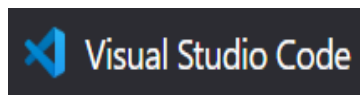
- 코드 수정을 위해 파일을 다시 열어야 하는 경우
 - ✓ py 파일 자체를 더블 클릭하면 작성한 코드가 실행된다
- 소스 코드 보기 위해서
 1. 해당 파일 마우스 우 클릭 edit with IDLE 선택
 2. IDLE창에서 [File] -[Open] 을 통해서 해당 파일을 열어서 작성



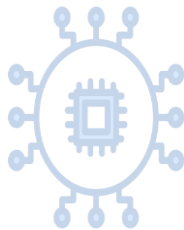
6

4 파이썬 실행 프로그램

- 설치 프로그램 : 아나콘다(스파이더, 주피터), 파이참, 비주얼스튜디오 코드 등
- 온라인 사이트 접속 : 구글 코랩 등



6



SW중심대학사업 교재공동개발

14. 파이썬 변수, 자료형, 표준입출력

Computational Thinking



0 목차

➤ 표준 입출력

➤ 변수

➤ 기본 자료형

1 표준 입출력

- 표준 출력 print() 함수
 - ✓ 큰 따옴표나 작은 따옴표 안에 있는 텍스트 데이터를 출력하기 위해 사용

```
>>> print("Hello world!")    >>> print('Hello world!')
Hello world!                Hello world!
>>>                          >>>
```

- 출력문 작성 형태
 1. 콤마(,)나 덧셈 연산자(+)를 활용한 형태
 2. % 형식 지정자를 활용한 형태
 3. str.format() 함수를 활용한 형태
 4. f-string을 활용한 형태

1 표준 입출력

- 콤마(,)나 덧셈 연산자(+)를 활용한 형태

```
>>> print("성균관대학교", "컴퓨팅사고", "문제해결")
성균관대학교 컴퓨팅사고 문제해결
>>>
>>> print("성균관대학교"+"컴퓨팅사고"+"문제해결")
성균관대학교컴퓨팅사고문제해결
>>>
```

- % 형식 지정자를 활용한 형태

```
>>> print("PI : %d"%(3.1415))
PI : 3
>>> print("PI : %.2f"%(3.1415))
PI : 3.14
>>> print("PI : %.2f, %d"%(3.1415, 3.1415))
PI : 3.14, 3
>>>
```

1 표준 입출력

- str.format() 함수를 활용한 형태

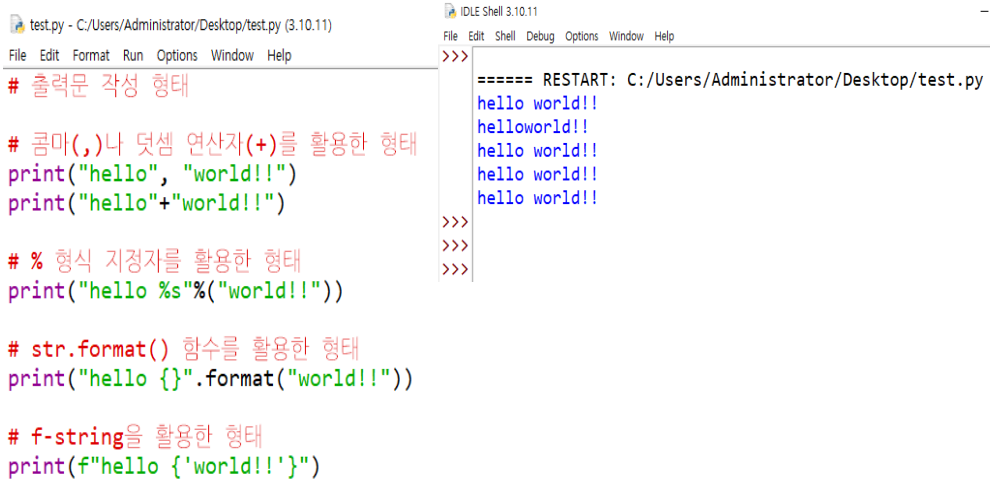
```
>>> print('{0}, {1}, {2}'.format('hello', 'world', '!!'))
hello, world, !!
>>>
>>> print('{} * {} = {}'.format(2, 1, 2*1))
2 * 1 = 2
>>>
```

- f-string을 활용한 형태

```
>>> print(f"{'hello'},{'world'},{'!!'}")
hello,world,!!
>>>
>>> print(f'{2} * {2} = {2*2}')
2 * 2 = 4
>>>
```

1 표준 입출력

• 출력문 작성 형태



```
test.py - C:/Users/Administrator/Desktop/test.py (3.10.11)
File Edit Format Run Options Window Help

# 출력문 작성 형태

# 콤마(,)나 덧셈 연산자(+)를 활용한 형태
print("hello", "world!!")
print("hello"+"world!!")

# % 형식 지정자를 활용한 형태
print("hello %s" % ("world!!"))

# str.format() 함수를 활용한 형태
print("hello {}".format("world!!"))

# f-string을 활용한 형태
print(f"hello {'world!!'}")

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help

>>> ===== RESTART: C:/Users/Administrator/Desktop/test.py
hello world!!
helloworld!!
hello world!!
hello world!!
hello world!!
>>>
>>>
>>>
```

1 표준 입출력

• input() 함수

✓ 입력 받은 값(데이터)을 문자열 형태로 받기 위해 사용

```
>>> strA = input("")
hello world
>>> print(strA)
hello world
>>>

>>> strA = input("input: ")
input: hello world
>>> print(strA)
hello world
>>>
```

1 표준 입출력

- input() 함수
 - ✓ 입력 받은 값(데이터)을 문자열 형태로 받기 위해 사용

```
x = input("input Num#1 : ")
y = input("input Num#2 : ")
sumXY = x + y
print("sumXY : ", sumXY)
```

```
>>>
===== RESTART: C:/Users/
input Num#1 : 10
input Num#2 : 20
sumXY : 1020
>>>
```

```
x = int(input("input Num#1 : "))
y = int(input("input Num#2 : "))
sumXY = x + y
print("sumXY : ", sumXY)
```

```
>>>
===== RESTART: C:/Users/
input Num#1 : 10
input Num#2 : 20
sumXY : 30
>>>
```

2 변수

- 값(데이터)를 저장하기 위해 사용
 - ✓ 한번 저장하고 여러 번 사용하기 위함
 - ✓ 값의 의미를 이름을 통해 전달

```
>>> x = 10
>>> PI = 3.14
>>> str_A = 'hello'
>>>
>>> print(x, PI, str_A)
10 3.14 hello
>>>
```

➤ 변수 이름 작성 규칙

- 알파벳 대소문자, 한글, 숫자, 밑줄(_)만 사용
- 첫 문자는 숫자 작성 못함
- 키워드, 공백 작성 못함
- 대소문자 구별해야 함

2 변수

- [예제] 변수 값 할당하기
 - 사용자로부터 자신의 키를 소수점으로 입력 받는다
 - 자신의 키에 숫자 10을 더한다
 - 결과를 출력한다

```
>>>|===== RESTART: |
>>>|키 입력 : 165.3
>>>|늘어난 자신의 키는 175.30
```

```
height = float(input("키 입력 : "))
print("늘어난 자신의 키는 %.2f"%(height+10))
```

2 변수

- [예제] 변수 값 할당하기
 - 사용자로부터 자신의 학번과 이름을 입력 받는다
 - 결과를 출력한다

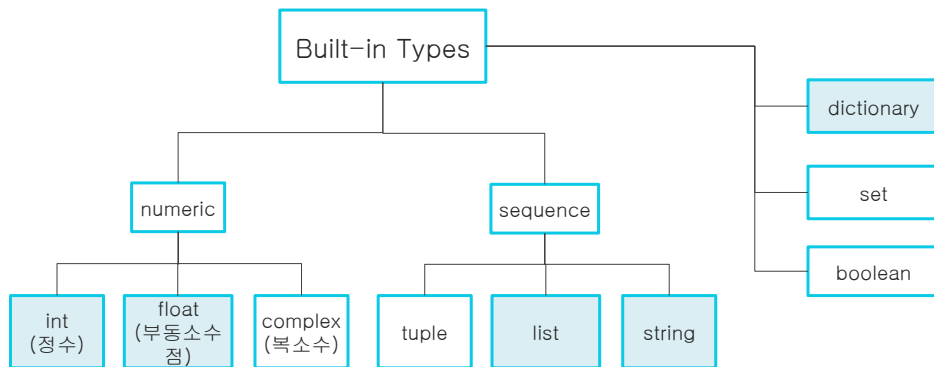
```
>>>|===== RESTART: C:/Users.
>>>|학번 입력: 203012345
>>>|이름 입력: 성균관
>>>|자신의 학번 203012345 성균관
>>>|자신의 학번, 203012345, 성균관
```

```
intStuId = int(input("학번 입력: "))
strName = input('이름 입력: ')

print('자신의 학번', intStuId, strName)
print(f'{"자신의 학번"}', {intStuId}, {strName}')
```

3 기본 자료형

- 값(데이터)이 대입되면 변수의 타입이 결정된다



[출처]
<https://docs.python.org/3/library/stdtypes.html#>

3 기본 자료형

- int
✓ 정수, integer를 뜻함

```
>>> numA = 10
>>> numA
10
>>> numB = 20
>>> numB
20
>>>
>>> numA = numB
>>> numA
20
```

```
>>> numA = 10
>>> numB = numA - 3
>>> numB
7
>>> numB = numA + numA
>>> numB
20
>>> numA = int(input(""))
100
>>> numB = numA * 2
>>> numB
200
...
```

← 사용자가 입력한 값

3 기본 자료형

- float
 - ✓ 부동소수점, 실수를 뜻함

```
>>> numA = 3.14
>>> numA
3.14
>>> numB = 1.912
>>> numB
1.912
>>> numA = numB
>>> numA
1.912

>>> numA = 3.14
>>> numB = numA + 1
>>> numB
4.140000000000001
>>> numB = numA + numA
>>> numB
6.28
>>> numA = float(input(""))
100 ← 사용자가 입력한 값
>>> numA
100.0
>>> numA = numA * 2
>>> numA
200.0
```

3 기본 자료형

- 값 할당 시 주의 사항

```
>>> numA = 10
>>> 12 = numA + 2
SyntaxError: cannot assign to literal here.
>>>
```

```
>>> numA + numB = 10
SyntaxError: cannot assign to expression here.
='?'
>>>
```

```
>>> numC = numC + 1
Traceback (most recent call last):
  File "<pyshell#67>", line 1, in <module>
    numC = numC + 1
NameError: name 'numC' is not defined. Did you mean: 'numA'?
>>>
```

3 기본 자료형

- string
 - ✓ 문자열, 문자들이 나열된 타입을 말함
 - ✓ 큰 따옴표나 작은 따옴표로 감싸면 문자열이 됨

```
>>> strA = 'hello'
>>> strA
'hello'
>>> strB = "hello"
>>> strB
'hello'
>>> print(strB)
hello
>>> strC = strA + strB
>>> strC
'hellohello'
```

[출처]
<https://docs.python.org/3/library/stdtypes.html#>

3 기본 자료형

- string
 - ✓ 문자열, 문자들이 나열된 타입을 말함

test.py - C:/Users/Administrator/Desktop/test.py (3.10.11)

File Edit Format Run Options Window Help

```
# string example

print('안녕하세요?')
name = input('이름이 어떻게 되시나요? ')
print('만나서 반갑습니다. \'\' + name + '\'\' 씨')
print('이름의 길이는 다음과 같군요:', end=' ')
print(len(name))
age = int(input("나이가 어떻게 되나요? "))
print("내년이면", str(age+1), "이 되시는군요.")
```

```
>>> ===== RESTART: C:/Users.
안녕하세요?
이름이 어떻게 되시나요? 조미영
만나서 반갑습니다. '조미영' 씨
이름의 길이는 다음과 같군요: 3
나이가 어떻게 되나요? 34
내년이면 35 이 되시는군요.
```

3 기본 자료형

- string
 - ✓ 문자열은 인덱스를 이용하여 문자를 추출할 수 있음

```
varStr = 'Monty Python'
print("varStr = ", varStr)
print("varStr[0] = ", varStr[0])
print("varStr[1] = ", varStr[1])
print("varStr[1:3] = ", varStr[1:3])
print("varStr[6:] = ", varStr[6:])
print("varStr[:5] = ", varStr[:5])

print("문자열 입력(예시 : [python]).")
varStr = input(">>> ")
print("특수기호 : ", varStr[0], varStr[-1])
print("문자열 : ", varStr[1:-1])
```

```
===== RESTART:
varStr = Monty Python
varStr[0] = M
varStr[1] = o
varStr[1:3] = on
varStr[6:] = Python
varStr[:5] = Monty
>>>

===== RESTART: C:
문자열 입력(예시 : [python]).
>>> {python}
특수기호 : { }
문자열 : python
>>>
```

3 기본 자료형

- string
 - ✓ in 키워드 : 문자열 포함 여부를 확인하기 위함

```
userId = ''      # user input
sysId = 'skku'   # sysId

userId = input("input ID >> ")

print("sysId in userId : ", sysId in userId)
print("sysId not in userId : ", sysId not in userId)
```

```
>>>
===== RESTART: C:/
input ID >> skku python
sysId in userId : True
sysId not in userId : False
>>>

===== RESTART: C:/
input ID >> university sk
sysId in userId : False
sysId not in userId : True
>>>
```

3 기본 자료형

- list
 - ✓ 여러 값(데이터)들을 모아서 저장 및 관리하기 위해 사용한다
 - ✓ 대괄호를 사용하여 생성하고 쉼표로 항목을 구분한다

```
>>>
>>> listStr = ['hello', 'a', 3.14, 10, ['world', 'str']]
>>> listStr
['hello', 'a', 3.14, 10, ['world', 'str']]
>>> print(listStr)
['hello', 'a', 3.14, 10, ['world', 'str']]
>>>
```

[출처]
<https://docs.python.org/3/library/stdtypes.html#list>

3 기본 자료형

- 리스트 사용 방법

```
>>>
===== RESTART: C:/Users/Administrator/Desktop/test.py
exList : []
playList : ['밤', '하늘', '별뿔별', '달밤의 체조']
numList : [10, 3.1415, -1, 203012345]
stuInfo : [203012345, '성균관대학교', ['컴퓨팅사고', 'A+']]

# 빈 리스트 생성
exList = []
print("exList :", exList)

# 리스트 내 문자열로 요소 생성
playList = ['밤', '하늘', '별뿔별', '달밤의 체조']
print("playList :", playList)

# 리스트 내 숫자형(정수, 실수 등) 생성
numList = [10, 3.1415, -1, 203012345]
print("numList :", numList)

# 리스트 내 여러 자료형으로 생성
stuInfo = [203012345, '성균관대학교', ['컴퓨팅사고', 'A+']]
print("stuInfo :", stuInfo)
```

3 기본 자료형

- 리스트의 인덱스

✓ 여러 개 데이터 중 인덱스를 이용하여 원하는 요소에 접근이 가능하다

```
listStr = ['hello', 'a', 3.14, 10, ['world', 'str']]
print("listStr[0] : ", listStr[0])
print("listStr[4] : ", listStr[4])
print("listStr[4][1] : ", listStr[4][1])
print("listStr[5] : ", listStr[5])
```

	[0]	[1]	[2]	[3]	[4]
listStr	'hello'	'a'	3.14	10	['world', 'str']

```
>>>
===== RESTART: C:/Users/Administrator/Desktop/test.py =====
listStr[0] : hello
listStr[4] : ['world', 'str']
listStr[4][1] : str
Traceback (most recent call last):
  File "C:/Users/Administrator/Desktop/test.py", line 68, in <module>
    print("listStr[5] : ", listStr[5])
IndexError: list index out of range
...

```

3 기본 자료형

- 리스트의 요소 변경

```
listStr = ['hello', 'a', 3.14, 10]
print("listStr : ", listStr)

# 리스트 요소 변경
listStr[1] = '!!!'
print("listStr : ", listStr)

# 리스트 + 연산
print("listStr : ", listStr + ['I', 'Love', 'U'])

# in, not in 연산자
print('I' in listStr)
print('Love' not in listStr)
```

```
===== RESTART: C:/Users/Administrator/Desktop
listStr : ['hello', 'a', 3.14, 10]
listStr : ['hello', '!!!', 3.14, 10]
listStr : ['hello', '!!!', 3.14, 10, 'I', 'Love', 'U']
False
True
>>>
```

3 기본 자료형

• 리스트의 요소 관련 함수

함수명	설명	사용법
append()	리스트의 마지막에 요소 추가	리스트명.append(요소)
insert()	리스트에 지정한 인덱스에 요소 삽입	리스트명.insert(인덱스,요소)
remove()	요소 값으로 삭제	리스트명.remove(요소)
pop()	리스트의 마지막 요소 리스트에서 삭제 지정한 인덱스의 요소 삭제	리스트명.pop() 리스트명.pop(인덱스)
sort()	오름차순 정렬 내림차순 정렬	리스트명.sort() 리스트명.sort(reverse=True)
index()	특정 값의 인덱스 반환	리스트명.index(요소)
count()	요소가 몇 개 있는지 확인	리스트명.count(요소)

3 기본 자료형

• dictionary

- ✓ 키와 값을 같이 저장하여 사용하는 타입을 말함
- ✓ 키는 유일해야 하며, 키를 통해 요소를 추가, 삭제, 검색, 변경이 가능함

빈 딕셔너리 생성

```
dicA = { }
```

딕셔너리 생성 및 초기화

```
dicA = {'학번':203012345, '이름':'조미영'}
```

```
print("dicA : ", dicA)
```

딕셔너리 요소 추가

```
dicA['학과'] = '컴퓨터공학'
```

```
print("dicA : ", dicA)
```

딕셔너리 요소 변경

```
dicA.update({'학번':'203011111'})
```

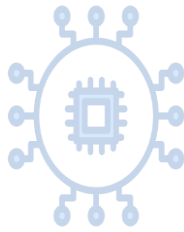
```
print("dicA : ", dicA)
```

딕셔너리 요소 삭제

```
dicA.pop('이름')
```

```
print("dicA : ", dicA)
```

```
===== RESTART: C:\Users\Administrator\Desktop\test.py
dicA : {'학번': 203012345, '이름': '조미영'}
dicA : {'학번': 203012345, '이름': '조미영', '학과': '컴퓨터공학'}
dicA : {'학번': '203011111', '이름': '조미영', '학과': '컴퓨터공학'}
dicA : {'학번': '203011111', '학과': '컴퓨터공학'}
>>>
```

SW중심대학사업 교재공동개발

15. 파이썬 제어문



Computational Thinking

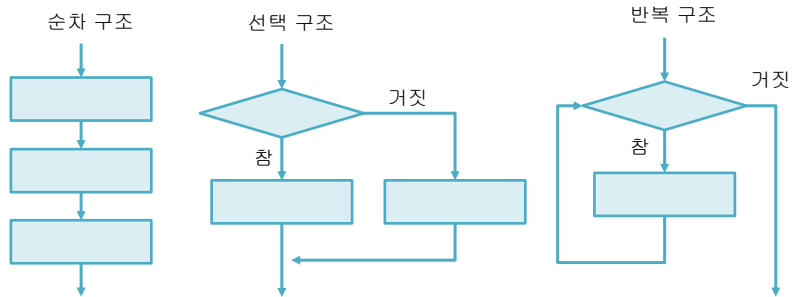
0 목차

➤ 제어문

- 선택문 if문
- 반복문 while문
- 반복문 for문
- break문

1 제어문

- 제어문의 종류
 - ✓ 순차 구조
 - ✓ 선택 구조
 - ✓ 반복 구조



2 선택문 if문

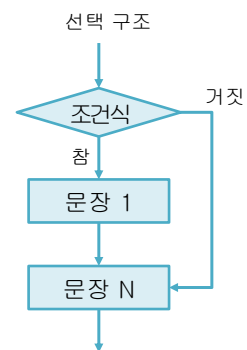
- 선택문 if 문
 - ✓ 조건식의 결과에 따라 실행하는 문장을 선택함

```
speedTest = int(input("speed>> "))

if speedTest > 30:
    print("규정 속도 위반 :", speedTest)

print("안전 운전 해주세요!")
```

```
===== RE
speed>> 29
안전 운전 해주세요!
>>>
===== RE
speed>> 31
규정 속도 위반 : 31
안전 운전 해주세요!
>>>
```



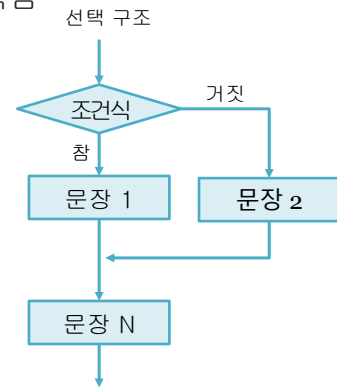
2 선택문 if문

- 선택문 if-else 문
 - ✓ 조건식의 결과에 따라 실행하는 문장을 선택함

```
numA = int(input("정수 입력 : "))
```

```
if numA % 2 == 0:
    print("짝수")
else:
    print("홀수")
```

```
=====
정수 입력 : 10
짝수
>>>
=====
정수 입력 : 11
홀수
>>>
```



2 선택문 if문

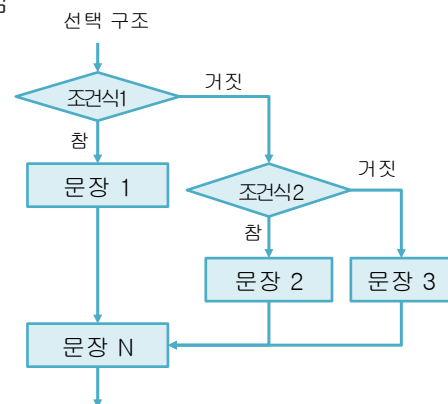
- 선택문 다중 if-elif-else 문
 - ✓ 조건식이 여러 개로 구성된 선택문임

```
numA = int(input("1st : "))
numB = int(input("2nd : "))
```

```
if numA > numB:
    print(numA, "더 크다")
elif numA < numB:
    print(numB, "더 크다")
else:
    print("두 수는 같다")

print("end")
```

```
=====
1st : 10
2nd : 20
20 더 크다
end
>>>
```



3 반복문 while문

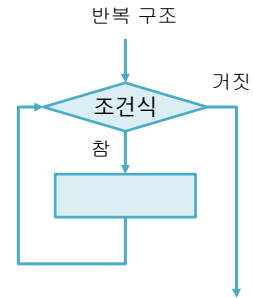
- while 문
 - ✓ 특정 조건이 만족되는 동안 반복함

```
numA = 0
while numA < 5:
    print("python")
    numA = numA + 1

strPWD = ""
while strPWD != "python":
    strPWD = input("암호 입력 :")
    print("로그인 성공")
```

```
=====
python
python
python
python
python
>>>

=====
암호 입력 :py
암호 입력 :pyt
암호 입력 :pytn
암호 입력 :pytho
암호 입력 :pythnn
암호 입력 :python
로그인 성공
>>>
```



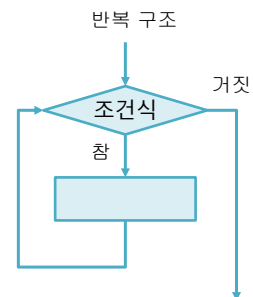
3 반복문 while문

- while 문
 - ✓ 특정 조건이 만족되는 동안 반복함

```
i = numC = 0
while i < 101:
    if i % 2 != 0:
        numC = numC + i
    i += 1

print("홀수의 합 = %d"%numC)
```

```
=====
홀수의 합 = 2500
>>>
```

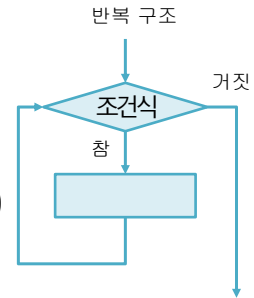


3 반복문 for문

- for 문
 - ✓ 정해진 횟수만큼 반복함

```
for 변수 in 리스트 :
    문장 1
    문장 n
```

- ✓ 리스트
 - 시퀀스 자료형(리스트, range, 문자열, 딕셔너리..)



3 반복문 for문

- for 문
 - ✓ 시퀀스 자료형(리스트, range, 문자열, 딕셔너리..)

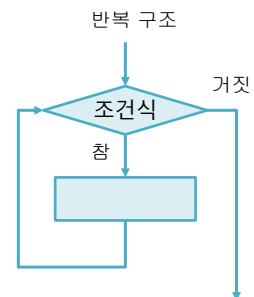
```
for i in [0,0,0,0,0]:
    print("i : ", i)

for strA in 'python' :
    print("string : ", strA)

strB = input("input:")
for i in strB:
    print("string : ", i)

for i in range(4):
    print("i : ", i)
```

```
i : 0
i : 0
i : 0
i : 0
i : 0
string : p
string : y
string : t
string : h
string : o
string : n
input:py
string : p
string : y
i : 0
i : 1
i : 2
i : 3
```



3 반복문 for문

- for 문
 - ✓ 정해진 횟수만큼 반복함

```
i = numC = 0
while i < 101:
    if i % 2 != 0:
        numC = numC + i
    i += 1
print("홀수의 합 = %d"%numC)
```

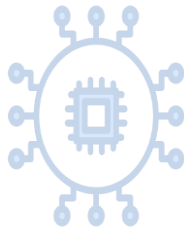
```
numC = 0
for i in range(1, 101):
    if i % 2 != 0:
        numC = numC + i
print("홀수의 합 = %d"%numC)
```

4 break문

- break문
 - ✓ 반복을 제어하기 위한 키워드
 - ✓ 가장 가까운 반복문을 벗어나기 위해 사용함

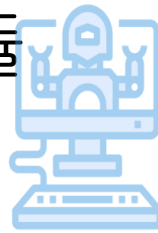
```
while True:
    userInput = input("select>> ")
    if userInput == 'go':
        break
print("END")
```

```
=====
select>> stop
select>> stop
select>> st
select>> g
select>> go
END
>>>
```



SW중심대학사업 교재공동개발

16. 파이썬 함수, 모듈



Computational Thinking

0 목차

➤ 함수

➤ 모듈

1 함수

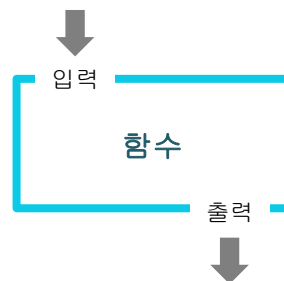
- 함수의 정의
 - 특정 작업을 수행하는 명령어들의 모아 놓은 것을 말함
- 필요성
 - 한번 작성된 코드를 여러 번 사용할 수 있게 함
 - 코드의 기능에 대한 흐름 파악이 용이해짐
- 특징
 - 매개 변수와 리턴 값이 존재한다
 - 매개 변수와 리턴 값은 있을 수도 있고 없을 수도 있다

6

1 함수

- 함수 수행 방법
 - 함수가 호출되면 함수 내부의 문장들이 실행되고 실행이 끝나면 호출한 위치로 되돌아간다
- 함수 종류
 - 내장 함수 : 파이썬에서 기본으로 제공하는 함수
 - 사용자 정의 함수 : 사용자가 필요에 따라 만들어서 사용하는 함수

```
def 함수명 (매개변수) :  
    수행할 문장1  
    ...  
    수행할 문장 n  
    return 반환값
```



1 함수

- 내장 함수
 - 파이썬에서 기본으로 제공하는 함수

Built-in Functions			
A abs() aiter() all() anext() any() ascii()	E enumerate() eval() exec()	L len() list() locals()	R range() repr() reversed() round()
B bin() bool() breakpoint() bytearray() bytes()	F filter() float() format() frozenset()	M map() max() memoryview() min()	S set() setattr() slice() sorted() staticmethod() str() sum() super()
C callable() chr() classmethod() compile() complex()	G getattr() globals()	N next()	T tuple() type()
D delattr() dict() dir() divmod()	H hasattr() hash() help() hex()	O object() oct() open() ord()	V vars()
	I id() input() int() isinstance() issubclass() iter()	P pow() print() property()	Z zip() - __import__()

[출처]
<https://docs.python.org/3/library/functions.html>

1 함수

- 사용자 정의 함수
 - 함수를 정의하기 위해서 def 키워드를 사용함

```
result = 0

def getArea(radius):
    varArea = 3.14 * radius ** 2
    return varArea

result = getArea(10)

print("result : ", result)
```

```
def 함수명 (매개변수) :
    수행할 문장1
    ...
    수행할 문장 n
    return 반환값
```

2 모듈

- 모듈
 - 특정 기능을 가진 작은 프로그램을 말함
 - 미리 만들어진 다양한 함수의 집합을 말함
 - 이미 만들어진 모듈을 사용해도 되고 직접 만들어서 사용해도 됨
 - import 키워드를 작성하여 사용함
- 많이 사용하는 모듈
 - sys, random, time, calendar, turtle, math, webbrowser, tkinter ...

2 모듈

- 키워드 from, import, as 를 통해 모듈을 사용함

import 모듈명 ➤ 모듈에 있는 모든 내용을 포함시킴

import 모듈명 **as** 별칭 ➤ 모듈명의 별칭을 as 키워드를 통해 사용함

from 모듈명 **import** 함수명 ➤ 모듈 이름을 생략하고 함수를 사용할 수 있음

from 모듈명 **import** * ➤ * 키워드를 사용하여 모든 함수를 사용할 수 있음

2 모듈

- 키워드 from, import, as 를 통해 모듈을 사용함

```
import webbrowser

num = int(input("점수 입력: "))
if num >= 80:
    webbrowser.open("https://www.google.com")
else:
    webbrowser.open("https://icampus.skku.edu")
```

2 모듈

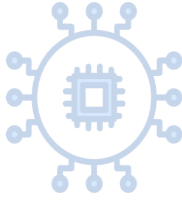
- 키워드 from, import, as 를 통해 모듈을 사용함

```
import random as rd
from time import sleep

for i in range(6):
    print(rd.randrange(1,45))
    print("당첨 되길 바랍니다")

    print("간단한 369 게임을 진행해보자")
    for i in range(1, 31):
        if i%3 == 0:
            print("짝!", end=" ")
        else:
            print(i, end=" ")

        sleep(0.5)
```



SW중심대학사업 교재공동개발

17. 파이썬 클래스, 라이브러리



Computational Thinking

0 목차

- 클래스
- 라이브러리

1 클래스

- 정의
 - ✓ 데이터와 기능을 묶는 방법을 제공함
 - ✓ 기본 자료형이 아닌 새로운 형태를 만들어서 사용하고자 함
- 객체의 이해
 - ✓ 클래스를 통해 만들어지는 것을 객체(object 혹은 instance)라 함

[출처]
<https://docs.python.org/ko/3/tutorial/classes.html>

6

1 클래스

- 작성 방법

```
class 클래스명 :  
    def __init__(self, ... ) :  
        ...  
    def 메소드1(self, ... ) :  
        ...
```

```
class pointXY:  
    def __init__(self):  
        self.x = 0  
        self.y = 0  
    def increment(self):  
        self.x = self.x + 1
```

```
pA = pointXY()  
pA.increment()  
print("pointXY x = ", pA.x)  
print("pointXY y = ", pA.y)
```

[출처]
<https://docs.python.org/ko/3/tutorial/classes.html>

6

1 클래스

- 작성 방법

```
class animal:
    kind = 'pet'

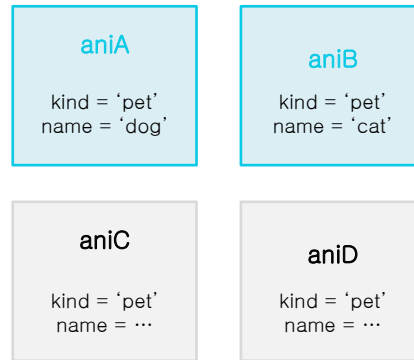
    def __init__(self, name):
        self.name = name

aniA = animal('dog')
aniB = animal('cat')

print("aniA.kind : ", aniA.kind)
print("aniB.kind : ", aniB.kind)

print("aniA.name : ", aniA.name)
print("aniB.name : ", aniB.name)
```

[출처]
<https://docs.python.org/ko/3/tutorial/classes.html>

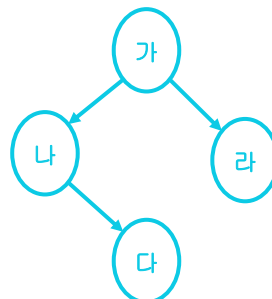


6

1 클래스

- 사용 예)
 - 그래프, 트리 등 자료구조에서 노드로 정의

```
class TreeNode():
    def __init__(self):
        self.left = None
        self.data = None
        self.right = None
```

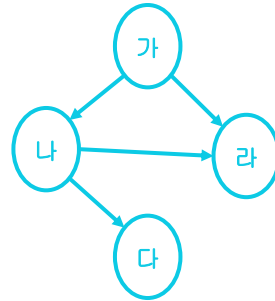


[출처]
<https://docs.python.org/ko/3/tutorial/classes.html>

6

1 클래스

- 사용 예)
 - 그래프, 트리 등 자료구조에서 노드로 정의



```
class Graph():
    def __init__(self, size):
        self.SIZE = size
        self.graph = [[0 for _ in range(size)] for _ in range(size)]

    def __gPrint__(self):
        for row in range(graphSize) :
            for col in range(graphSize) :
                print(self.graph[row][col], end=' ')
            print()
```

[출처]
<https://docs.python.org/ko/3/tutorial/classes.html>

6

2 라이브러리

- 파이썬의 표준 라이브러리

라이브러리명		설명
운영체제 인터페이스	OS	운영 체제와 상호 작용하기 위해 제공
파일 와일드카드	glob	디렉터리 와일드카드 검색으로 파일 목록을 생성
명령행 인자	argparse	명령 줄 인자를 처리하는 기능을 제공
문자열 패턴 매칭	re	문자열 처리를 위한 정규식을 제공
수학	math	부동 소수점 연산을 제공
	random	무작위 숫자를 제공
	statistics	기본적인 통계적 특성 제공
날짜와 시간	datetime	날짜와 시간을 제공

10. 표준 라이브러리 둘러보기

- 10.1. 운영 체제 인터페이스
- 10.2. 파일 와일드카드
- 10.3. 명령행 인자
- 10.4. 에러 출력 리디렉션과 프로그램 종료
- 10.5. 문자열 패턴 매칭
- 10.6. 수학
- 10.7. 인터넷 액세스
- 10.8. 날짜와 시간
- 10.9. 데이터 압축
- 10.10. 성능 측정
- 10.11. 품질 관리
- 10.12. 배터리가 포함됩니다

[출처]
<https://docs.python.org/ko/3/tutorial/stdlib.html#>

6

2 라이브러리

- 파이썬의 대표적 라이브러리

구분	라이브러리	설명
데이터 분석 및 과학 계산	Numpy	수치 계산
	Pandas	데이터 관리 및 분석
	SciPy	과학 및 기술 계산
머신러닝 및 인공지능	Scikit-learn	머신러닝
	TensorFlow	딥러닝 프레임워크(구글)
	PyTorch	딥러닝 프레임워크(facebook)
데이터 시각화	Matplotlib	기본적인 그래프 및 플롯 생성
	Seaborn	통계적 데이터 시각화 제공
	Plotly	인터랙티브한 그래프 생성
웹 개발	Django	풀스택 웹 프레임워크
	Flask	마이크로 웹 프레임워크
자동화 및 스크래핑	Requests	HTTP 요청을 쉽게 만들 수 있음
	Beautiful Soup	HTML 및 XML 파일에서 데이터 추출
	Selenium	웹 브라우저 자동화 도구
이미지 처리	Pillow	이미지 처리
	OpenCV-Python	컴퓨터 비전

6

2 라이브러리

- numpy : 수치 계산과 배열 처리를 위해 사용됨

```
import numpy as np
```

```
# 2차원 배열 생성
```

```
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

```
# 배열 출력
```

```
print("원본 배열:")
```

```
print(arr)
```

```
# 배열의 형태(shape) 출력
```

```
print("\n배열의 형태:", arr.shape)
```

```
# 배열의 평균 계산
```

```
print("\n배열의 평균:", np.mean(arr))
```

원본 배열:

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

배열의 형태: (3, 3)

배열의 평균: 5.0

6

2 라이브러리

- pandas : 구조화된 데이터의 처리와 분석을 위해 사용됨

```
import pandas as pd
```

```
# 데이터프레임 생성
```

```
data = {
    '이름': ['김철수', '이영희', '박민수', '정수진'],
    '나이': [25, 28, 32, 29],
    '직업': ['개발자', '디자이너', '마케터', '교사'],
    '급여': [3000000, 2800000, 3500000, 2700000]
}
```

```
df = pd.DataFrame(data)
```

```
# 데이터프레임 출력
```

```
print("원본 데이터프레임:")
print(df)
```

```
# 통계 요약
```

```
print("\n통계 요약:")
print(df.describe())
```

```
# 조건에 따른 필터링
```

```
print("\n30세 이상인 사람:")
print(df[df['나이'] >= 30])
```

```
# 그룹화 및 평균 계산
```

```
print("\n직업별 평균 나이와 급여:")
print(df.groupby('직업')[['나이', '급여']].mean())
```

```
원본 데이터프레임:
   이름  나이  직업  급여
0 김철수  25  개발자  3000000
1 이영희  28  디자이너  2800000
2 박민수  32  마케터  3500000
3 정수진  29  교사  2700000
```

```
통계 요약:
   count  나이  급여
mean  28.500000  3.000000e+06
std    2.886751  3.559026e+05
min    25.000000  2.700000e+06
25%    27.250000  2.775000e+06
50%    28.500000  2.900000e+06
75%    29.750000  3.125000e+06
max    32.000000  3.500000e+06
```

```
30세 이상인 사람:
   이름  나이  직업  급여
2 박민수  32  마케터  3500000
```

```
직업별 평균 나이와 급여:
   직업  나이  급여
개발자  25.0  3000000.0
교사    29.0  2700000.0
디자이너 28.0  2800000.0
마케터  32.0  3500000.0
```

6

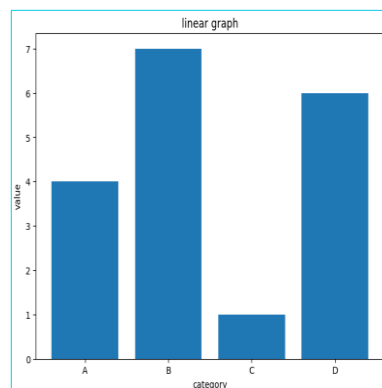
2 라이브러리

- matplotlib : numpy 라이브러리를 활용한 플로팅 라이브러리

```
import matplotlib.pyplot as plt
```

```
categories = ['A', 'B', 'C', 'D']
values = [4, 7, 1, 6]
```

```
plt.figure(figsize=(8, 6))
plt.bar(categories, values)
plt.title('linear graph')
plt.xlabel('category')
plt.ylabel('value')
plt.show()
```



6

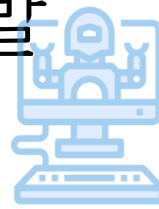
성균관대학교 소프트웨어융합대학
SW학과 황속희교수
건국대학교 공학교육연구소
조미영교수



SW중심대학사업 교재공동개발

문제해결 프로젝트 개발

(별책부록)



0 목차

- 프로젝트의 목적
- 문제해결 소프트웨어 구현 과정
- 각 단계에 대한 내용과 주의사항

1 프로젝트의 목적

개요

- ❖ ‘컴퓨팅 사고와 문제해결’ 과목을 통해 학습한 문제해결 방식으로 문제를 해결해 본다.
- ❖ 문제해결 결과인 알고리즘을 파이썬 프로그래밍 언어로 표현 하여 컴퓨터로 문제해결 결과를 확인해 본다.

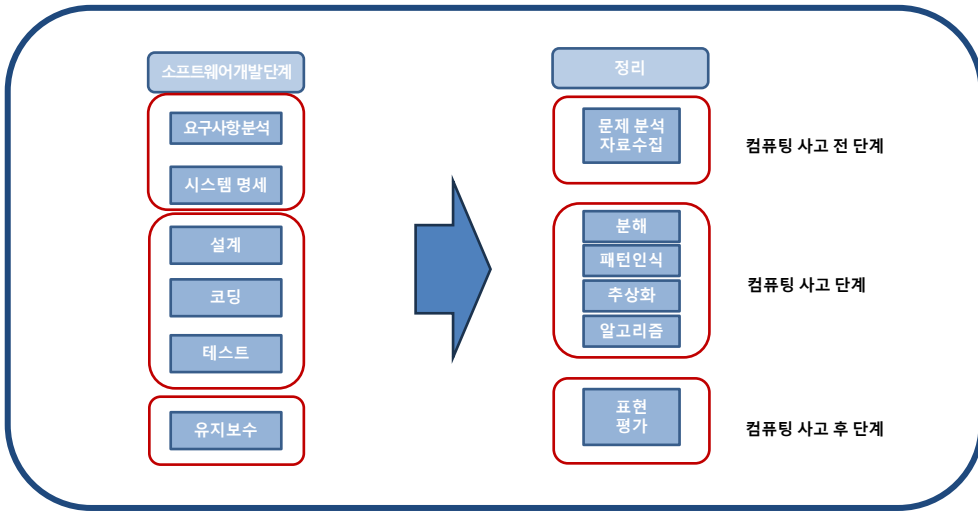
1 프로젝트의 목적

개요

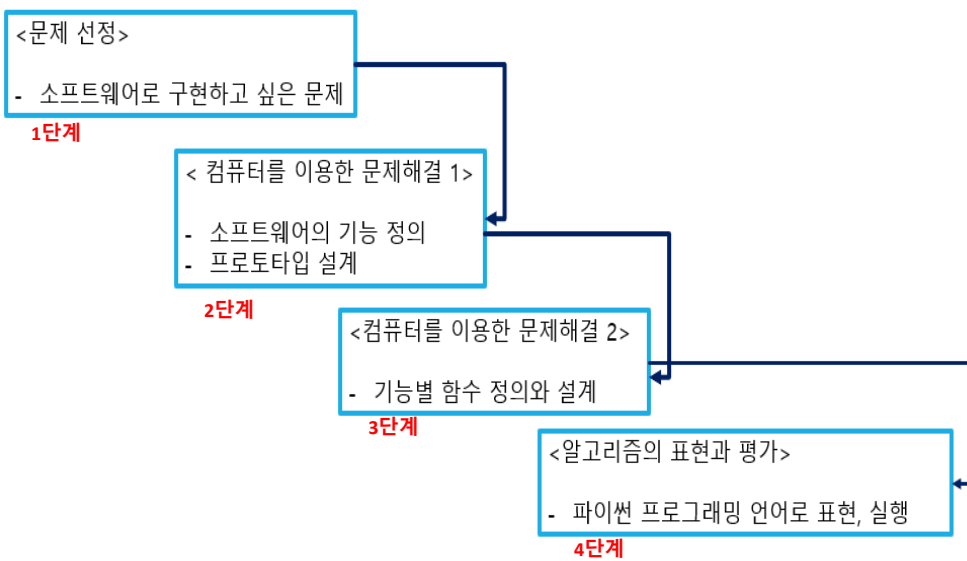
- ❖ 컴퓨터 과학자의 문제해결 방식에 대한 직접 경험을 통하여 컴퓨팅 사고력을 향상시킨다.
- ❖ 컴퓨터 과학자의 문제해결 방식을 간소화 하여 비 전공 학생들이 쉽게 이해하고 경험할 수 있도록 한다.
- ❖ 팀프로젝트로 진행하여 협업의 중요성을 인식하고, 각자의 역할에 대한 책임감을 높인다.

2 문제해결 소프트웨어 구현 과정

❖ 소프트웨어 개발 단계의 간소화



2 문제해결 소프트웨어 구현 과정



3 각 단계에 대한 내용과 주의사항

1단계 - 컴퓨팅 사고 전 단계 - 문제정의 단계

❖ 내용

- 팀원 보고 및 문제 선정 (팀당 4명 이하)
- 문제 선정 과정 (팀원 모두가 함께 고민한 흔적 남기기 - 보고서 형식)
- 팀원 별 역할 과 해야 할 일 작성 (역할에 대한 중요도는 모두 같음)

팀원 선정

2-4명

팀 이름 / 팀원 이름/ 소속 학과 / 학년 / 학번 / 팀장선정

문제 선정

문제 선정 과정 정리 - 회의록 (자유형식)

문제 해결을 위한 역할분담

각 팀원 별 해야 할 일 정리 (자유형식)

3 각 단계에 대한 내용과 주의사항

2단계 - 컴퓨팅 사고 단계 - 컴퓨터를 이용한 문제해결 단계

❖ 내용

- 소프트웨어 기능 정의
 - 기능의 제한성 분석 및 정리
 - 하드웨어적 한계와 개발 기술력에 대한 한계를 이해하고, 구현 가능한 기능과 구현 불가능한 기능을

정리

- 프로토타입 설계
 - 가상의 소프트웨어 완성 형태를 보여줌
 - 정리된 기능을 바탕으로 가상의 소프트웨어를 그림으로 표현
- 회의록
 - 회의 날짜, 참가자, 회의 내용

3 각 단계에 대한 내용과 주의사항

3단계 - 컴퓨팅 사고 단계 - 컴퓨터를 이용한 문제해결 단계

❖ 정리

- 프로그램 계획서 작성
- 2단계에서 결정한 기능별 프로그램 개발 계획 작성

❖ 내용

- 기능별 데이터 설계
 - 저장 데이터 설계 (자료형, 저장방식 정의)
 - 입력 데이터 설계 (자료형, 저장방식 정의)
- UI 설계
 - GUI / TUI 결정
 - 사용할 함수(모듈, 라이브러리 포함) 정의
- 기능별 함수 설계
 - 사용자 정의 함수, 모듈 라이브러리 함수 모두 작성
 - 각 함수의 기능, 함수의 입력 값, 출력 값 정리

3 각 단계에 대한 내용과 주의사항

4단계 - 컴퓨팅 사고 후 단계 - 알고리즘 표현 및 실행

❖ 내용

- 프로그래밍 언어로 표현하기
- 컴퓨터로 실제 실행 시켜보고, 결과 확인하기
- 문제해결 결과와 처음 예상했던 결과에 대해 비교 분석하기
- 결과물에 대한 유지보수 전략 세우기

❖ 제출물

- 소스코드, 실행 결과 화면, 전체 프로젝트 결과 보고서

컴퓨팅 사고와 문제해결

발행일 2024년 12월 15일

저 자 황숙희, 조미영

발 행 홍기표

펴낸곳 글통

등록일 2011년 4월 4일 (제 319-2011-18호)

팩 스 0260040276

페이스북 <https://www.facebook.com/Geultong>

이메일 geultong@daum.net

ISBN 979-11-850

값: 비매품

- 본 교재는 2024년도 과학기술정보통신부 및 정보통신기획평가원의 ‘SW중심대학사업’ 지원을 받아 제작되었습니다.
- 본 결과물의 내용을 전재할 수 없으며, 인용(재사용)할 때에는 반드시 과학기술정보통신부와 정보통신기획평가원이 지원한 ‘SW중심대학’의 결과물이라는 출처를 밝혀야 합니다.